VASTRA

Manual Setup, API Keys & SQL Deployment Runbook

Exactly what must be created, copied, configured, tested and deployed by hand

Version 1.0
Prepared 11 July 2026
Supabase/PostgreSQL implementation blueprint

Customer App • Merchant App • Captain App • Admin & Operations

How this runbook is organized

Item	Details
Part A	Accounts, API keys and information you must collect
Part B	Supabase project, Auth, Storage, Realtime and database setup
Part C	App/backend environment variables and external providers
Part D	Migration, deployment, testing, launch and maintenance
Appendix	Complete numbered SQL migration queries matching the downloadable ZIP pack

Recommended operating model

Create separate development/staging and production Supabase projects. Store the schema as versioned migration files in Git. Once migration workflow begins, do not make ad-hoc schema changes directly on the production dashboard.

Part A - What you must create or provide

1. Supabase information checklist

Item	Details
Supabase organization/account	Owner account and team members with least-privilege dashboard access.
Project reference	Found in the project dashboard URL; required by CLI and CI.
Database password	Chosen during project creation. Store in a password manager; needed for CLI/direct DB connections.
Project URL	Used by all applications and backend.
Publishable key	Safe for client apps only when RLS is correct. Prefer current sb_publishable_... keys.
Secret key	Backend only; bypasses RLS. Prefer current sb_secret_... key and never ship it in an app.
Personal access token	Used by Supabase CLI/CI, stored as SUPABASE_ACCESS_TOKEN.
Database connection string	Backend/migrations only; choose pooler/direct form appropriate to the runtime.
Region	Choose the closest supported region to the first operating city and legal/business needs.

2. External keys and accounts

Item	Details
FCM / Firebase	Project ID, service-account client email/private key, Android google-services.json; APNs credentials later for iOS.
Payment gateway	Cashfree or Razorpay test/live application ID, secret, webhook signing secret and allowed callback URLs.
Maps/routing	Google Maps, Ola Maps, Mapbox or selected provider keys; separate public mobile key and restricted server key.
SMS/OTP	Supabase phone provider credentials or MSG91 account/auth key/template IDs.
Email	SMTP provider if sending transactional email outside built-in defaults.
Monitoring	Sentry/observability DSN and alert destinations.
AI later	Image matching, body measurement or virtual try-on provider credentials only when those modules are enabled.

Item	Details
Encryption	A strong application-level encryption key for especially sensitive values, stored in a secrets manager/Vault.

Never paste into frontend code

SUPABASE_SECRET_KEY, service-role key, DB password/URL, payment secret, webhook secret, FCM private key, SMS auth key, application encryption key, AI provider secret or Supabase access token.

3. Environment variable inventory

Item	Details
All mobile apps	EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY, EXPO_PUBLIC_API_BASE_URL, public maps key, environment name.
Admin web	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, NEXT_PUBLIC_API_BASE_URL.
Backend/worker	SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, SUPABASE_SECRET_KEY, SUPABASE_DB_URL/PASSWORD, FCM, payment, maps server, SMS, encryption and monitoring secrets.
CI/CD	SUPABASE_ACCESS_TOKEN, SUPABASE_PROJECT_ID, SUPABASE_DB_PASSWORD.
Edge Functions	Default Supabase URL/key dictionaries are injected; add payment, FCM, maps, SMS and webhook secrets under Edge Function Secrets.

Part B - Supabase project setup

4. Create projects and local repository

1. Create at least a staging project and a production project. A local Supabase stack is used for daily development.
2. Record project references, URLs, database passwords and API keys in an approved password/secrets manager.
3. Install Node.js and the Supabase CLI; initialize the repository with supabase init.
4. Log in with supabase login and link the staging project using supabase link --project-ref <project-ref>.
5. If the remote project is new, do not run db pull. If it already contains changes, run supabase db pull once and review the generated migration.
6. Copy the numbered migration SQL from the ZIP into supabase/migrations using timestamped filenames, or generate equivalent migrations through the CLI.
7. Start locally and run supabase db reset. Fix every SQL/RLS problem before any remote push.

Core CLI commands

```
npm install -g supabase
# or use npx supabase for each command

supabase init
supabase start
supabase login
supabase link --project-ref YOUR_PROJECT_REF
supabase db reset
supabase db push --dry-run
supabase db push
supabase gen types typescript --linked > packages/types/database.types.ts
```

5. Dashboard settings you must review

Item	Details
Settings > API Keys	Create/use publishable and secret keys. Name backend keys by service where useful so they can rotate independently.
Authentication > Providers	Enable phone/password/social providers actually used by each app. Add provider credentials.
Authentication > URL Configuration	Set production site URL, customer/merchant/captain/admin deep-link redirect URLs and localhost/tunnel test URLs.

Item	Details
Authentication > Rate Limits/CAPTCHA	Protect OTP and sign-up endpoints from abuse.
Database > Extensions	Enable pgcrypto, postgis, citext, pg_trgm and vector as required by the migrations.
Storage	Confirm buckets, public/private state, file-size limits and MIME restrictions after migration.
Realtime	Confirm selected tables are in supabase_realtime publication and RLS policies permit intended subscriptions.
Edge Functions > Secrets	Add payment, FCM, maps, SMS, encryption and provider secrets.
Logs/Reports	Review Auth, Postgres, Realtime, Storage and Function logs during testing.
Backups/PITR	Choose a plan and recovery objective appropriate to production orders/payments.

6. Authentication setup

- Customer: phone OTP is the simplest launch path. Merchant and captain can also use phone OTP, but their account_type changes only after onboarding approval.
- Admin: use email/password or SSO plus MFA/2FA. Never create public admin self-registration.
- The auth.users trigger creates public.profiles with account_type CUSTOMER regardless of user-supplied metadata.
- Trusted backend/admin workflow changes a profile to MERCHANT/CAPTAIN/ADMIN after verification and writes an audit log.
- Configure mobile deep links for OTP/OAuth callbacks and test expired/invalid sessions on physical devices.

7. Storage setup

Item	Details
Public buckets	product-images and shop-images. Public read, controlled merchant upload path and moderation.
Private KYC	merchant-documents and captain-documents. Access through owner/admin policy or short-lived signed URL.
Private evidence	return-evidence and support-attachments.
Highly private/temporary	body-scans and virtual-tryon. Add automated retention deletion jobs.
Path convention	<auth-user-id>/<entity-id>/<random-filename.ext>; validate entity ownership in backend before storing DB references.
Upload validation	MIME type, extension, byte size, image dimensions, malware/content checks where applicable.

8. Realtime, push and merchant order sound

8. Subscribe to orders, order_status_history, merchant_order_alerts, delivery_tasks/events, notifications and support_messages only for rows allowed by RLS.
9. When Merchant App is foregrounded, Realtime opens the incoming-order screen, loops the Vastra order sound and vibrates.
10. When backgrounded/terminated, the trusted backend sends a high-priority FCM/APNs notification using a dedicated New Orders notification channel.
11. Create one merchant_order_alerts row per alert cycle; retry until acknowledged or expired. Store provider failures for support.
12. The merchant dashboard must include notification permission, sound channel and battery-optimization checks plus a Test Ringtone action.
13. Do not promise ringing through every iOS silent/Do Not Disturb setting without the required Apple entitlement.

Part C - App and provider configuration

9. Environment templates

.env.customer.example

```
EXPO_PUBLIC_SUPABASE_URL=
EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
EXPO_PUBLIC_API_BASE_URL=
EXPO_PUBLIC_MAPS_PROVIDER_KEY=
EXPO_PUBLIC_ENVIRONMENT=development
```

.env.merchant.example

```
EXPO_PUBLIC_SUPABASE_URL=
EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
EXPO_PUBLIC_API_BASE_URL=
EXPO_PUBLIC_MAPS_PROVIDER_KEY=
EXPO_PUBLIC_ENVIRONMENT=development
```

.env.captain.example

```
EXPO_PUBLIC_SUPABASE_URL=
EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
EXPO_PUBLIC_API_BASE_URL=
EXPO_PUBLIC_MAPS_PROVIDER_KEY=
EXPO_PUBLIC_ENVIRONMENT=development
```

.env.admin.example

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
NEXT_PUBLIC_API_BASE_URL=
NEXT_PUBLIC_ENVIRONMENT=development
```

.env.backend.example

```
NODE_ENV=development
PORT=8080
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SECRET_KEY=
SUPABASE_DB_URL=
SUPABASE_DB_PASSWORD=
FCM_PROJECT_ID=
FCM_CLIENT_EMAIL=
FCM_PRIVATE_KEY=
CASHFREE_APP_ID=
CASHFREE_SECRET_KEY=
CASHFREE_WEBHOOK_SECRET=
MAPS_SERVER_KEY=
MSG91_AUTH_KEY=
MSG91_TEMPLATE_ID=
APP_ENCRYPTION_KEY=
SENTRY_DSN=
```

.env.ci.example

```
SUPABASE_ACCESS_TOKEN=
SUPABASE_PROJECT_ID=
SUPABASE_DB_PASSWORD=
```

Git rule

Commit only .env.example templates. Add .env, .env.local, service-account JSON, certificates and private keys to .gitignore. Rotate any key that is accidentally committed.

10. Payment configuration

- Create separate sandbox and live credentials. Never use the client callback alone as proof of payment.
- Create payment order from trusted backend, then verify provider signature/webhook server-side.
- Store every provider webhook in payment_events using provider_event_id as an idempotency key.
- Use an externally reachable HTTPS webhook endpoint; verify the exact raw body/signature required by the provider.
- Refunds and settlement adjustments above configured limits create approval_requests.
- Keep gateway IDs and status history; never store full card data.

11. Maps, SMS and notifications

Item	Details
Mobile maps key	Restrict by Android package/SHA and iOS bundle identifier. Enable only required SDKs.
Server maps key	Restrict by server IP/service and required routing/geocoding APIs.

Item	Details
SMS templates	Register OTP/transactional templates and sender IDs under provider and Indian regulatory requirements.
FCM service account	Store private key with newline escaping correctly in secrets; do not place service-account JSON inside the app bundle.
Push token lifecycle	Refresh user_devices token on app start/login/token refresh; revoke on logout/uninstall failure feedback.

Part D - Deployment and operation

12. Migration order

Migration	Purpose
0001_extensions_helpers.sql	Extensions, private helpers and auth profile trigger
0002_identity_access.sql	Profiles, devices, addresses and RBAC
0003_marketplace_catalog_inventory.sql	Shops, catalogue, inventory, favourites and offers
0004_commerce_finance.sql	Carts, orders, payments, returns and settlements
0005_logistics.sql	Captain and delivery logistics
0006_group_style_engagement.sql	Group Style, notifications and reviews
0007_intelligence_ai.sql	Recommendations, fit, body scan and virtual try-on
0008_operations.sql	Support, campaigns, field operations, moderation, risk and configuration
0009_foreign_keys.sql	All foreign keys after table creation
0010_triggers_views_rpc.sql	updated_at triggers, views and private helper/RPC functions
0011_rls_policies.sql	RLS enablement, grants and policy templates
0012_indexes_storage_realtime.sql	Indexes, Storage buckets/policies and Realtime publication

Deployment commands

```
# Test all migrations locally
supabase db reset

# Inspect what will be applied remotely
supabase db push --dry-run

# Push to staging first
supabase db push

# Optional seed for a controlled non-production environment
supabase db push --include-seed

# Compare local and remote migration history
supabase migration list
```

13. Create the first Super Admin safely

14. Create the admin identity through a controlled dashboard/backend process, not public sign-up UI.
15. Update public.profiles.account_type to ADMIN from a privileged migration/admin script after verifying the account.
16. Insert admin_profiles with employee code, department and required city scope.
17. Assign the SUPER_ADMIN role in user_roles. Require MFA immediately.
18. Use this account to create narrower operational accounts. Do not use Super Admin for normal daily work.
19. Record the bootstrap action in audit_logs and remove any temporary bootstrap script/credential.

Bootstrap SQL - run only in a trusted admin session

```
-- Replace values after creating the Auth user and reviewing the account.
begin;
update public.profiles
set account_type = 'ADMIN', status = 'ACTIVE'
where id = 'AUTH_USER_UUID';

insert into public.admin_profiles(user_id, employee_code, department, city_scope, two_factor_enabled)
values ('AUTH_USER_UUID','EMP-0001','FOUNDERS',array['Tirupati'],true);

insert into public.user_roles(user_id, role_id, assigned_by)
select 'AUTH_USER_UUID', id, 'AUTH_USER_UUID'
```

```
from public.roles where code='SUPER_ADMIN';
commit;
```

14. Mandatory tests before launch

Item	Details
RLS isolation	Customer A cannot read Customer B orders/body data; Merchant A cannot read Merchant B inventory/orders; Captain sees only offered/assigned tasks.
Catalogue	Public/anonymous users can read only approved active products and verified shops.
Order concurrency	Two simultaneous purchases of the final unit result in one success and one clean failure.
Offline sale sync	Barcode/photo/manual sale decrements exact variant and updates customer availability.
Merchant ringing	Foreground Realtime alert, background FCM alert, acknowledgement, retry and timeout all work.
Payments	Duplicate webhooks do not duplicate orders/refunds; invalid signature is rejected.
Delivery	Captain cannot pick up from far away or without assignment/code; OTP completion is idempotent.
Returns	Evidence privacy, return task, merchant inspection, refund approval and inventory disposition work.
Admin permissions	Support cannot alter bank/settlement; finance cannot publish campaigns; catalogue moderator cannot view private body data.
Storage	Public buckets expose only public media; private bucket objects cannot be listed/read by unrelated users.
Privacy	Delete/retention jobs remove body images and try-on inputs according to consent.
Recovery	Restore procedure and backup access are tested before accepting real payments.

15. Routine maintenance

- Review slow query and RLS plans; add indexes before increasing compute.
- Rotate backend secret keys and external provider secrets; keep named keys per backend service where practical.
- Remove stale push tokens and expired inventory reservations; retry/close stuck payment/refund/payout workflows.
- Archive/partition high-volume event, location and audit tables according to retention rules.
- Generate and commit database types after every migration.
- Deploy to staging, run automated RLS/transaction tests, then deploy production through one controlled CI job.

16. Official references

- Current API keys and migration: https://supabase.com/docs/guides/getting-started/migrating-to-new-api-keys
- Local development/migrations: https://supabase.com/docs/guides/local-development/overview
- Database migrations: https://supabase.com/docs/guides/deployment/database-migrations
- RLS: https://supabase.com/docs/guides/database/postgres/row-level-security
- Storage policies: https://supabase.com/docs/guides/storage/security/access-control
- Edge Function secrets: https://supabase.com/docs/guides/functions/secrets
- Secure Edge Functions: https://supabase.com/docs/guides/functions/auth
- Realtime: https://supabase.com/docs/guides/realtime

Appendix - Complete table creation, constraints, RLS, Storage and Realtime SQL

The following numbered queries match the downloadable SQL pack. For actual implementation, use the files from the ZIP so formatting and migration history are preserved. Review and test in local/staging Supabase before production.

0001_extensions_helpers.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.

create extension if not exists pgcrypto;
create extension if not exists postgis;
create extension if not exists citext;
create extension if not exists pg_trgm;
create extension if not exists vector;

create schema if not exists private;
revoke all on schema private from anon, authenticated;

-- updated_at helper
create or replace function private.set_updated_at()
returns trigger
language plpgsql
security invoker
set search_path = ''
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

-- Self-registration is always CUSTOMER. Merchant, captain and admin elevation must
-- happen through trusted backend/admin workflows, never raw_user_meta_data.
create or replace function public.handle_new_auth_user()
returns trigger
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into public.profiles (id, account_type, full_name, phone_number, email)
  values (
    new.id,
    'CUSTOMER',
    coalesce(new.raw_user_meta_data ->> 'full_name', ''),
    new.phone,
    new.email
  )
  on conflict (id) do nothing;
  return new;
end;
$$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_auth_user();
```

0002_identity_access.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Identity & Access: profiles
create table if not exists public."profiles" (
  "id" uuid not null primary key,
  "account_type" text default 'CUSTOMER' not null,
  "full_name" text,
  "phone_number" text,
  "email" citext,
  "avatar_path" text,
  "status" text default 'ACTIVE' not null,
  "preferred_language" text default 'en' not null,
  "last_login_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  "deleted_at" timestamptz,
  constraint "profiles_account_type_check" check ("account_type" in ('CUSTOMER', 'MERCHANT', 'CAPTAIN', 'ADMIN')),
  constraint "profiles_status_check" check ("status" in ('ACTIVE', 'PENDING', 'BLOCKED', 'SUSPENDED', 'DELETED'))
);

comment on table public."profiles" is 'Application profile linked one-to-one with Supabase-managed auth.users. Self-sign-up always starts as CUSTOMER; trusted workflows promote merchant, captain or
admin accounts.';

-- Identity & Access: user_devices
create table if not exists public."user_devices" (
  "id" uuid default gen_random_uuid() not null primary key,
  "user_id" uuid not null,
  "device_fingerprint" text not null,
  "platform" text not null,
  "push_provider" text,
  "push_token" text,
  "app_name" text not null,
  "app_version" text,
  "device_model" text,
  "os_version" text,
  "notification_enabled" boolean default true not null,
  "order_sound_enabled" boolean default true not null,
  "last_seen_at" timestamptz,
  "revoked_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "user_devices_rule_1" unique (user_id, device_fingerprint),
  constraint "user_devices_platform_check" check ("platform" in ('ANDROID', 'IOS', 'WEB')),
  constraint "user_devices_push_provider_check" check ("push_provider" is null or "push_provider" in ('FCM', 'APNS', 'WEB_PUSH')),
  constraint "user_devices_app_name_check" check ("app_name" in ('CUSTOMER', 'MERCHANT', 'CAPTAIN', 'ADMIN'))
);

comment on table public."user_devices" is 'Registered mobile/web devices used for session visibility, push tokens, merchant ringing alerts and security controls.';

-- Identity & Access: addresses
create table if not exists public."addresses" (
  "id" uuid default gen_random_uuid() not null primary key,
  "user_id" uuid not null,
  "label" text,
  "recipient_name" text not null,
  "phone_number" text not null,
  "line1" text not null,
  "line2" text,
  "landmark" text,
  "area" text not null,
  "city" text not null,
  "state" text not null,
  "postal_code" text not null,
  "country_code" text default 'IN' not null,
  "location" geography(Point,4326),
  "is_default" boolean default false not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null
);

comment on table public."addresses" is 'Reusable customer delivery addresses and contact snapshots. Shop addresses are stored directly on shops to avoid ownership ambiguity.';
```

```
-- Identity & Access: customer_profiles
create table if not exists public."customer_profiles" (
  "user_id" uuid not null primary key,
  "date_of_birth" date,
  "gender_preference" text,
  "default_address_id" uuid,
  "profile_completed" boolean default false not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null
);

comment on table public."customer_profiles" is 'Customer-specific data that should not be mixed into the generic profile.';

-- Identity & Access: customer_preferences
create table if not exists public."customer_preferences" (
  "customer_id" uuid not null primary key,
  "preferred_category_ids" uuid[] default '{}'::uuid[] not null,
  "preferred_colours" text[] default '{}'::text[] not null,
  "avoided_colours" text[] default '{}'::text[] not null,
  "preferred_styles" text[] default '{}'::text[] not null,
  "preferred_occasions" text[] default '{}'::text[] not null,
  "preferred_brands" text[] default '{}'::text[] not null,
  "min_budget_paise" bigint,
  "max_budget_paise" bigint,
  "fit_preference" text default 'REGULAR' not null,
  "include_footwear" boolean default true not null,
  "include_accessories" boolean default true not null,
  "updated_at" timestamptz default now() not null,
  constraint "customer_preferences_min_budget_paise_nonnegative" check ("min_budget_paise" is null or "min_budget_paise" >= 0),
  constraint "customer_preferences_max_budget_paise_nonnegative" check ("max_budget_paise" is null or "max_budget_paise" >= 0)
);

comment on table public."customer_preferences" is 'Explicit fashion, budget, fit and notification preferences used by rule-based recommendations and later ML models.';

-- Identity & Access: merchant_profiles
create table if not exists public."merchant_profiles" (
  "user_id" uuid not null primary key,
  "legal_name" text not null,
  "business_type" text,
  "pan_last4" text,
  "pan_encrypted" text,
  "GST_number" text,
  "onboarding_status" text default 'STARTED' not null,
  "kyc_status" text default 'PENDING' not null,
  "approved_at" timestamptz,
  "approved_by" uuid,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "merchant_profiles_onboarding_status_check" check ("onboarding_status" in ('STARTED', 'DOCUMENTS_PENDING', 'VERIFICATION_PENDING', 'CORRECTION_REQUIRED', 'APPROVED', 'CATALOGUE_SETUP',
'TRAINING_PENDING', 'ACTIVE', 'PAUSED', 'SUSPENDED', 'REJECTED')),
  constraint "merchant_profiles_kyc_status_check" check ("kyc_status" in ('PENDING', 'IN_REVIEW', 'VERIFIED', 'REJECTED'))
);

comment on table public."merchant_profiles" is 'Merchant legal/onboarding profile. A single merchant login may own multiple shops later while MVP can enforce one active shop.';

-- Identity & Access: captain_profiles
create table if not exists public."captain_profiles" (
  "user_id" uuid not null primary key,
  "captain_code" text not null,
  "kyc_status" text default 'PENDING' not null,
  "availability_status" text default 'OFFLINE' not null,
  "vehicle_type" text,
  "vehicle_number" text,
  "driving_licence_last4" text,
  "driving_licence_encrypted" text,
  "rating_average" numeric(3,2) default 0 not null,
  "rating_count" integer default 0 not null,
  "completed_deliveries" integer default 0 not null,
  "cash_balance_paise" bigint default 0 not null,
  "approved_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "captain_profiles_captain_code_key" unique ("captain_code"),
  constraint "captain_profiles_kyc_status_check" check ("kyc_status" in ('PENDING', 'IN_REVIEW', 'VERIFIED', 'REJECTED')),
  constraint "captain_profiles_availability_status_check" check ("availability_status" in ('OFFLINE', 'AVAILABLE', 'OFFERED', 'ASSIGNED', 'AT_PICKUP', 'DELIVERING', 'ON_BREAK', 'SUSPENDED')),
  constraint "captain_profiles_vehicle_type_check" check ("vehicle_type" is null or "vehicle_type" in ('BIKE', 'SCOOTER', 'BICYCLE', 'WALKER', 'AUTO', 'CAR')),
  constraint "captain_profiles_rating_average_check" check (rating_average between 0 and 5),
```

```
  constraint "captain_profiles_rating_count_nonnegative" check ("rating_count" is null or "rating_count" >= 0),
  constraint "captain_profiles_completed_deliveries_nonnegative" check ("completed_deliveries" is null or "completed_deliveries" >= 0)
);

comment on table public."captain_profiles" is 'Delivery partner profile, availability, vehicle and performance data.';

-- Identity & Access: admin_profiles
create table if not exists public."admin_profiles" (
  "user_id" uuid not null primary key,
  "employee_code" text not null,
  "department" text not null,
  "city_scope" text[] default '{}'::text[] not null,
  "manager_id" uuid,
  "two_factor_enabled" boolean default false not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "admin_profiles_employee_code_key" unique ("employee_code")
);

comment on table public."admin_profiles" is 'Internal employee profile, department, city scope and second-factor status.';

-- Identity & Access: roles
create table if not exists public."roles" (
  "id" uuid default gen_random_uuid() not null primary key,
  "code" text not null,
  "name" text not null,
  "description" text,
  "is_system_role" boolean default true not null,
  "created_at" timestamptz default now() not null,
  constraint "roles_code_key" unique ("code")
);

comment on table public."roles" is 'Named application roles for granular admin and operational access.';

-- Identity & Access: permissions
create table if not exists public."permissions" (
  "id" uuid default gen_random_uuid() not null primary key,
  "code" text not null,
  "module" text not null,
  "name" text not null,
  "description" text,
  "created_at" timestamptz default now() not null,
  constraint "permissions_code_key" unique ("code")
);

comment on table public."permissions" is 'Fine-grained actions used by admin RLS helpers and backend authorization.';

-- Identity & Access: user_roles
create table if not exists public."user_roles" (
  "id" uuid default gen_random_uuid() not null primary key,
  "user_id" uuid not null,
  "role_id" uuid not null,
  "assigned_by" uuid,
  "assigned_at" timestamptz default now() not null,
  "revoked_at" timestamptz,
  constraint "user_roles_rule_1" unique (user_id, role_id)
);

comment on table public."user_roles" is 'Many-to-many mapping of users to internal roles; normally only admins receive these roles.';

-- Identity & Access: role_permissions
create table if not exists public."role_permissions" (
  "id" uuid default gen_random_uuid() not null primary key,
  "role_id" uuid not null,
  "permission_id" uuid not null,
  "created_at" timestamptz default now() not null,
  constraint "role_permissions_rule_1" unique (role_id, permission_id)
);

comment on table public."role_permissions" is 'Many-to-many mapping between roles and allowed actions.';
```

0003_marketplace_catalog_inventory.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Marketplace: shops
create table if not exists public."shops" (
  "id" uuid default gen_random_uuid() not null primary key,
  "merchant_id" uuid not null,
  "shop_code" text not null,
  "name" text not null,
  "slug" text not null,
  "description" text,
  "phone_number" text not null,
  "email" citext,
  "address_line1" text not null,
  "address_line2" text,
  "landmark" text,
  "area" text not null,
  "city" text not null,
  "state" text not null,
  "postal_code" text not null,
  "country_code" text default 'IN' not null,
  "location" geography(Point,4326) not null,
  "logo_path" text,
  "cover_image_path" text,
  "verification_status" text default 'PENDING' not null,
  "operational_status" text default 'CLOSED_FOR_DAY' not null,
  "accepts_online_orders" boolean default false not null,
  "service_radius_meters" integer default 5000 not null,
  "minimum_order_paise" bigint default 0 not null,
  "average_preparation_minutes" integer default 15 not null,
  "rating_average" numeric(3,2) default 0 not null,
  "rating_count" integer default 0 not null,
  "follower_count" integer default 0 not null,
  "version" integer default 1 not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  "deleted_at" timestamptz,
  constraint "shops_shop_code_key" unique ("shop_code"),
  constraint "shops_slug_key" unique ("slug"),
  constraint "shops_verification_status_check" check ("verification_status" in ('PENDING', 'IN_REVIEW', 'VERIFIED', 'REJECTED')),
  constraint "shops_operational_status_check" check ("operational_status" in ('OPEN', 'BUSY', 'TEMPORARILY_CLOSED', 'CLOSED_FOR_DAY', 'PAUSED', 'SUSPENDED')),
  constraint "shops_service_radius_meters_nonnegative" check ("service_radius_meters" is null or "service_radius_meters" >= 0),
  constraint "shops_minimum_order_paise_nonnegative" check ("minimum_order_paise" is null or "minimum_order_paise" >= 0),
  constraint "shops_average_preparation_minutes_nonnegative" check ("average_preparation_minutes" is null or "average_preparation_minutes" >= 0),
  constraint "shops_rating_average_check" check (rating_average between 0 and 5),
  constraint "shops_rating_count_nonnegative" check ("rating_count" is null or "rating_count" >= 0),
  constraint "shops_follower_count_nonnegative" check ("follower_count" is null or "follower_count" >= 0),
  constraint "shops_version_nonnegative" check ("version" is null or "version" >= 0)
);

comment on table public."shops" is 'Canonical shop record shown in the customer marketplace and controlled by the approved merchant.';

-- Marketplace: shop_hours
create table if not exists public."shop_hours" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "schedule_type" text default 'WEEKLY' not null,
  "day_of_week" smallint,
  "special_date" date,
  "open_time" time,
  "close_time" time,
  "is_closed" boolean default false not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "shop_hours_rule_1" unique (shop_id, schedule_type, day_of_week, special_date),
  constraint "shop_hours_schedule_type_check" check ("schedule_type" in ('WEEKLY', 'SPECIAL_DATE')),
  constraint "shop_hours_day_of_week_check" check (day_of_week between 0 and 6)
);

comment on table public."shop_hours" is 'Weekly and date-specific opening hours.';

-- Marketplace: shop_documents
```

```
create table if not exists public."shop_documents" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "uploaded_by" uuid not null,
  "document_type" text not null,
  "document_number_last4" text,
  "document_number_encrypted" text,
  "storage_path" text not null,
  "verification_status" text default 'PENDING' not null,
  "verified_by" uuid,
  "verified_at" timestamptz,
  "rejection_reason" text,
  "created_at" timestamptz default now() not null,
  constraint "shop_documents_document_type_check" check ("document_type" in ('PAN', 'GST', 'LICENSE', 'ADDRESS_PROOF', 'OWNER_ID', 'SHOP_PHOTO', 'OTHER')),
  constraint "shop_documents_verification_status_check" check ("verification_status" in ('PENDING', 'VERIFIED', 'REJECTED'))
);

comment on table public."shop_documents" is 'Private merchant/shop KYC files and verification status.';

-- Marketplace: shop_bank_accounts
create table if not exists public."shop_bank_accounts" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "account_holder_name" text not null,
  "account_number_last4" text not null,
  "account_number_encrypted" text not null,
  "ifsc_code" text not null,
  "bank_name" text,
  "branch_name" text,
  "is_primary" boolean default true not null,
  "verification_status" text default 'PENDING' not null,
  "verified_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "shop_bank_accounts_verification_status_check" check ("verification_status" in ('PENDING', 'VERIFIED', 'REJECTED'))
);

comment on table public."shop_bank_accounts" is 'Private merchant payout account records.';

-- Catalogue: categories
create table if not exists public."categories" (
  "id" uuid default gen_random_uuid() not null primary key,
  "parent_id" uuid,
  "name" text not null,
  "slug" text not null,
  "description" text,
  "icon_path" text,
  "display_order" integer default 0 not null,
  "is_active" boolean default true not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "categories_slug_key" unique ("slug")
);

comment on table public."categories" is 'Hierarchical catalogue taxonomy covering clothing, footwear and accessories.';

-- Catalogue: products
create table if not exists public."products" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "category_id" uuid not null,
  "name" text not null,
  "slug" text not null,
  "description" text,
  "brand" text,
  "material" text,
  "gender_category" text default 'UNISEX' not null,
  "style_tags" text[] default '{}'::text[] not null,
  "occasion_tags" text[] default '{}'::text[] not null,
  "care_instructions" text,
  "return_eligible" boolean default true not null,
  "return_window_days" smallint default 7 not null,
  "moderation_status" text default 'PENDING' not null,
  "is_active" boolean default true not null,
  "search_vector" tsvector,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  "deleted_at" timestamptz,
```

```
  constraint "products_rule_1" unique (shop_id, slug),
  constraint "products_gender_category_check" check ("gender_category" in ('MEN', 'WOMEN', 'KIDS', 'UNISEX')),
  constraint "products_return_window_days_check" check (return_window_days between 0 and 90),
  constraint "products_moderation_status_check" check ("moderation_status" in ('PENDING', 'APPROVED', 'REJECTED', 'CORRECTION_REQUIRED'))
);

comment on table public."products" is 'General merchant product record independent of colour/size variants.';

-- Catalogue: product_images
create table if not exists public."product_images" (
  "id" uuid default gen_random_uuid() not null primary key,
  "product_id" uuid not null,
  "variant_id" uuid,
  "storage_path" text not null,
  "thumbnail_path" text,
  "image_type" text default 'FRONT' not null,
  "alt_text" text,
  "display_order" smallint default 0 not null,
  "is_primary" boolean default false not null,
  "width_px" integer,
  "height_px" integer,
  "created_at" timestamptz default now() not null,
  constraint "product_images_image_type_check" check ("image_type" in ('FRONT', 'BACK', 'SIDE', 'DETAIL', 'MODEL', 'SIZE_CHART'))
);

comment on table public."product_images" is 'Ordered product and variant media references stored in Supabase Storage.';

-- Catalogue: product_variants
create table if not exists public."product_variants" (
  "id" uuid default gen_random_uuid() not null primary key,
  "product_id" uuid not null,
  "shop_id" uuid not null,
  "sku" text not null,
  "colour_name" text,
  "colour_hex" text,
  "size_label" text,
  "mrp_paise" bigint not null,
  "selling_price_paise" bigint not null,
  "cost_price_paise" bigint,
  "weight_grams" integer,
  "length_cm" numeric(8,2),
  "width_cm" numeric(8,2),
  "height_cm" numeric(8,2),
  "attributes" jsonb default '{}'::jsonb not null,
  "is_active" boolean default true not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "product_variants_rule_1" unique (shop_id, sku),
  constraint "product_variants_rule_2" check (selling_price_paise <= mrp_paise),
  constraint "product_variants_mrp_paise_nonnegative" check ("mrp_paise" is null or "mrp_paise" >= 0),
  constraint "product_variants_selling_price_paise_nonnegative" check ("selling_price_paise" is null or "selling_price_paise" >= 0),
  constraint "product_variants_cost_price_paise_nonnegative" check ("cost_price_paise" is null or "cost_price_paise" >= 0),
  constraint "product_variants_weight_grams_nonnegative" check ("weight_grams" is null or "weight_grams" >= 0),
  constraint "product_variants_length_cm_nonnegative" check ("length_cm" is null or "length_cm" >= 0),
  constraint "product_variants_width_cm_nonnegative" check ("width_cm" is null or "width_cm" >= 0),
  constraint "product_variants_height_cm_nonnegative" check ("height_cm" is null or "height_cm" >= 0)
);

comment on table public."product_variants" is 'Exact sellable SKU for each colour, size and optional attribute combination.';

-- Catalogue: variant_barcodes
create table if not exists public."variant_barcodes" (
  "id" uuid default gen_random_uuid() not null primary key,
  "variant_id" uuid not null,
  "barcode_value" text not null,
  "barcode_type" text default 'CODE128' not null,
  "source" text default 'MERCHANT_ENTERED' not null,
  "is_primary" boolean default true not null,
  "created_by" uuid,
  "created_at" timestamptz default now() not null,
  constraint "variant_barcodes_barcode_value_key" unique ("barcode_value"),
  constraint "variant_barcodes_barcode_type_check" check ("barcode_type" in ('EAN13', 'UPC', 'CODE128', 'QR', 'INTERNAL')),
  constraint "variant_barcodes_source_check" check ("source" in ('MANUFACTURER', 'VASTRA_GENERATED', 'MERCHANT_ENTERED'))
);

comment on table public."variant_barcodes" is 'Manufacturer, merchant-entered and Vastra-generated barcodes mapped to exact variants.';

-- Inventory: inventory_balances
```

```
create table if not exists public."inventory_balances" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "variant_id" uuid not null,
  "stock_on_hand" integer default 0 not null,
  "reserved_quantity" integer default 0 not null,
  "damaged_quantity" integer default 0 not null,
  "reorder_level" integer default 0 not null,
  "version" integer default 1 not null,
  "last_counted_at" timestamptz,
  "updated_at" timestamptz default now() not null,
  constraint "inventory_balances_rule_1" unique (shop_id, variant_id),
  constraint "inventory_balances_rule_2" check (stock_on_hand >= reserved_quantity + damaged_quantity),
  constraint "inventory_balances_stock_on_hand_nonnegative" check ("stock_on_hand" is null or "stock_on_hand" >= 0),
  constraint "inventory_balances_reserved_quantity_nonnegative" check ("reserved_quantity" is null or "reserved_quantity" >= 0),
  constraint "inventory_balances_damaged_quantity_nonnegative" check ("damaged_quantity" is null or "damaged_quantity" >= 0),
  constraint "inventory_balances_reorder_level_nonnegative" check ("reorder_level" is null or "reorder_level" >= 0),
  constraint "inventory_balances_version_nonnegative" check ("version" is null or "version" >= 0)
);

comment on table public."inventory_balances" is 'Current variant-level physical, reserved and damaged quantities. This is the authoritative balance row.';

-- Inventory: inventory_movements
create table if not exists public."inventory_movements" (
  "id" bigint default generated always as identity not null primary key,
  "shop_id" uuid not null,
  "variant_id" uuid not null,
  "movement_type" text not null,
  "quantity_change" integer not null,
  "reserved_change" integer default 0 not null,
  "damaged_change" integer default 0 not null,
  "stock_before" integer not null,
  "stock_after" integer not null,
  "reserved_before" integer not null,
  "reserved_after" integer not null,
  "damaged_before" integer not null,
  "damaged_after" integer not null,
  "reference_type" text,
  "reference_id" uuid,
  "reason" text,
  "performed_by" uuid,
  "source_method" text default 'SYSTEM' not null,
  "created_at" timestamptz default now() not null,
  constraint "inventory_movements_movement_type_check" check ("movement_type" in ('STOCK_RECEIVED', 'OFFLINE_SALE', 'ONLINE_ORDER_RESERVED', 'ONLINE_ORDER_RELEASED', 'ONLINE_ORDER_COMPLETED',
'RETURN_TO_STOCK', 'MARKED_DAMAGED', 'STOCK_CORRECTION', 'STOCK_AUDIT')),
  constraint "inventory_movements_source_method_check" check ("source_method" in ('BARCODE', 'PHOTO_MATCH', 'MANUAL_SEARCH', 'SYSTEM', 'ADMIN'))
);

comment on table public."inventory_movements" is 'Immutable audit ledger for every stock change from barcode, photo, manual action, order or admin correction.';

-- Inventory: inventory_reservations
create table if not exists public."inventory_reservations" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "variant_id" uuid not null,
  "cart_id" uuid,
  "order_id" uuid,
  "quantity" integer not null,
  "status" text default 'ACTIVE' not null,
  "expires_at" timestamptz not null,
  "created_at" timestamptz default now() not null,
  "released_at" timestamptz,
  constraint "inventory_reservations_quantity_nonnegative" check ("quantity" is null or "quantity" >= 0),
  constraint "inventory_reservations_status_check" check ("status" in ('ACTIVE', 'CONVERTED', 'RELEASED', 'EXPIRED'))
);

comment on table public."inventory_reservations" is 'Temporary or order-linked reservations that prevent double-selling the final unit.';

-- Inventory: product_photo_matches
create table if not exists public."product_photo_matches" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "uploaded_by" uuid not null,
  "uploaded_image_path" text not null,
  "matched_variant_id" uuid,
  "confidence_score" numeric(6,5),
  "candidate_matches" jsonb default '[]'::jsonb not null,
  "merchant_confirmed" boolean default false not null,
```

```
  "match_model_version" text,
  "created_at" timestamptz default now() not null,
  constraint "product_photo_matches_confidence_score_check" check (confidence_score between 0 and 1)
);

comment on table public."product_photo_matches" is 'History and feedback for merchant photo-based catalogue matching.';

-- Catalogue: shop_collections
create table if not exists public."shop_collections" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "name" text not null,
  "description" text,
  "collection_type" text default 'MANUAL' not null,
  "cover_image_path" text,
  "is_active" boolean default true not null,
  "display_order" integer default 0 not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "shop_collections_collection_type_check" check ("collection_type" in ('MANUAL', 'COMPLETE_LOOK', 'OCCASION', 'GROUP_STYLE'))
);

comment on table public."shop_collections" is 'Merchant-curated occasion, complete-look and promotional collections.';

-- Catalogue: shop_collection_items
create table if not exists public."shop_collection_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "collection_id" uuid not null,
  "product_id" uuid not null,
  "variant_id" uuid,
  "item_role" text,
  "display_order" integer default 0 not null,
  "created_at" timestamptz default now() not null,
  constraint "shop_collection_items_rule_1" unique (collection_id, product_id, variant_id)
);

comment on table public."shop_collection_items" is 'Ordered products/variants inside merchant collections.';

-- Promotions: offers
create table if not exists public."offers" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid not null,
  "name" text not null,
  "description" text,
  "offer_type" text not null,
  "discount_value" numeric(12,2),
  "minimum_order_paise" bigint default 0 not null,
  "maximum_discount_paise" bigint,
  "usage_limit_total" integer,
  "usage_limit_per_customer" integer default 1 not null,
  "starts_at" timestamptz not null,
  "ends_at" timestamptz not null,
  "status" text default 'DRAFT' not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "offers_offer_type_check" check ("offer_type" in ('PERCENT', 'FIXED', 'FREE_DELIVERY', 'BUY_X_GET_Y', 'BUNDLE')),
  constraint "offers_discount_value_nonnegative" check ("discount_value" is null or "discount_value" >= 0),
  constraint "offers_minimum_order_paise_nonnegative" check ("minimum_order_paise" is null or "minimum_order_paise" >= 0),
  constraint "offers_maximum_discount_paise_nonnegative" check ("maximum_discount_paise" is null or "maximum_discount_paise" >= 0),
  constraint "offers_usage_limit_total_nonnegative" check ("usage_limit_total" is null or "usage_limit_total" >= 0),
  constraint "offers_usage_limit_per_customer_nonnegative" check ("usage_limit_per_customer" is null or "usage_limit_per_customer" >= 0),
  constraint "offers_status_check" check ("status" in ('DRAFT', 'ACTIVE', 'PAUSED', 'EXPIRED'))
);

comment on table public."offers" is 'Merchant-created discounts and bundle/free-delivery offers.';

-- Promotions: offer_products
create table if not exists public."offer_products" (
  "id" uuid default gen_random_uuid() not null primary key,
  "offer_id" uuid not null,
  "product_id" uuid not null,
  "variant_id" uuid,
  "created_at" timestamptz default now() not null,
  constraint "offer_products_rule_1" unique (offer_id, product_id, variant_id)
);

comment on table public."offer_products" is 'Products/variants explicitly included in an offer.';
```

```
-- Promotions: offer_redemptions
create table if not exists public."offer_redemptions" (
  "id" uuid default gen_random_uuid() not null primary key,
  "offer_id" uuid not null,
  "customer_id" uuid not null,
  "order_id" uuid not null,
  "discount_paise" bigint not null,
  "created_at" timestamptz default now() not null,
  constraint "offer_redemptions_rule_1" unique (offer_id, customer_id, order_id),
  constraint "offer_redemptions_discount_paise_nonnegative" check ("discount_paise" is null or "discount_paise" >= 0)
);

comment on table public."offer_redemptions" is 'Immutable application of an offer to a completed/placed order.';
```

0004_commerce_finance.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Commerce: offline_sales
create table if not exists public."offline_sales" (
  "id" uuid default gen_random_uuid() not null primary key,
  "sale_number" text not null,
  "shop_id" uuid not null,
  "merchant_id" uuid not null,
  "customer_phone" text,
  "subtotal_paise" bigint default 0 not null,
  "discount_paise" bigint default 0 not null,
  "tax_paise" bigint default 0 not null,
  "total_paise" bigint default 0 not null,
  "payment_method" text not null,
  "status" text default 'COMPLETED' not null,
  "recorded_by" uuid not null,
  "created_at" timestamptz default now() not null,
  "voided_at" timestamptz,
  constraint "offline_sales_sale_number_key" unique ("sale_number"),
  constraint "offline_sales_subtotal_paise_nonnegative" check ("subtotal_paise" is null or "subtotal_paise" >= 0),
  constraint "offline_sales_discount_paise_nonnegative" check ("discount_paise" is null or "discount_paise" >= 0),
  constraint "offline_sales_tax_paise_nonnegative" check ("tax_paise" is null or "tax_paise" >= 0),
  constraint "offline_sales_total_paise_nonnegative" check ("total_paise" is null or "total_paise" >= 0),
  constraint "offline_sales_payment_method_check" check ("payment_method" in ('CASH', 'UPI', 'CARD', 'OTHER')),
  constraint "offline_sales_status_check" check ("status" in ('DRAFT', 'COMPLETED', 'VOIDED', 'REFUNDED'))
);

comment on table public."offline_sales" is 'Physical shop sale recorded by barcode, photo match or manual product selection so online inventory stays accurate.';

-- Commerce: offline_sale_items
create table if not exists public."offline_sale_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "offline_sale_id" uuid not null,
  "variant_id" uuid not null,
  "quantity" integer not null,
  "unit_price_paise" bigint not null,
  "discount_paise" bigint default 0 not null,
  "total_paise" bigint not null,
  "identification_method" text default 'MANUAL_SEARCH' not null,
  "created_at" timestamptz default now() not null,
  constraint "offline_sale_items_quantity_nonnegative" check ("quantity" is null or "quantity" >= 0),
  constraint "offline_sale_items_unit_price_paise_nonnegative" check ("unit_price_paise" is null or "unit_price_paise" >= 0),
  constraint "offline_sale_items_discount_paise_nonnegative" check ("discount_paise" is null or "discount_paise" >= 0),
  constraint "offline_sale_items_total_paise_nonnegative" check ("total_paise" is null or "total_paise" >= 0),
  constraint "offline_sale_items_identification_method_check" check ("identification_method" in ('BARCODE', 'PHOTO_MATCH', 'MANUAL_SEARCH'))
);

comment on table public."offline_sale_items" is 'Line items in a physical shop sale.';

-- Commerce: carts
create table if not exists public."carts" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "shop_id" uuid not null,
  "status" text default 'ACTIVE' not null,
  "coupon_code" text,
  "expires_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "carts_status_check" check ("status" in ('ACTIVE', 'CONVERTED', 'ABANDONED', 'EXPIRED'))
);

comment on table public."carts" is 'One active single-shop cart per customer in the MVP.';

-- Commerce: cart_items
create table if not exists public."cart_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "cart_id" uuid not null,
  "variant_id" uuid not null,
  "quantity" integer default 1 not null,
  "unit_price_snapshot_paise" bigint not null,
```

```
  "added_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "cart_items_rule_1" unique (cart_id, variant_id),
  constraint "cart_items_quantity_nonnegative" check ("quantity" is null or "quantity" >= 0),
  constraint "cart_items_unit_price_snapshot_paise_nonnegative" check ("unit_price_snapshot_paise" is null or "unit_price_snapshot_paise" >= 0)
);

comment on table public."cart_items" is 'Variant-level cart lines with price snapshots for display; final prices are revalidated at order placement.';

-- Commerce: orders
create table if not exists public."orders" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_number" text not null,
  "customer_id" uuid not null,
  "shop_id" uuid not null,
  "cart_id" uuid,
  "style_group_id" uuid,
  "delivery_address_id" uuid not null,
  "address_snapshot" jsonb default '{}'::jsonb not null,
  "status" text default 'PAYMENT_PENDING' not null,
  "payment_status" text default 'PENDING' not null,
  "fulfilment_type" text default 'DELIVERY' not null,
  "subtotal_paise" bigint default 0 not null,
  "product_discount_paise" bigint default 0 not null,
  "coupon_discount_paise" bigint default 0 not null,
  "delivery_fee_paise" bigint default 0 not null,
  "platform_fee_paise" bigint default 0 not null,
  "tax_paise" bigint default 0 not null,
  "total_paise" bigint default 0 not null,
  "merchant_preparation_minutes" integer,
  "estimated_delivery_at" timestamptz,
  "customer_note" text,
  "cancellation_reason_code" text,
  "cancellation_note" text,
  "version" integer default 1 not null,
  "placed_at" timestamptz,
  "accepted_at" timestamptz,
  "ready_at" timestamptz,
  "picked_up_at" timestamptz,
  "delivered_at" timestamptz,
  "cancelled_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "orders_order_number_key" unique ("order_number"),
  constraint "orders_status_check" check ("status" in ('PAYMENT_PENDING', 'WAITING_FOR_MERCHANT', 'MERCHANT_ACCEPTED', 'PACKING', 'READY_FOR_PICKUP', 'CAPTAIN_SEARCHING', 'CAPTAIN_ASSIGNED',
'CAPTAIN_AT_STORE', 'PICKED_UP', 'OUT_FOR_DELIVERY', 'CAPTAIN_AT_CUSTOMER', 'DELIVERED', 'COMPLETED', 'PROBLEM_REPORTED', 'CANCELLED')),
  constraint "orders_payment_status_check" check ("payment_status" in ('PENDING', 'AUTHORIZED', 'CAPTURED', 'FAILED', 'PARTIALLY_REFUNDED', 'REFUNDED', 'COD_PENDING', 'COD_COLLECTED')),
  constraint "orders_fulfilment_type_check" check ("fulfilment_type" in ('DELIVERY', 'CUSTOMER_PICKUP')),
  constraint "orders_subtotal_paise_nonnegative" check ("subtotal_paise" is null or "subtotal_paise" >= 0),
  constraint "orders_product_discount_paise_nonnegative" check ("product_discount_paise" is null or "product_discount_paise" >= 0),
  constraint "orders_coupon_discount_paise_nonnegative" check ("coupon_discount_paise" is null or "coupon_discount_paise" >= 0),
  constraint "orders_delivery_fee_paise_nonnegative" check ("delivery_fee_paise" is null or "delivery_fee_paise" >= 0),
  constraint "orders_platform_fee_paise_nonnegative" check ("platform_fee_paise" is null or "platform_fee_paise" >= 0),
  constraint "orders_tax_paise_nonnegative" check ("tax_paise" is null or "tax_paise" >= 0),
  constraint "orders_total_paise_nonnegative" check ("total_paise" is null or "total_paise" >= 0),
  constraint "orders_merchant_preparation_minutes_nonnegative" check ("merchant_preparation_minutes" is null or "merchant_preparation_minutes" >= 0),
  constraint "orders_version_nonnegative" check ("version" is null or "version" >= 0)
);

comment on table public."orders" is 'Authoritative customer order header and current lifecycle state.';

-- Commerce: order_items
create table if not exists public."order_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_id" uuid not null,
  "product_id" uuid not null,
  "variant_id" uuid not null,
  "product_name_snapshot" text not null,
  "sku_snapshot" text not null,
  "colour_snapshot" text,
  "size_snapshot" text,
  "image_path_snapshot" text,
  "quantity" integer not null,
  "unit_mrp_paise" bigint not null,
  "unit_selling_price_paise" bigint not null,
  "discount_paise" bigint default 0 not null,
  "total_paise" bigint not null,
  "fulfilment_status" text default 'PENDING' not null,
```

```
  "created_at" timestamptz default now() not null,
  constraint "order_items_quantity_nonnegative" check ("quantity" is null or "quantity" >= 0),
  constraint "order_items_unit_mrp_paise_nonnegative" check ("unit_mrp_paise" is null or "unit_mrp_paise" >= 0),
  constraint "order_items_unit_selling_price_paise_nonnegative" check ("unit_selling_price_paise" is null or "unit_selling_price_paise" >= 0),
  constraint "order_items_discount_paise_nonnegative" check ("discount_paise" is null or "discount_paise" >= 0),
  constraint "order_items_total_paise_nonnegative" check ("total_paise" is null or "total_paise" >= 0),
  constraint "order_items_fulfilment_status_check" check ("fulfilment_status" in ('PENDING', 'VERIFIED', 'PACKED', 'HANDED_OVER', 'RETURNED', 'CANCELLED'))
);

comment on table public."order_items" is 'Immutable product and price snapshots for each exact ordered variant.';

-- Commerce: order_status_history
create table if not exists public."order_status_history" (
  "id" bigint default generated always as identity not null primary key,
  "order_id" uuid not null,
  "previous_status" text,
  "new_status" text not null,
  "changed_by_user_id" uuid,
  "changed_by_role" text default 'SYSTEM' not null,
  "reason_code" text,
  "note" text,
  "location" geography(Point,4326),
  "created_at" timestamptz default now() not null
);

comment on table public."order_status_history" is 'Append-only timeline for every state transition and operational reason.';

-- Commerce: merchant_order_alerts
create table if not exists public."merchant_order_alerts" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_id" uuid not null,
  "shop_id" uuid not null,
  "device_id" uuid,
  "alert_status" text default 'PENDING' not null,
  "attempt_count" smallint default 0 not null,
  "first_sent_at" timestamptz,
  "last_sent_at" timestamptz,
  "acknowledged_at" timestamptz,
  "acknowledged_by" uuid,
  "expires_at" timestamptz not null,
  "sound_name" text default 'vastra_new_order' not null,
  "failure_reason" text,
  "created_at" timestamptz default now() not null,
  constraint "merchant_order_alerts_alert_status_check" check ("alert_status" in ('PENDING', 'SENT', 'DELIVERED', 'ACKNOWLEDGED', 'EXPIRED', 'FAILED')),
  constraint "merchant_order_alerts_attempt_count_check" check (attempt_count between 0 and 20)
);

comment on table public."merchant_order_alerts" is 'Tracks every urgent order alert, retry, delivery and acknowledgement for the merchant ringing experience.';

-- Commerce: merchant_order_issues
create table if not exists public."merchant_order_issues" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_id" uuid not null,
  "order_item_id" uuid,
  "issue_type" text not null,
  "description" text,
  "evidence_path" text,
  "reported_by" uuid not null,
  "resolution_status" text default 'OPEN' not null,
  "resolved_by" uuid,
  "resolution_note" text,
  "created_at" timestamptz default now() not null,
  "resolved_at" timestamptz,
  constraint "merchant_order_issues_issue_type_check" check ("issue_type" in ('OUT_OF_STOCK', 'SIZE_UNAVAILABLE', 'COLOUR_UNAVAILABLE', 'DAMAGED_ITEM', 'INVENTORY_MISMATCH', 'ITEM_NOT_FOUND',
'PRICING_ERROR', 'SHOP_BUSY', 'SHOP_CLOSING', 'OTHER')),
  constraint "merchant_order_issues_resolution_status_check" check ("resolution_status" in ('OPEN', 'ACCEPTED', 'REPLACEMENT_REQUESTED', 'RESOLVED', 'REJECTED'))
);

comment on table public."merchant_order_issues" is 'Merchant-reported inability to fulfil an order because of stock, size, colour, damage, pricing or operational issues.';

-- Commerce: order_item_verifications
create table if not exists public."order_item_verifications" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_item_id" uuid not null,
  "verification_method" text not null,
  "scanned_barcode" text,
  "photo_path" text,
  "verified_variant_id" uuid,
```

```
  "result" text not null,
  "verified_by" uuid not null,
  "verified_at" timestamptz default now() not null,
  "override_reason" text,
  constraint "order_item_verifications_verification_method_check" check ("verification_method" in ('BARCODE', 'PHOTO', 'MANUAL')),
  constraint "order_item_verifications_result_check" check ("result" in ('MATCH', 'MISMATCH', 'OVERRIDDEN'))
);

comment on table public."order_item_verifications" is 'Packing verification result using barcode, photo or manual confirmation.';

-- Payments: payments
create table if not exists public."payments" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_id" uuid not null,
  "customer_id" uuid not null,
  "provider" text not null,
  "provider_order_id" text,
  "provider_payment_id" text,
  "method" text not null,
  "amount_paise" bigint not null,
  "currency" text default 'INR' not null,
  "status" text default 'CREATED' not null,
  "signature_verified" boolean default false not null,
  "failure_code" text,
  "failure_message" text,
  "paid_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "payments_method_check" check ("method" in ('UPI', 'CARD', 'NETBANKING', 'WALLET', 'COD', 'OTHER')),
  constraint "payments_amount_paise_nonnegative" check ("amount_paise" is null or "amount_paise" >= 0),
  constraint "payments_status_check" check ("status" in ('CREATED', 'PENDING', 'AUTHORIZED', 'CAPTURED', 'FAILED', 'CANCELLED', 'PARTIALLY_REFUNDED', 'REFUNDED'))
);

comment on table public."payments" is 'Payment attempt and provider identifiers for online and COD orders.';

-- Payments: payment_events
create table if not exists public."payment_events" (
  "id" bigint default generated always as identity not null primary key,
  "payment_id" uuid,
  "provider" text not null,
  "provider_event_id" text not null,
  "event_type" text not null,
  "payload" jsonb not null,
  "signature_valid" boolean default false not null,
  "processing_status" text default 'RECEIVED' not null,
  "processing_error" text,
  "received_at" timestamptz default now() not null,
  "processed_at" timestamptz,
  constraint "payment_events_provider_event_id_key" unique ("provider_event_id"),
  constraint "payment_events_processing_status_check" check ("processing_status" in ('RECEIVED', 'PROCESSED', 'IGNORED', 'FAILED'))
);

comment on table public."payment_events" is 'Idempotent raw gateway webhook/event log.';

-- Returns: return_requests
create table if not exists public."return_requests" (
  "id" uuid default gen_random_uuid() not null primary key,
  "return_number" text not null,
  "order_id" uuid not null,
  "customer_id" uuid not null,
  "shop_id" uuid not null,
  "reason_code" text not null,
  "customer_note" text,
  "status" text default 'REQUESTED' not null,
  "refund_method" text default 'ORIGINAL' not null,
  "requested_at" timestamptz default now() not null,
  "approved_at" timestamptz,
  "completed_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "return_requests_return_number_key" unique ("return_number"),
  constraint "return_requests_reason_code_check" check ("reason_code" in ('WRONG_SIZE', 'WRONG_COLOUR', 'WRONG_PRODUCT', 'DAMAGED', 'QUALITY', 'DIFFERENT_FROM_IMAGE', 'MISSING_ITEM', 'CHANGED_MIND',
'OTHER')),
  constraint "return_requests_status_check" check ("status" in ('REQUESTED', 'REVIEW', 'APPROVED', 'REJECTED', 'PICKUP_ASSIGNED', 'PICKED_UP', 'RECEIVED', 'VERIFIED', 'REFUND_PENDING', 'REFUNDED',
'CLOSED')),
  constraint "return_requests_refund_method_check" check ("refund_method" in ('ORIGINAL', 'WALLET', 'MANUAL'))
);
```

```
comment on table public."return_requests" is 'Customer return request header and current workflow status.';

-- Returns: return_items
create table if not exists public."return_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "return_request_id" uuid not null,
  "order_item_id" uuid not null,
  "quantity" integer default 1 not null,
  "reason_code" text not null,
  "requested_refund_paise" bigint not null,
  "inspection_status" text default 'PENDING' not null,
  "merchant_decision" text,
  "merchant_note" text,
  "inspected_by" uuid,
  "inspected_at" timestamptz,
  constraint "return_items_quantity_nonnegative" check ("quantity" is null or "quantity" >= 0),
  constraint "return_items_requested_refund_paise_nonnegative" check ("requested_refund_paise" is null or "requested_refund_paise" >= 0),
  constraint "return_items_inspection_status_check" check ("inspection_status" in ('PENDING', 'SELLABLE', 'DAMAGED', 'USED', 'WRONG_ITEM', 'DISPUTED')),
  constraint "return_items_merchant_decision_check" check ("merchant_decision" is null or "merchant_decision" in ('ACCEPTED', 'DISPUTED', 'PARTIAL'))
);

comment on table public."return_items" is 'Per-order-item quantities, inspection and merchant decision.';

-- Returns: return_evidence
create table if not exists public."return_evidence" (
  "id" uuid default gen_random_uuid() not null primary key,
  "return_request_id" uuid not null,
  "uploaded_by" uuid not null,
  "evidence_type" text not null,
  "storage_path" text,
  "description" text,
  "created_at" timestamptz default now() not null,
  constraint "return_evidence_evidence_type_check" check ("evidence_type" in ('CUSTOMER_PHOTO', 'MERCHANT_PHOTO', 'CAPTAIN_PHOTO', 'VIDEO', 'DOCUMENT', 'NOTE'))
);

comment on table public."return_evidence" is 'Customer, merchant, captain and admin evidence files for return disputes.';

-- Returns: return_status_history
create table if not exists public."return_status_history" (
  "id" bigint default generated always as identity not null primary key,
  "return_request_id" uuid not null,
  "previous_status" text,
  "new_status" text not null,
  "changed_by" uuid,
  "reason_code" text,
  "note" text,
  "created_at" timestamptz default now() not null
);

comment on table public."return_status_history" is 'Append-only return state timeline.';

-- Payments: refunds
create table if not exists public."refunds" (
  "id" uuid default gen_random_uuid() not null primary key,
  "refund_number" text not null,
  "order_id" uuid not null,
  "payment_id" uuid,
  "return_request_id" uuid,
  "amount_paise" bigint not null,
  "reason_code" text not null,
  "provider_refund_id" text,
  "status" text default 'PENDING' not null,
  "initiated_by" uuid not null,
  "approved_by" uuid,
  "initiated_at" timestamptz,
  "completed_at" timestamptz,
  "failure_message" text,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "refunds_refund_number_key" unique ("refund_number"),
  constraint "refunds_amount_paise_nonnegative" check ("amount_paise" is null or "amount_paise" >= 0),
  constraint "refunds_status_check" check ("status" in ('PENDING', 'APPROVAL_REQUIRED', 'INITIATED', 'PROCESSING', 'COMPLETED', 'FAILED', 'CANCELLED'))
);

comment on table public."refunds" is 'Gateway/manual refund record linked to order, payment and optional return.';

-- Finance: merchant_settlements
create table if not exists public."merchant_settlements" (
```

```
  "id" uuid default gen_random_uuid() not null primary key,
  "settlement_number" text not null,
  "shop_id" uuid not null,
  "bank_account_id" uuid not null,
  "period_start" date not null,
  "period_end" date not null,
  "gross_sales_paise" bigint default 0 not null,
  "commission_paise" bigint default 0 not null,
  "discount_share_paise" bigint default 0 not null,
  "refund_deduction_paise" bigint default 0 not null,
  "tax_paise" bigint default 0 not null,
  "adjustment_paise" bigint default 0 not null,
  "net_payable_paise" bigint default 0 not null,
  "status" text default 'DRAFT' not null,
  "provider_transfer_id" text,
  "scheduled_at" timestamptz,
  "paid_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "merchant_settlements_settlement_number_key" unique ("settlement_number"),
  constraint "merchant_settlements_status_check" check ("status" in ('DRAFT', 'REVIEW', 'APPROVED', 'PROCESSING', 'PAID', 'FAILED', 'ON_HOLD'))
);

comment on table public."merchant_settlements" is 'Periodic merchant payout batch with commissions, discount share, refunds and adjustments.';

-- Finance: merchant_settlement_items
create table if not exists public."merchant_settlement_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "settlement_id" uuid not null,
  "order_id" uuid,
  "refund_id" uuid,
  "entry_type" text not null,
  "amount_paise" bigint not null,
  "description" text,
  "created_at" timestamptz default now() not null,
  constraint "merchant_settlement_items_entry_type_check" check ("entry_type" in ('ORDER_CREDIT', 'COMMISSION', 'DISCOUNT', 'REFUND', 'TAX', 'PENALTY', 'MANUAL_ADJUSTMENT'))
);

comment on table public."merchant_settlement_items" is 'Settlement ledger rows linking orders, refunds and manual adjustments.';

-- Finance: captain_payouts
create table if not exists public."captain_payouts" (
  "id" uuid default gen_random_uuid() not null primary key,
  "payout_number" text not null,
  "captain_id" uuid not null,
  "period_start" date not null,
  "period_end" date not null,
  "earnings_paise" bigint default 0 not null,
  "incentives_paise" bigint default 0 not null,
  "penalties_paise" bigint default 0 not null,
  "cod_adjustment_paise" bigint default 0 not null,
  "net_payout_paise" bigint default 0 not null,
  "status" text default 'DRAFT' not null,
  "provider_transfer_id" text,
  "paid_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "captain_payouts_payout_number_key" unique ("payout_number"),
  constraint "captain_payouts_status_check" check ("status" in ('DRAFT', 'REVIEW', 'APPROVED', 'PROCESSING', 'PAID', 'FAILED', 'ON_HOLD'))
);

comment on table public."captain_payouts" is 'Periodic captain payout batch including earnings, incentives, penalties and COD adjustments.';

-- Finance: captain_payout_items
create table if not exists public."captain_payout_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "payout_id" uuid not null,
  "captain_earning_id" uuid,
  "entry_type" text not null,
  "amount_paise" bigint not null,
  "description" text,
  "created_at" timestamptz default now() not null,
  constraint "captain_payout_items_entry_type_check" check ("entry_type" in ('EARNING', 'INCENTIVE', 'PENALTY', 'COD_ADJUSTMENT', 'MANUAL_ADJUSTMENT'))
);

comment on table public."captain_payout_items" is 'Links a payout batch to individual captain earning records and adjustments.';
```

0005_logistics.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Logistics: captain_documents
create table if not exists public."captain_documents" (
  "id" uuid default gen_random_uuid() not null primary key,
  "captain_id" uuid not null,
  "document_type" text not null,
  "storage_path" text not null,
  "document_number_last4" text,
  "document_number_encrypted" text,
  "expiry_date" date,
  "verification_status" text default 'PENDING' not null,
  "verified_by" uuid,
  "verified_at" timestamptz,
  "rejection_reason" text,
  "created_at" timestamptz default now() not null,
  constraint "captain_documents_document_type_check" check ("document_type" in ('AADHAAR', 'PAN', 'DRIVING_LICENCE', 'VEHICLE_RC', 'INSURANCE', 'BANK_PROOF', 'PROFILE_PHOTO', 'OTHER')),
  constraint "captain_documents_verification_status_check" check ("verification_status" in ('PENDING', 'VERIFIED', 'REJECTED', 'EXPIRED'))
);

comment on table public."captain_documents" is 'Private KYC, licence, vehicle and bank documents for captains.';

-- Logistics: delivery_tasks
create table if not exists public."delivery_tasks" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_id" uuid,
  "return_request_id" uuid,
  "task_type" text default 'FORWARD_DELIVERY' not null,
  "pickup_shop_id" uuid,
  "pickup_address_snapshot" jsonb default '{}'::jsonb not null,
  "drop_address_snapshot" jsonb default '{}'::jsonb not null,
  "pickup_location" geography(Point,4326) not null,
  "drop_location" geography(Point,4326) not null,
  "status" text default 'CREATED' not null,
  "estimated_distance_meters" integer,
  "estimated_duration_seconds" integer,
  "delivery_fee_paise" bigint default 0 not null,
  "captain_earning_paise" bigint default 0 not null,
  "pickup_code_hash" text,
  "delivery_otp_hash" text,
  "assigned_captain_id" uuid,
  "assignment_attempts" smallint default 0 not null,
  "scheduled_at" timestamptz,
  "assigned_at" timestamptz,
  "picked_up_at" timestamptz,
  "completed_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "delivery_tasks_task_type_check" check ("task_type" in ('FORWARD_DELIVERY', 'RETURN_PICKUP', 'RETURN_TO_MERCHANT', 'EXCHANGE_DELIVERY')),
  constraint "delivery_tasks_status_check" check ("status" in ('CREATED', 'SEARCHING', 'OFFERED', 'ASSIGNED', 'AT_PICKUP', 'PICKED_UP', 'IN_TRANSIT', 'AT_DROP', 'COMPLETED', 'FAILED', 'CANCELLED')),
  constraint "delivery_tasks_estimated_distance_meters_nonnegative" check ("estimated_distance_meters" is null or "estimated_distance_meters" >= 0),
  constraint "delivery_tasks_estimated_duration_seconds_nonnegative" check ("estimated_duration_seconds" is null or "estimated_duration_seconds" >= 0),
  constraint "delivery_tasks_delivery_fee_paise_nonnegative" check ("delivery_fee_paise" is null or "delivery_fee_paise" >= 0),
  constraint "delivery_tasks_captain_earning_paise_nonnegative" check ("captain_earning_paise" is null or "captain_earning_paise" >= 0),
  constraint "delivery_tasks_assignment_attempts_nonnegative" check ("assignment_attempts" is null or "assignment_attempts" >= 0)
);

comment on table public."delivery_tasks" is 'Reusable delivery job for forward delivery, return pickup, return-to-merchant and later exchange flows.';

-- Logistics: delivery_assignments
create table if not exists public."delivery_assignments" (
  "id" uuid default gen_random_uuid() not null primary key,
  "delivery_task_id" uuid not null,
  "captain_id" uuid not null,
  "assignment_status" text default 'OFFERED' not null,
  "offered_earning_paise" bigint default 0 not null,
  "pickup_distance_meters" integer,
  "offered_at" timestamptz default now() not null,
  "expires_at" timestamptz not null,
  "responded_at" timestamptz,
  "rejection_reason" text,
```

```
  "assigned_by" text default 'AUTO' not null,
  "assigned_by_user_id" uuid,
  "created_at" timestamptz default now() not null,
  constraint "delivery_assignments_assignment_status_check" check ("assignment_status" in ('OFFERED', 'ACCEPTED', 'REJECTED', 'TIMED_OUT', 'CANCELLED', 'RELEASED', 'COMPLETED')),
  constraint "delivery_assignments_offered_earning_paise_nonnegative" check ("offered_earning_paise" is null or "offered_earning_paise" >= 0),
  constraint "delivery_assignments_pickup_distance_meters_nonnegative" check ("pickup_distance_meters" is null or "pickup_distance_meters" >= 0),
  constraint "delivery_assignments_assigned_by_check" check ("assigned_by" in ('AUTO', 'ADMIN'))
);

comment on table public."delivery_assignments" is 'Complete dispatch offer history including rejections, timeouts and reassignments.';

-- Logistics: captain_current_locations
create table if not exists public."captain_current_locations" (
  "captain_id" uuid not null primary key,
  "location" geography(Point,4326) not null,
  "heading" numeric(6,2),
  "speed_mps" numeric(8,2),
  "accuracy_meters" numeric(8,2),
  "battery_percent" smallint,
  "active_delivery_task_id" uuid,
  "updated_at" timestamptz default now() not null,
  constraint "captain_current_locations_speed_mps_nonnegative" check ("speed_mps" is null or "speed_mps" >= 0),
  constraint "captain_current_locations_accuracy_meters_nonnegative" check ("accuracy_meters" is null or "accuracy_meters" >= 0),
  constraint "captain_current_locations_battery_percent_check" check (battery_percent between 0 and 100)
);

comment on table public."captain_current_locations" is 'Latest captain location for dispatch and customer tracking; hot copies may also live in Redis.';

-- Logistics: captain_location_history
create table if not exists public."captain_location_history" (
  "id" bigint default generated always as identity not null primary key,
  "captain_id" uuid not null,
  "delivery_task_id" uuid,
  "location" geography(Point,4326) not null,
  "heading" numeric(6,2),
  "speed_mps" numeric(8,2),
  "accuracy_meters" numeric(8,2),
  "recorded_at" timestamptz default now() not null,
  "received_at" timestamptz default now() not null
);

comment on table public."captain_location_history" is 'Durable sampled location trail for active deliveries and dispute investigation. Partition monthly.';

-- Logistics: delivery_events
create table if not exists public."delivery_events" (
  "id" bigint default generated always as identity not null primary key,
  "delivery_task_id" uuid not null,
  "event_type" text not null,
  "actor_user_id" uuid,
  "location" geography(Point,4326),
  "note" text,
  "evidence_path" text,
  "metadata" jsonb default '{}'::jsonb not null,
  "created_at" timestamptz default now() not null,
  constraint "delivery_events_event_type_check" check ("event_type" in ('CAPTAIN_ASSIGNED', 'ARRIVED_AT_STORE', 'MERCHANT_DELAY', 'PICKUP_CONFIRMED', 'LEFT_STORE', 'ARRIVED_AT_CUSTOMER',
'DELIVERY_CONFIRMED', 'CUSTOMER_UNAVAILABLE', 'INVALID_ADDRESS', 'CUSTOMER_REFUSED', 'PACKAGE_DAMAGED', 'RETURNING_TO_STORE', 'TASK_COMPLETED', 'OTHER'))
);

comment on table public."delivery_events" is 'Append-only operational event log for pickup, delays, failures, arrival and completion.';

-- Logistics: cod_collections
create table if not exists public."cod_collections" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_id" uuid not null,
  "delivery_task_id" uuid not null,
  "captain_id" uuid not null,
  "amount_paise" bigint not null,
  "status" text default 'PENDING_COLLECTION' not null,
  "collected_at" timestamptz,
  "deposited_at" timestamptz,
  "reconciled_by" uuid,
  "reconciled_at" timestamptz,
  "created_at" timestamptz default now() not null,
  constraint "cod_collections_rule_1" unique (order_id),
  constraint "cod_collections_amount_paise_nonnegative" check ("amount_paise" is null or "amount_paise" >= 0),
  constraint "cod_collections_status_check" check ("status" in ('PENDING_COLLECTION', 'COLLECTED', 'DEPOSIT_PENDING', 'DEPOSITED', 'RECONCILED', 'DISPUTED'))
);
```

```
comment on table public."cod_collections" is 'Cash-on-delivery collection and reconciliation record.';

-- Logistics: captain_earnings
create table if not exists public."captain_earnings" (
  "id" uuid default gen_random_uuid() not null primary key,
  "captain_id" uuid not null,
  "delivery_task_id" uuid not null,
  "base_fare_paise" bigint default 0 not null,
  "distance_fare_paise" bigint default 0 not null,
  "waiting_fee_paise" bigint default 0 not null,
  "peak_incentive_paise" bigint default 0 not null,
  "other_incentive_paise" bigint default 0 not null,
  "tip_paise" bigint default 0 not null,
  "penalty_paise" bigint default 0 not null,
  "total_paise" bigint default 0 not null,
  "status" text default 'PENDING' not null,
  "created_at" timestamptz default now() not null,
  constraint "captain_earnings_rule_1" unique (delivery_task_id),
  constraint "captain_earnings_status_check" check ("status" in ('PENDING', 'AVAILABLE', 'INCLUDED_IN_PAYOUT', 'PAID', 'REVERSED'))
);

comment on table public."captain_earnings" is 'Per-task captain earning breakdown.';
```

0006_group_style_engagement.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Engagement: shop_favourites
create table if not exists public."shop_favourites" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "shop_id" uuid not null,
  "source" text default 'SHOP_PAGE' not null,
  "new_arrival_notifications" boolean default true not null,
  "offer_notifications" boolean default true not null,
  "restock_notifications" boolean default true not null,
  "favourited_at" timestamptz default now() not null,
  "unfavourited_at" timestamptz,
  constraint "shop_favourites_rule_1" unique (customer_id, shop_id),
  constraint "shop_favourites_source_check" check ("source" in ('ORGANIC_SEARCH', 'SHOP_PAGE', 'MERCHANT_REQUEST', 'ORDER_HISTORY', 'PRODUCT_PAGE', 'CAMPAIGN'))
);

comment on table public."shop_favourites" is 'Customer-to-shop follow/favourite relationship created voluntarily after search, visit or merchant request.';

-- Engagement: wishlist_items
create table if not exists public."wishlist_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "product_id" uuid not null,
  "variant_id" uuid,
  "created_at" timestamptz default now() not null,
  constraint "wishlist_items_rule_1" unique (customer_id, product_id, variant_id)
);

comment on table public."wishlist_items" is 'Customer product/variant wishlist.';

-- Group Style: style_groups
create table if not exists public."style_groups" (
  "id" uuid default gen_random_uuid() not null primary key,
  "creator_customer_id" uuid not null,
  "name" text not null,
  "occasion_type" text not null,
  "custom_occasion_name" text,
  "event_start_date" date,
  "event_end_date" date,
  "location_name" text,
  "event_location" geography(Point,4326),
  "budget_min_paise" bigint,
  "budget_max_paise" bigint,
  "coordination_mode" text default 'COORDINATED_LOOKS' not null,
  "include_footwear" boolean default true not null,
  "include_accessories" boolean default true not null,
  "status" text default 'PLANNING' not null,
  "invite_code" text not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "style_groups_invite_code_key" unique ("invite_code"),
  constraint "style_groups_occasion_type_check" check ("occasion_type" in ('WEDDING', 'PARTY', 'TRIP', 'FESTIVAL', 'GET_TOGETHER', 'COLLEGE_EVENT', 'OFFICE_EVENT', 'PHOTOSHOOT', 'CUSTOM')),
  constraint "style_groups_budget_min_paise_nonnegative" check ("budget_min_paise" is null or "budget_min_paise" >= 0),
  constraint "style_groups_budget_max_paise_nonnegative" check ("budget_max_paise" is null or "budget_max_paise" >= 0),
  constraint "style_groups_coordination_mode_check" check ("coordination_mode" in ('SAME_OUTFIT', 'SAME_COLOUR', 'SAME_COLOUR_FAMILY', 'SAME_THEME', 'MATCHING_ACCESSORIES', 'COORDINATED_LOOKS')),
  constraint "style_groups_status_check" check ("status" in ('PLANNING', 'VOTING', 'FINALIZED', 'ORDERING', 'COMPLETED', 'ARCHIVED'))
);

comment on table public."style_groups" is 'Occasion-planning group for wedding, party, trip, festival, get-together and custom events.';

-- Group Style: style_group_members
create table if not exists public."style_group_members" (
  "id" uuid default gen_random_uuid() not null primary key,
  "group_id" uuid not null,
  "customer_id" uuid,
  "invited_phone" text,
  "display_name" text not null,
  "event_role" text,
  "membership_role" text default 'MEMBER' not null,
  "status" text default 'INVITED' not null,
```

```
  "joined_at" timestamptz,
  "created_at" timestamptz default now() not null,
  constraint "style_group_members_rule_1" unique (group_id, customer_id),
  constraint "style_group_members_membership_role_check" check ("membership_role" in ('ADMIN', 'MEMBER')),
  constraint "style_group_members_status_check" check ("status" in ('INVITED', 'JOINED', 'DECLINED', 'REMOVED', 'LEFT'))
);

comment on table public."style_group_members" is 'Invited and joined participants, including event roles such as bride, groom, friends or family.';

-- Group Style: style_group_member_preferences
create table if not exists public."style_group_member_preferences" (
  "id" uuid default gen_random_uuid() not null primary key,
  "group_member_id" uuid not null,
  "clothing_size" text,
  "footwear_size" text,
  "preferred_colours" text[] default '{}'::text[] not null,
  "avoided_colours" text[] default '{}'::text[] not null,
  "preferred_styles" text[] default '{}'::text[] not null,
  "budget_min_paise" bigint,
  "budget_max_paise" bigint,
  "include_footwear" boolean default true not null,
  "include_accessories" boolean default true not null,
  "private_body_profile_id" uuid,
  "updated_at" timestamptz default now() not null,
  constraint "style_group_member_preferences_rule_1" unique (group_member_id),
  constraint "style_group_member_preferences_budget_min_paise_nonnegative" check ("budget_min_paise" is null or "budget_min_paise" >= 0),
  constraint "style_group_member_preferences_budget_max_paise_nonnegative" check ("budget_max_paise" is null or "budget_max_paise" >= 0)
);

comment on table public."style_group_member_preferences" is 'Private size, footwear, budget and style preferences for one group member.';

-- Group Style: style_themes
create table if not exists public."style_themes" (
  "id" uuid default gen_random_uuid() not null primary key,
  "group_id" uuid not null,
  "name" text not null,
  "primary_colours" text[] default '{}'::text[] not null,
  "secondary_colours" text[] default '{}'::text[] not null,
  "style_tags" text[] default '{}'::text[] not null,
  "material_preferences" text[] default '{}'::text[] not null,
  "formality_level" text,
  "weather_context" jsonb default '{}'::jsonb not null,
  "created_by" uuid not null,
  "is_selected" boolean default false not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null
);

comment on table public."style_themes" is 'Colour palette, visual style, material and formality rules selected by a group.';

-- Group Style: style_polls
create table if not exists public."style_polls" (
  "id" uuid default gen_random_uuid() not null primary key,
  "group_id" uuid not null,
  "question" text not null,
  "poll_type" text not null,
  "allows_multiple" boolean default false not null,
  "closes_at" timestamptz,
  "status" text default 'OPEN' not null,
  "created_by" uuid not null,
  "created_at" timestamptz default now() not null,
  constraint "style_polls_poll_type_check" check ("poll_type" in ('COLOUR', 'THEME', 'PRODUCT', 'MERCHANT', 'BUDGET', 'CUSTOM')),
  constraint "style_polls_status_check" check ("status" in ('OPEN', 'CLOSED', 'CANCELLED'))
);

comment on table public."style_polls" is 'Group polls for colours, themes, products, merchants or budgets.';

-- Group Style: style_poll_options
create table if not exists public."style_poll_options" (
  "id" uuid default gen_random_uuid() not null primary key,
  "poll_id" uuid not null,
  "label" text not null,
  "product_id" uuid,
  "theme_id" uuid,
  "image_path" text,
  "metadata" jsonb default '{}'::jsonb not null,
  "display_order" integer default 0 not null,
  "created_at" timestamptz default now() not null
```

```
);

comment on table public."style_poll_options" is 'Selectable poll options that can reference a theme, product or free text.';

-- Group Style: style_poll_votes
create table if not exists public."style_poll_votes" (
  "id" uuid default gen_random_uuid() not null primary key,
  "poll_id" uuid not null,
  "option_id" uuid not null,
  "group_member_id" uuid not null,
  "created_at" timestamptz default now() not null,
  constraint "style_poll_votes_rule_1" unique (poll_id, option_id, group_member_id)
);

comment on table public."style_poll_votes" is 'One member vote per poll option, constrained by poll rules.';

-- Group Style: style_moodboards
create table if not exists public."style_moodboards" (
  "id" uuid default gen_random_uuid() not null primary key,
  "group_id" uuid not null,
  "theme_id" uuid,
  "name" text not null,
  "cover_image_path" text,
  "status" text default 'DRAFT' not null,
  "created_by" uuid not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "style_moodboards_status_check" check ("status" in ('DRAFT', 'SHARED', 'FINAL'))
);

comment on table public."style_moodboards" is 'Shareable group mood board containing colour palette, member looks and accessories.';

-- Group Style: style_moodboard_items
create table if not exists public."style_moodboard_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "moodboard_id" uuid not null,
  "group_member_id" uuid,
  "product_id" uuid,
  "variant_id" uuid,
  "item_type" text not null,
  "item_role" text,
  "position_data" jsonb default '{}'::jsonb not null,
  "created_at" timestamptz default now() not null,
  constraint "style_moodboard_items_item_type_check" check ("item_type" in ('PRODUCT', 'COLOUR_SWATCH', 'TEXT', 'IMAGE'))
);

comment on table public."style_moodboard_items" is 'Products, palettes and member recommendation cards positioned on a mood board.';

-- Notifications: notifications
create table if not exists public."notifications" (
  "id" uuid default gen_random_uuid() not null primary key,
  "user_id" uuid not null,
  "notification_type" text not null,
  "title" text not null,
  "body" text not null,
  "entity_type" text,
  "entity_id" uuid,
  "priority" text default 'NORMAL' not null,
  "data" jsonb default '{}'::jsonb not null,
  "created_at" timestamptz default now() not null,
  "read_at" timestamptz,
  constraint "notifications_priority_check" check ("priority" in ('LOW', 'NORMAL', 'HIGH', 'URGENT'))
);

comment on table public."notifications" is 'In-app notification record for customer, merchant, captain and admin users.';

-- Notifications: notification_deliveries
create table if not exists public."notification_deliveries" (
  "id" bigint default generated always as identity not null primary key,
  "notification_id" uuid not null,
  "device_id" uuid,
  "channel" text not null,
  "provider" text,
  "provider_message_id" text,
  "status" text default 'PENDING' not null,
  "attempt_count" smallint default 0 not null,
  "sent_at" timestamptz,
  "received_at" timestamptz,
```

```
  "failed_reason" text,
  "created_at" timestamptz default now() not null,
  constraint "notification_deliveries_channel_check" check ("channel" in ('PUSH', 'SMS', 'WHATSAPP', 'EMAIL', 'IN_APP')),
  constraint "notification_deliveries_status_check" check ("status" in ('PENDING', 'SENT', 'DELIVERED', 'READ', 'FAILED')),
  constraint "notification_deliveries_attempt_count_nonnegative" check ("attempt_count" is null or "attempt_count" >= 0)
);

comment on table public."notification_deliveries" is 'Channel-specific send, retry and delivery receipt history.';

-- Notifications: notification_preferences
create table if not exists public."notification_preferences" (
  "user_id" uuid not null primary key,
  "order_updates" boolean default true not null,
  "delivery_updates" boolean default true not null,
  "shop_offers" boolean default true not null,
  "new_arrivals" boolean default true not null,
  "recommendations" boolean default true not null,
  "group_updates" boolean default true not null,
  "support_updates" boolean default true not null,
  "quiet_hours_start" time,
  "quiet_hours_end" time,
  "updated_at" timestamptz default now() not null
);

comment on table public."notification_preferences" is 'Global non-transactional notification preferences. Critical order and security alerts are controlled separately.';

-- Engagement: reviews
create table if not exists public."reviews" (
  "id" uuid default gen_random_uuid() not null primary key,
  "order_id" uuid not null,
  "customer_id" uuid not null,
  "review_type" text not null,
  "shop_id" uuid,
  "product_id" uuid,
  "captain_id" uuid,
  "rating" smallint not null,
  "review_text" text,
  "moderation_status" text default 'PUBLISHED' not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "reviews_review_type_check" check ("review_type" in ('PRODUCT', 'SHOP', 'CAPTAIN')),
  constraint "reviews_rating_check" check (rating between 1 and 5),
  constraint "reviews_moderation_status_check" check ("moderation_status" in ('PUBLISHED', 'PENDING', 'HIDDEN', 'REJECTED'))
);

comment on table public."reviews" is 'Verified-order reviews for product, shop and captain.';
```

0007_intelligence_ai.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Intelligence: customer_events
create table if not exists public."customer_events" (
  "id" bigint default generated always as identity not null primary key,
  "customer_id" uuid,
  "anonymous_id" uuid,
  "session_id" uuid not null,
  "app_name" text default 'CUSTOMER' not null,
  "event_type" text not null,
  "entity_type" text,
  "entity_id" uuid,
  "shop_id" uuid,
  "query_text" text,
  "context" jsonb default '{}'::jsonb not null,
  "occurred_at" timestamptz default now() not null,
  "received_at" timestamptz default now() not null,
  constraint "customer_events_event_type_check" check ("event_type" in ('PRODUCT_VIEW', 'PRODUCT_CLICK', 'SEARCH', 'SHOP_VIEW', 'FAVOURITE_SHOP', 'WISHLIST_ADD', 'CART_ADD', 'PURCHASE', 'RETURN',
'GROUP_SELECTION', 'FIT_FEEDBACK'))
);

comment on table public."customer_events" is 'High-volume behavioural event stream for recommendation training and analytics.';

-- Intelligence: recommendation_sets
create table if not exists public."recommendation_sets" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid,
  "session_id" uuid,
  "recommendation_type" text not null,
  "context" jsonb default '{}'::jsonb not null,
  "algorithm_version" text default 'rules-v1' not null,
  "generated_at" timestamptz default now() not null,
  "expires_at" timestamptz,
  constraint "recommendation_sets_recommendation_type_check" check ("recommendation_type" in ('HOME_PERSONALIZED', 'SIMILAR_PRODUCTS', 'COMPLETE_THE_LOOK', 'TRENDING_NEARBY', 'BASED_ON_OCCASION',
'BASED_ON_FAVOURITE_SHOP', 'GROUP_STYLE'))
);

comment on table public."recommendation_sets" is 'One generated ranking for a customer and context.';

-- Intelligence: recommendation_items
create table if not exists public."recommendation_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "recommendation_set_id" uuid not null,
  "product_id" uuid not null,
  "variant_id" uuid,
  "rank_position" integer not null,
  "score" numeric(10,8) not null,
  "reason_codes" text[] default '{}'::text[] not null,
  "score_components" jsonb default '{}'::jsonb not null,
  "shown_at" timestamptz,
  "clicked_at" timestamptz,
  "purchased_at" timestamptz,
  "created_at" timestamptz default now() not null,
  constraint "recommendation_items_rule_1" unique (recommendation_set_id, rank_position),
  constraint "recommendation_items_rank_position_nonnegative" check ("rank_position" is null or "rank_position" >= 0)
);

comment on table public."recommendation_items" is 'Ranked recommendation output with explanation and engagement timestamps.';

-- Intelligence: product_embeddings
create table if not exists public."product_embeddings" (
  "product_id" uuid not null primary key,
  "embedding_type" text default 'MULTIMODAL' not null,
  "embedding" vector(768) not null,
  "model_version" text not null,
  "source_hash" text not null,
  "updated_at" timestamptz default now() not null
);

comment on table public."product_embeddings" is 'Vector representation for image/text similarity and outfit compatibility.';
```

```
-- Intelligence: customer_embeddings
create table if not exists public."customer_embeddings" (
  "customer_id" uuid not null primary key,
  "embedding" vector(768) not null,
  "model_version" text not null,
  "source_event_count" integer default 0 not null,
  "updated_at" timestamptz default now() not null,
  constraint "customer_embeddings_source_event_count_nonnegative" check ("source_event_count" is null or "source_event_count" >= 0)
);

comment on table public."customer_embeddings" is 'Learned customer preference vector for ML ranking.';

-- Intelligence: size_charts
create table if not exists public."size_charts" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_id" uuid,
  "brand" text,
  "category_id" uuid not null,
  "gender_category" text default 'UNISEX' not null,
  "country_standard" text default 'IN' not null,
  "name" text not null,
  "version" integer default 1 not null,
  "is_active" boolean default true not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "size_charts_version_nonnegative" check ("version" is null or "version" >= 0)
);

comment on table public."size_charts" is 'Brand/shop/category size-chart version.';

-- Intelligence: size_chart_measurements
create table if not exists public."size_chart_measurements" (
  "id" uuid default gen_random_uuid() not null primary key,
  "size_chart_id" uuid not null,
  "size_label" text not null,
  "measurement_name" text not null,
  "min_cm" numeric(8,2),
  "max_cm" numeric(8,2),
  "garment_measurement_cm" numeric(8,2),
  "tolerance_cm" numeric(8,2),
  "created_at" timestamptz default now() not null,
  constraint "size_chart_measurements_rule_1" unique (size_chart_id, size_label, measurement_name),
  constraint "size_chart_measurements_min_cm_nonnegative" check ("min_cm" is null or "min_cm" >= 0),
  constraint "size_chart_measurements_max_cm_nonnegative" check ("max_cm" is null or "max_cm" >= 0),
  constraint "size_chart_measurements_garment_measurement_cm_nonnegative" check ("garment_measurement_cm" is null or "garment_measurement_cm" >= 0),
  constraint "size_chart_measurements_tolerance_cm_nonnegative" check ("tolerance_cm" is null or "tolerance_cm" >= 0)
);

comment on table public."size_chart_measurements" is 'Measurement ranges and garment dimensions for each label in a size chart.';

-- Intelligence: customer_body_profiles
create table if not exists public."customer_body_profiles" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "profile_name" text default 'My Fit Profile' not null,
  "source" text default 'MANUAL_ENTRY' not null,
  "fit_preference" text default 'REGULAR' not null,
  "height_cm" numeric(8,2),
  "weight_kg" numeric(8,2),
  "consent_version" text not null,
  "consent_given_at" timestamptz not null,
  "is_active" boolean default true not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  "deleted_at" timestamptz,
  constraint "customer_body_profiles_source_check" check ("source" in ('MANUAL_ENTRY', 'CAMERA_SCAN', 'PURCHASE_HISTORY')),
  constraint "customer_body_profiles_fit_preference_check" check ("fit_preference" in ('SLIM', 'REGULAR', 'RELAXED', 'OVERSIZED')),
  constraint "customer_body_profiles_height_cm_nonnegative" check ("height_cm" is null or "height_cm" >= 0),
  constraint "customer_body_profiles_weight_kg_nonnegative" check ("weight_kg" is null or "weight_kg" >= 0)
);

comment on table public."customer_body_profiles" is 'Consent-controlled body profile for manual measurements, camera scan or learned fit history.';

-- Intelligence: body_measurements
create table if not exists public."body_measurements" (
  "id" uuid default gen_random_uuid() not null primary key,
  "body_profile_id" uuid not null,
  "measurement_name" text not null,
```

```
  "value_cm" numeric(8,2) not null,
  "confidence_score" numeric(6,5),
  "source" text not null,
  "captured_at" timestamptz default now() not null,
  constraint "body_measurements_rule_1" unique (body_profile_id, measurement_name),
  constraint "body_measurements_value_cm_nonnegative" check ("value_cm" is null or "value_cm" >= 0),
  constraint "body_measurements_confidence_score_nonnegative" check ("confidence_score" is null or "confidence_score" >= 0),
  constraint "body_measurements_source_check" check ("source" in ('USER_ENTERED', 'CAMERA_ESTIMATED', 'PURCHASE_LEARNED'))
);

comment on table public."body_measurements" is 'Individual measurements with confidence and source.';

-- Intelligence: body_scan_sessions
create table if not exists public."body_scan_sessions" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "body_profile_id" uuid,
  "status" text default 'CREATED' not null,
  "front_image_path" text,
  "side_image_path" text,
  "back_image_path" text,
  "processing_provider" text,
  "model_version" text,
  "quality_score" numeric(6,5),
  "quality_issues" text[] default '{}'::text[] not null,
  "error_message" text,
  "deletion_scheduled_at" timestamptz,
  "created_at" timestamptz default now() not null,
  "completed_at" timestamptz,
  constraint "body_scan_sessions_status_check" check ("status" in ('CREATED', 'UPLOADING', 'QUALITY_CHECK', 'PROCESSING', 'COMPLETED', 'FAILED', 'DELETED')),
  constraint "body_scan_sessions_quality_score_check" check (quality_score between 0 and 1)
);

comment on table public."body_scan_sessions" is 'Privacy-sensitive guided body scan processing session. Raw images should be deleted by default after extraction.';

-- Intelligence: fit_recommendations
create table if not exists public."fit_recommendations" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "body_profile_id" uuid,
  "product_id" uuid not null,
  "variant_id" uuid,
  "recommended_size" text not null,
  "confidence_score" numeric(6,5) not null,
  "fit_prediction" text default 'EXPECTED_FIT' not null,
  "reason_codes" text[] default '{}'::text[] not null,
  "inputs_snapshot" jsonb default '{}'::jsonb not null,
  "model_version" text default 'rules-v1' not null,
  "created_at" timestamptz default now() not null,
  constraint "fit_recommendations_confidence_score_check" check (confidence_score between 0 and 1),
  constraint "fit_recommendations_fit_prediction_check" check ("fit_prediction" in ('TOO_SMALL', 'SLIGHTLY_SMALL', 'EXPECTED_FIT', 'SLIGHTLY_LARGE', 'TOO_LARGE'))
);

comment on table public."fit_recommendations" is 'Per-product size recommendation, confidence, explanation and model version.';

-- Intelligence: fit_feedback
create table if not exists public."fit_feedback" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "order_item_id" uuid not null,
  "fit_recommendation_id" uuid,
  "predicted_size" text,
  "purchased_size" text not null,
  "feedback" text not null,
  "returned_due_to_fit" boolean default false not null,
  "created_at" timestamptz default now() not null,
  constraint "fit_feedback_rule_1" unique (order_item_id),
  constraint "fit_feedback_feedback_check" check ("feedback" in ('TOO_SMALL', 'SLIGHTLY_SMALL', 'PERFECT', 'SLIGHTLY_LARGE', 'TOO_LARGE'))
);

comment on table public."fit_feedback" is 'Post-delivery fit outcome used to improve brand/product sizing.';

-- Intelligence: virtual_tryon_sessions
create table if not exists public."virtual_tryon_sessions" (
  "id" uuid default gen_random_uuid() not null primary key,
  "customer_id" uuid not null,
  "product_id" uuid not null,
  "variant_id" uuid,
```

```
  "input_type" text not null,
  "customer_image_path" text,
  "body_profile_id" uuid,
  "status" text default 'CREATED' not null,
  "provider" text,
  "model_version" text,
  "consent_version" text not null,
  "deletion_scheduled_at" timestamptz,
  "error_message" text,
  "created_at" timestamptz default now() not null,
  "completed_at" timestamptz,
  constraint "virtual_tryon_sessions_input_type_check" check ("input_type" in ('UPLOADED_PHOTO', 'SAVED_BODY_PROFILE', 'GENERIC_MODEL')),
  constraint "virtual_tryon_sessions_status_check" check ("status" in ('CREATED', 'PROCESSING', 'COMPLETED', 'FAILED', 'DELETED'))
);

comment on table public."virtual_tryon_sessions" is 'Photo/avatar virtual try-on job with provider/model, consent and retention state.';

-- Intelligence: virtual_tryon_outputs
create table if not exists public."virtual_tryon_outputs" (
  "id" uuid default gen_random_uuid() not null primary key,
  "tryon_session_id" uuid not null,
  "view_type" text default 'FRONT' not null,
  "output_image_path" text not null,
  "quality_score" numeric(6,5),
  "is_saved_by_customer" boolean default false not null,
  "created_at" timestamptz default now() not null,
  constraint "virtual_tryon_outputs_view_type_check" check ("view_type" in ('FRONT', 'SIDE', 'BACK', 'GROUP_MOODBOARD')),
  constraint "virtual_tryon_outputs_quality_score_check" check (quality_score between 0 and 1)
);

comment on table public."virtual_tryon_outputs" is 'Generated front/side/back try-on output references.';

-- Intelligence: style_recommendation_sets
create table if not exists public."style_recommendation_sets" (
  "id" uuid default gen_random_uuid() not null primary key,
  "group_id" uuid not null,
  "theme_id" uuid,
  "algorithm_version" text default 'rules-v1' not null,
  "total_score" numeric(10,8) default 0 not null,
  "score_components" jsonb default '{}'::jsonb not null,
  "status" text default 'GENERATED' not null,
  "created_at" timestamptz default now() not null,
  constraint "style_recommendation_sets_status_check" check ("status" in ('GENERATED', 'SHORTLISTED', 'SELECTED', 'EXPIRED'))
);

comment on table public."style_recommendation_sets" is 'Scored coordinated set for a group theme and members.';

-- Intelligence: style_recommendation_items
create table if not exists public."style_recommendation_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "recommendation_set_id" uuid not null,
  "group_member_id" uuid not null,
  "product_id" uuid not null,
  "variant_id" uuid,
  "item_role" text not null,
  "compatibility_score" numeric(10,8) default 0 not null,
  "availability_score" numeric(10,8) default 0 not null,
  "reason_codes" text[] default '{}'::text[] not null,
  "created_at" timestamptz default now() not null
);

comment on table public."style_recommendation_items" is 'Member-level clothing, footwear and accessory products in a coordinated group set.';
```

0008_operations.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Operations: support_tickets
create table if not exists public."support_tickets" (
  "id" uuid default gen_random_uuid() not null primary key,
  "ticket_number" text not null,
  "raised_by_user_id" uuid not null,
  "raised_by_type" text not null,
  "order_id" uuid,
  "shop_id" uuid,
  "delivery_task_id" uuid,
  "return_request_id" uuid,
  "category" text not null,
  "priority" text default 'MEDIUM' not null,
  "status" text default 'OPEN' not null,
  "subject" text not null,
  "description" text not null,
  "assigned_team" text,
  "assigned_to" uuid,
  "resolution_code" text,
  "resolution_note" text,
  "created_at" timestamptz default now() not null,
  "first_response_at" timestamptz,
  "resolved_at" timestamptz,
  "closed_at" timestamptz,
  "updated_at" timestamptz default now() not null,
  constraint "support_tickets_ticket_number_key" unique ("ticket_number"),
  constraint "support_tickets_raised_by_type_check" check ("raised_by_type" in ('CUSTOMER', 'MERCHANT', 'CAPTAIN', 'ADMIN')),
  constraint "support_tickets_priority_check" check ("priority" in ('LOW', 'MEDIUM', 'HIGH', 'URGENT')),
  constraint "support_tickets_status_check" check ("status" in ('OPEN', 'ASSIGNED', 'IN_PROGRESS', 'WAITING_FOR_USER', 'ESCALATED', 'RESOLVED', 'CLOSED'))
);

comment on table public."support_tickets" is 'Unified support ticket for customer, merchant, captain and internal issues.';

-- Operations: support_messages
create table if not exists public."support_messages" (
  "id" bigint default generated always as identity not null primary key,
  "ticket_id" uuid not null,
  "sender_id" uuid not null,
  "message_type" text default 'TEXT' not null,
  "message" text,
  "attachment_path" text,
  "is_internal_note" boolean default false not null,
  "created_at" timestamptz default now() not null,
  constraint "support_messages_message_type_check" check ("message_type" in ('TEXT', 'IMAGE', 'FILE', 'SYSTEM'))
);

comment on table public."support_messages" is 'Ticket conversation, attachments and internal notes.';

-- Campaigns: campaigns
create table if not exists public."campaigns" (
  "id" uuid default gen_random_uuid() not null primary key,
  "name" text not null,
  "campaign_type" text not null,
  "objective" text,
  "audience_rules" jsonb default '{}'::jsonb not null,
  "budget_paise" bigint,
  "status" text default 'DRAFT' not null,
  "starts_at" timestamptz not null,
  "ends_at" timestamptz not null,
  "created_by" uuid not null,
  "approved_by" uuid,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "campaigns_campaign_type_check" check ("campaign_type" in ('BANNER', 'COUPON', 'FREE_DELIVERY', 'FESTIVAL', 'GROUP_STYLE', 'MERCHANT_SPONSORED')),
  constraint "campaigns_budget_paise_nonnegative" check ("budget_paise" is null or "budget_paise" >= 0),
  constraint "campaigns_status_check" check ("status" in ('DRAFT', 'APPROVAL_REQUIRED', 'SCHEDULED', 'ACTIVE', 'PAUSED', 'COMPLETED', 'CANCELLED'))
);

comment on table public."campaigns" is 'Admin-created banners, coupons, city promotions and Group Style campaigns.';
```

```
-- Campaigns: banners
create table if not exists public."banners" (
  "id" uuid default gen_random_uuid() not null primary key,
  "campaign_id" uuid not null,
  "mobile_image_path" text not null,
  "desktop_image_path" text,
  "title" text not null,
  "subtitle" text,
  "cta_text" text,
  "destination_type" text not null,
  "destination_id" uuid,
  "external_url" text,
  "priority" integer default 0 not null,
  "placement" text default 'HOME_TOP' not null,
  "impression_count" bigint default 0 not null,
  "click_count" bigint default 0 not null,
  "created_at" timestamptz default now() not null,
  constraint "banners_destination_type_check" check ("destination_type" in ('CATEGORY', 'PRODUCT', 'SHOP', 'COLLECTION', 'GROUP_STYLE', 'OFFER', 'URL'))
);

comment on table public."banners" is 'Responsive banner creative and destination shown on customer home/discovery surfaces.';

-- Field Operations: merchant_leads
create table if not exists public."merchant_leads" (
  "id" uuid default gen_random_uuid() not null primary key,
  "shop_name" text not null,
  "owner_name" text,
  "phone_number" text not null,
  "category_summary" text,
  "address_text" text,
  "location" geography(Point,4326),
  "lead_source" text default 'FIELD' not null,
  "lead_status" text default 'NEW' not null,
  "assigned_executive_id" uuid,
  "next_followup_at" timestamptz,
  "notes" text,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "merchant_leads_lead_status_check" check ("lead_status" in ('NEW', 'CONTACTED', 'INTERESTED', 'VISIT_SCHEDULED', 'ONBOARDING_STARTED', 'ACTIVE', 'NOT_INTERESTED', 'FOLLOW_UP'))
);

comment on table public."merchant_leads" is 'Prospective shop pipeline for field acquisition and onboarding.';

-- Field Operations: field_visits
create table if not exists public."field_visits" (
  "id" uuid default gen_random_uuid() not null primary key,
  "merchant_lead_id" uuid,
  "shop_id" uuid,
  "executive_id" uuid not null,
  "visit_type" text not null,
  "scheduled_at" timestamptz not null,
  "started_at" timestamptz,
  "completed_at" timestamptz,
  "start_location" geography(Point,4326),
  "completion_location" geography(Point,4326),
  "merchant_confirmation_method" text,
  "merchant_confirmation_at" timestamptz,
  "status" text default 'SCHEDULED' not null,
  "notes" text,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "field_visits_visit_type_check" check ("visit_type" in ('SALES', 'ONBOARDING', 'TRAINING', 'SUPPORT', 'INVENTORY_AUDIT', 'RETENTION')),
  constraint "field_visits_merchant_confirmation_method_check" check ("merchant_confirmation_method" is null or "merchant_confirmation_method" in ('OTP', 'SIGNATURE', 'APP_CONFIRMATION')),
  constraint "field_visits_status_check" check ("status" in ('SCHEDULED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED', 'RESCHEDULED'))
);

comment on table public."field_visits" is 'GPS and merchant-confirmed merchant onboarding/support visit.';

-- Field Operations: field_visit_checklist_items
create table if not exists public."field_visit_checklist_items" (
  "id" uuid default gen_random_uuid() not null primary key,
  "field_visit_id" uuid not null,
  "item_code" text not null,
  "label" text not null,
  "is_required" boolean default true not null,
  "is_completed" boolean default false not null,
  "evidence_path" text,
  "completed_by" uuid,
```

```
  "completed_at" timestamptz,
  "notes" text,
  constraint "field_visit_checklist_items_rule_1" unique (field_visit_id, item_code)
);

comment on table public."field_visit_checklist_items" is 'Structured onboarding/training/support checklist evidence.';

-- Moderation: product_moderation_cases
create table if not exists public."product_moderation_cases" (
  "id" uuid default gen_random_uuid() not null primary key,
  "product_id" uuid not null,
  "case_type" text not null,
  "status" text default 'OPEN' not null,
  "moderator_id" uuid,
  "merchant_response" text,
  "resolution_note" text,
  "created_at" timestamptz default now() not null,
  "resolved_at" timestamptz,
  constraint "product_moderation_cases_case_type_check" check ("case_type" in ('NEW_LISTING', 'IMAGE_QUALITY', 'WRONG_CATEGORY', 'DUPLICATE', 'MISLEADING_PRICE', 'PROHIBITED', 'CUSTOMER_REPORT')),
  constraint "product_moderation_cases_status_check" check ("status" in ('OPEN', 'ASSIGNED', 'MERCHANT_ACTION_REQUIRED', 'APPROVED', 'REJECTED', 'CLOSED'))
);

comment on table public."product_moderation_cases" is 'Product quality, duplicate, category, pricing and prohibited-content moderation case.';

-- Governance: approval_requests
create table if not exists public."approval_requests" (
  "id" uuid default gen_random_uuid() not null primary key,
  "action_type" text not null,
  "entity_type" text not null,
  "entity_id" uuid not null,
  "requested_by" uuid not null,
  "requested_changes" jsonb not null,
  "reason" text not null,
  "status" text default 'PENDING' not null,
  "approved_by" uuid,
  "decision_note" text,
  "created_at" timestamptz default now() not null,
  "decided_at" timestamptz,
  "executed_at" timestamptz,
  constraint "approval_requests_action_type_check" check ("action_type" in ('LARGE_REFUND', 'BANK_CHANGE', 'MERCHANT_SUSPENSION', 'CAPTAIN_SUSPENSION', 'SETTLEMENT_ADJUSTMENT', 'CAMPAIGN_BUDGET',
'OTHER')),
  constraint "approval_requests_status_check" check ("status" in ('PENDING', 'APPROVED', 'REJECTED', 'CANCELLED', 'EXECUTED'))
);

comment on table public."approval_requests" is 'Maker-checker workflow for large refunds, bank changes, suspensions, settlement adjustments and campaigns.';

-- Risk: fraud_cases
create table if not exists public."fraud_cases" (
  "id" uuid default gen_random_uuid() not null primary key,
  "case_number" text not null,
  "case_type" text not null,
  "subject_user_id" uuid,
  "shop_id" uuid,
  "order_id" uuid,
  "risk_score" numeric(6,3),
  "signals" jsonb default '{}'::jsonb not null,
  "status" text default 'OPEN' not null,
  "assigned_to" uuid,
  "action_taken" text,
  "notes" text,
  "created_at" timestamptz default now() not null,
  "resolved_at" timestamptz,
  constraint "fraud_cases_case_number_key" unique ("case_number"),
  constraint "fraud_cases_case_type_check" check ("case_type" in ('FAKE_ORDER', 'DUPLICATE_ACCOUNT', 'REFUND_ABUSE', 'COD_ABUSE', 'MERCHANT_MANIPULATION', 'CAPTAIN_FRAUD', 'ACCOUNT_TAKEOVER',
'OTHER')),
  constraint "fraud_cases_risk_score_check" check (risk_score between 0 and 100),
  constraint "fraud_cases_status_check" check ("status" in ('OPEN', 'INVESTIGATING', 'ACTION_REQUIRED', 'CLEARED', 'CONFIRMED', 'CLOSED'))
);

comment on table public."fraud_cases" is 'Investigation record for suspicious orders, referrals, refunds, COD, merchants or captains.';

-- Governance: audit_logs
create table if not exists public."audit_logs" (
  "id" bigint default generated always as identity not null primary key,
  "actor_user_id" uuid,
  "actor_role" text default 'SYSTEM' not null,
  "action" text not null,
  "entity_type" text not null,
```

```
  "entity_id" uuid,
  "old_values" jsonb,
  "new_values" jsonb,
  "reason" text,
  "ip_address" inet,
  "device_id" uuid,
  "request_id" uuid,
  "created_at" timestamptz default now() not null
);

comment on table public."audit_logs" is 'Append-only record of sensitive mutations and admin/backend actions. Partition monthly and never expose to clients.';

-- Configuration: system_settings
create table if not exists public."system_settings" (
  "id" uuid default gen_random_uuid() not null primary key,
  "setting_key" text not null,
  "setting_value" jsonb not null,
  "value_type" text not null,
  "scope_type" text default 'GLOBAL' not null,
  "scope_id" text,
  "is_secret" boolean default false not null,
  "version" integer default 1 not null,
  "updated_by" uuid not null,
  "updated_at" timestamptz default now() not null,
  constraint "system_settings_setting_key_key" unique ("setting_key"),
  constraint "system_settings_rule_1" unique (setting_key, scope_type, scope_id),
  constraint "system_settings_value_type_check" check ("value_type" in ('BOOLEAN', 'NUMBER', 'STRING', 'JSON')),
  constraint "system_settings_scope_type_check" check ("scope_type" in ('GLOBAL', 'CITY', 'SHOP')),
  constraint "system_settings_version_nonnegative" check ("version" is null or "version" >= 0)
);

comment on table public."system_settings" is 'Versioned platform configuration such as timeouts, limits and feature flags.';

-- Configuration: service_zones
create table if not exists public."service_zones" (
  "id" uuid default gen_random_uuid() not null primary key,
  "name" text not null,
  "city" text not null,
  "zone_type" text default 'DELIVERY' not null,
  "boundary" geography(Polygon,4326) not null,
  "is_active" boolean default true not null,
  "base_delivery_fee_paise" bigint default 0 not null,
  "per_km_fee_paise" bigint default 0 not null,
  "minimum_order_paise" bigint default 0 not null,
  "created_at" timestamptz default now() not null,
  "updated_at" timestamptz default now() not null,
  constraint "service_zones_zone_type_check" check ("zone_type" in ('DELIVERY', 'MERCHANT_ONBOARDING', 'CAPTAIN_OPERATIONS')),
  constraint "service_zones_base_delivery_fee_paise_nonnegative" check ("base_delivery_fee_paise" is null or "base_delivery_fee_paise" >= 0),
  constraint "service_zones_per_km_fee_paise_nonnegative" check ("per_km_fee_paise" is null or "per_km_fee_paise" >= 0),
  constraint "service_zones_minimum_order_paise_nonnegative" check ("minimum_order_paise" is null or "minimum_order_paise" >= 0)
);

comment on table public."service_zones" is 'Polygonal city/service areas used for customer serviceability and operational routing.';
```

0009_foreign_keys.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Foreign keys are applied after every table exists.

alter table public."profiles" drop constraint if exists "profiles_id_fkey";
alter table public."profiles" add constraint "profiles_id_fkey" foreign key ("id") references auth.users ("id") on update cascade on delete restrict;

alter table public."user_devices" drop constraint if exists "user_devices_user_id_fkey";
alter table public."user_devices" add constraint "user_devices_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."addresses" drop constraint if exists "addresses_user_id_fkey";
alter table public."addresses" add constraint "addresses_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."customer_profiles" drop constraint if exists "customer_profiles_user_id_fkey";
alter table public."customer_profiles" add constraint "customer_profiles_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."customer_profiles" drop constraint if exists "customer_profiles_default_address_id_fkey";
alter table public."customer_profiles" add constraint "customer_profiles_default_address_id_fkey" foreign key ("default_address_id") references public."addresses" ("id") on update cascade on delete
set null;

alter table public."customer_preferences" drop constraint if exists "customer_preferences_customer_id_fkey";
alter table public."customer_preferences" add constraint "customer_preferences_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete cascade;

alter table public."merchant_profiles" drop constraint if exists "merchant_profiles_user_id_fkey";
alter table public."merchant_profiles" add constraint "merchant_profiles_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."merchant_profiles" drop constraint if exists "merchant_profiles_approved_by_fkey";
alter table public."merchant_profiles" add constraint "merchant_profiles_approved_by_fkey" foreign key ("approved_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."captain_profiles" drop constraint if exists "captain_profiles_user_id_fkey";
alter table public."captain_profiles" add constraint "captain_profiles_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."admin_profiles" drop constraint if exists "admin_profiles_user_id_fkey";
alter table public."admin_profiles" add constraint "admin_profiles_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."admin_profiles" drop constraint if exists "admin_profiles_manager_id_fkey";
alter table public."admin_profiles" add constraint "admin_profiles_manager_id_fkey" foreign key ("manager_id") references public."admin_profiles" ("user_id") on update cascade on delete set null;

alter table public."user_roles" drop constraint if exists "user_roles_user_id_fkey";
alter table public."user_roles" add constraint "user_roles_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."user_roles" drop constraint if exists "user_roles_role_id_fkey";
alter table public."user_roles" add constraint "user_roles_role_id_fkey" foreign key ("role_id") references public."roles" ("id") on update cascade on delete restrict;

alter table public."user_roles" drop constraint if exists "user_roles_assigned_by_fkey";
alter table public."user_roles" add constraint "user_roles_assigned_by_fkey" foreign key ("assigned_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."role_permissions" drop constraint if exists "role_permissions_role_id_fkey";
alter table public."role_permissions" add constraint "role_permissions_role_id_fkey" foreign key ("role_id") references public."roles" ("id") on update cascade on delete restrict;

alter table public."role_permissions" drop constraint if exists "role_permissions_permission_id_fkey";
alter table public."role_permissions" add constraint "role_permissions_permission_id_fkey" foreign key ("permission_id") references public."permissions" ("id") on update cascade on delete restrict;

alter table public."shops" drop constraint if exists "shops_merchant_id_fkey";
alter table public."shops" add constraint "shops_merchant_id_fkey" foreign key ("merchant_id") references public."merchant_profiles" ("user_id") on update cascade on delete restrict;

alter table public."shop_hours" drop constraint if exists "shop_hours_shop_id_fkey";
alter table public."shop_hours" add constraint "shop_hours_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."shop_documents" drop constraint if exists "shop_documents_shop_id_fkey";
alter table public."shop_documents" add constraint "shop_documents_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."shop_documents" drop constraint if exists "shop_documents_uploaded_by_fkey";
alter table public."shop_documents" add constraint "shop_documents_uploaded_by_fkey" foreign key ("uploaded_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."shop_documents" drop constraint if exists "shop_documents_verified_by_fkey";
alter table public."shop_documents" add constraint "shop_documents_verified_by_fkey" foreign key ("verified_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."shop_bank_accounts" drop constraint if exists "shop_bank_accounts_shop_id_fkey";
alter table public."shop_bank_accounts" add constraint "shop_bank_accounts_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;
```

```
alter table public."categories" drop constraint if exists "categories_parent_id_fkey";
alter table public."categories" add constraint "categories_parent_id_fkey" foreign key ("parent_id") references public."categories" ("id") on update cascade on delete set null;

alter table public."products" drop constraint if exists "products_shop_id_fkey";
alter table public."products" add constraint "products_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."products" drop constraint if exists "products_category_id_fkey";
alter table public."products" add constraint "products_category_id_fkey" foreign key ("category_id") references public."categories" ("id") on update cascade on delete restrict;

alter table public."product_images" drop constraint if exists "product_images_product_id_fkey";
alter table public."product_images" add constraint "product_images_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."product_images" drop constraint if exists "product_images_variant_id_fkey";
alter table public."product_images" add constraint "product_images_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete set null;

alter table public."product_variants" drop constraint if exists "product_variants_product_id_fkey";
alter table public."product_variants" add constraint "product_variants_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."product_variants" drop constraint if exists "product_variants_shop_id_fkey";
alter table public."product_variants" add constraint "product_variants_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."variant_barcodes" drop constraint if exists "variant_barcodes_variant_id_fkey";
alter table public."variant_barcodes" add constraint "variant_barcodes_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete restrict;

alter table public."variant_barcodes" drop constraint if exists "variant_barcodes_created_by_fkey";
alter table public."variant_barcodes" add constraint "variant_barcodes_created_by_fkey" foreign key ("created_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."inventory_balances" drop constraint if exists "inventory_balances_shop_id_fkey";
alter table public."inventory_balances" add constraint "inventory_balances_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."inventory_balances" drop constraint if exists "inventory_balances_variant_id_fkey";
alter table public."inventory_balances" add constraint "inventory_balances_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete
restrict;

alter table public."inventory_movements" drop constraint if exists "inventory_movements_shop_id_fkey";
alter table public."inventory_movements" add constraint "inventory_movements_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."inventory_movements" drop constraint if exists "inventory_movements_variant_id_fkey";
alter table public."inventory_movements" add constraint "inventory_movements_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete
restrict;

alter table public."inventory_movements" drop constraint if exists "inventory_movements_performed_by_fkey";
alter table public."inventory_movements" add constraint "inventory_movements_performed_by_fkey" foreign key ("performed_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."inventory_reservations" drop constraint if exists "inventory_reservations_shop_id_fkey";
alter table public."inventory_reservations" add constraint "inventory_reservations_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."inventory_reservations" drop constraint if exists "inventory_reservations_variant_id_fkey";
alter table public."inventory_reservations" add constraint "inventory_reservations_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete
restrict;

alter table public."inventory_reservations" drop constraint if exists "inventory_reservations_cart_id_fkey";
alter table public."inventory_reservations" add constraint "inventory_reservations_cart_id_fkey" foreign key ("cart_id") references public."carts" ("id") on update cascade on delete set null;

alter table public."inventory_reservations" drop constraint if exists "inventory_reservations_order_id_fkey";
alter table public."inventory_reservations" add constraint "inventory_reservations_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete set null;

alter table public."product_photo_matches" drop constraint if exists "product_photo_matches_shop_id_fkey";
alter table public."product_photo_matches" add constraint "product_photo_matches_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."product_photo_matches" drop constraint if exists "product_photo_matches_uploaded_by_fkey";
alter table public."product_photo_matches" add constraint "product_photo_matches_uploaded_by_fkey" foreign key ("uploaded_by") references public."profiles" ("id") on update cascade on delete
restrict;

alter table public."product_photo_matches" drop constraint if exists "product_photo_matches_matched_variant_id_fkey";
alter table public."product_photo_matches" add constraint "product_photo_matches_matched_variant_id_fkey" foreign key ("matched_variant_id") references public."product_variants" ("id") on update
cascade on delete set null;

alter table public."shop_favourites" drop constraint if exists "shop_favourites_customer_id_fkey";
alter table public."shop_favourites" add constraint "shop_favourites_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete
cascade;

alter table public."shop_favourites" drop constraint if exists "shop_favourites_shop_id_fkey";
alter table public."shop_favourites" add constraint "shop_favourites_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."wishlist_items" drop constraint if exists "wishlist_items_customer_id_fkey";
alter table public."wishlist_items" add constraint "wishlist_items_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete cascade;
```

```
alter table public."wishlist_items" drop constraint if exists "wishlist_items_product_id_fkey";
alter table public."wishlist_items" add constraint "wishlist_items_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."wishlist_items" drop constraint if exists "wishlist_items_variant_id_fkey";
alter table public."wishlist_items" add constraint "wishlist_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete set null;

alter table public."shop_collections" drop constraint if exists "shop_collections_shop_id_fkey";
alter table public."shop_collections" add constraint "shop_collections_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."shop_collection_items" drop constraint if exists "shop_collection_items_collection_id_fkey";
alter table public."shop_collection_items" add constraint "shop_collection_items_collection_id_fkey" foreign key ("collection_id") references public."shop_collections" ("id") on update cascade on
delete restrict;

alter table public."shop_collection_items" drop constraint if exists "shop_collection_items_product_id_fkey";
alter table public."shop_collection_items" add constraint "shop_collection_items_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."shop_collection_items" drop constraint if exists "shop_collection_items_variant_id_fkey";
alter table public."shop_collection_items" add constraint "shop_collection_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete
set null;

alter table public."offers" drop constraint if exists "offers_shop_id_fkey";
alter table public."offers" add constraint "offers_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."offer_products" drop constraint if exists "offer_products_offer_id_fkey";
alter table public."offer_products" add constraint "offer_products_offer_id_fkey" foreign key ("offer_id") references public."offers" ("id") on update cascade on delete restrict;

alter table public."offer_products" drop constraint if exists "offer_products_product_id_fkey";
alter table public."offer_products" add constraint "offer_products_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."offer_products" drop constraint if exists "offer_products_variant_id_fkey";
alter table public."offer_products" add constraint "offer_products_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete set null;

alter table public."offer_redemptions" drop constraint if exists "offer_redemptions_offer_id_fkey";
alter table public."offer_redemptions" add constraint "offer_redemptions_offer_id_fkey" foreign key ("offer_id") references public."offers" ("id") on update cascade on delete restrict;

alter table public."offer_redemptions" drop constraint if exists "offer_redemptions_customer_id_fkey";
alter table public."offer_redemptions" add constraint "offer_redemptions_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete
cascade;

alter table public."offer_redemptions" drop constraint if exists "offer_redemptions_order_id_fkey";
alter table public."offer_redemptions" add constraint "offer_redemptions_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."offline_sales" drop constraint if exists "offline_sales_shop_id_fkey";
alter table public."offline_sales" add constraint "offline_sales_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."offline_sales" drop constraint if exists "offline_sales_merchant_id_fkey";
alter table public."offline_sales" add constraint "offline_sales_merchant_id_fkey" foreign key ("merchant_id") references public."merchant_profiles" ("user_id") on update cascade on delete restrict;

alter table public."offline_sales" drop constraint if exists "offline_sales_recorded_by_fkey";
alter table public."offline_sales" add constraint "offline_sales_recorded_by_fkey" foreign key ("recorded_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."offline_sale_items" drop constraint if exists "offline_sale_items_offline_sale_id_fkey";
alter table public."offline_sale_items" add constraint "offline_sale_items_offline_sale_id_fkey" foreign key ("offline_sale_id") references public."offline_sales" ("id") on update cascade on delete
cascade;

alter table public."offline_sale_items" drop constraint if exists "offline_sale_items_variant_id_fkey";
alter table public."offline_sale_items" add constraint "offline_sale_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete cascade;

alter table public."carts" drop constraint if exists "carts_customer_id_fkey";
alter table public."carts" add constraint "carts_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete cascade;

alter table public."carts" drop constraint if exists "carts_shop_id_fkey";
alter table public."carts" add constraint "carts_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."cart_items" drop constraint if exists "cart_items_cart_id_fkey";
alter table public."cart_items" add constraint "cart_items_cart_id_fkey" foreign key ("cart_id") references public."carts" ("id") on update cascade on delete cascade;

alter table public."cart_items" drop constraint if exists "cart_items_variant_id_fkey";
alter table public."cart_items" add constraint "cart_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete cascade;

alter table public."orders" drop constraint if exists "orders_customer_id_fkey";
alter table public."orders" add constraint "orders_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete cascade;

alter table public."orders" drop constraint if exists "orders_shop_id_fkey";
alter table public."orders" add constraint "orders_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."orders" drop constraint if exists "orders_cart_id_fkey";
```

```
alter table public."orders" add constraint "orders_cart_id_fkey" foreign key ("cart_id") references public."carts" ("id") on update cascade on delete set null;

alter table public."orders" drop constraint if exists "orders_style_group_id_fkey";
alter table public."orders" add constraint "orders_style_group_id_fkey" foreign key ("style_group_id") references public."style_groups" ("id") on update cascade on delete set null;

alter table public."orders" drop constraint if exists "orders_delivery_address_id_fkey";
alter table public."orders" add constraint "orders_delivery_address_id_fkey" foreign key ("delivery_address_id") references public."addresses" ("id") on update cascade on delete restrict;

alter table public."order_items" drop constraint if exists "order_items_order_id_fkey";
alter table public."order_items" add constraint "order_items_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete cascade;

alter table public."order_items" drop constraint if exists "order_items_product_id_fkey";
alter table public."order_items" add constraint "order_items_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete cascade;

alter table public."order_items" drop constraint if exists "order_items_variant_id_fkey";
alter table public."order_items" add constraint "order_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete cascade;

alter table public."order_status_history" drop constraint if exists "order_status_history_order_id_fkey";
alter table public."order_status_history" add constraint "order_status_history_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."order_status_history" drop constraint if exists "order_status_history_changed_by_user_id_fkey";
alter table public."order_status_history" add constraint "order_status_history_changed_by_user_id_fkey" foreign key ("changed_by_user_id") references public."profiles" ("id") on update cascade on
delete set null;

alter table public."merchant_order_alerts" drop constraint if exists "merchant_order_alerts_order_id_fkey";
alter table public."merchant_order_alerts" add constraint "merchant_order_alerts_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."merchant_order_alerts" drop constraint if exists "merchant_order_alerts_shop_id_fkey";
alter table public."merchant_order_alerts" add constraint "merchant_order_alerts_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."merchant_order_alerts" drop constraint if exists "merchant_order_alerts_device_id_fkey";
alter table public."merchant_order_alerts" add constraint "merchant_order_alerts_device_id_fkey" foreign key ("device_id") references public."user_devices" ("id") on update cascade on delete set
null;

alter table public."merchant_order_alerts" drop constraint if exists "merchant_order_alerts_acknowledged_by_fkey";
alter table public."merchant_order_alerts" add constraint "merchant_order_alerts_acknowledged_by_fkey" foreign key ("acknowledged_by") references public."profiles" ("id") on update cascade on delete
set null;

alter table public."merchant_order_issues" drop constraint if exists "merchant_order_issues_order_id_fkey";
alter table public."merchant_order_issues" add constraint "merchant_order_issues_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."merchant_order_issues" drop constraint if exists "merchant_order_issues_order_item_id_fkey";
alter table public."merchant_order_issues" add constraint "merchant_order_issues_order_item_id_fkey" foreign key ("order_item_id") references public."order_items" ("id") on update cascade on delete
set null;

alter table public."merchant_order_issues" drop constraint if exists "merchant_order_issues_reported_by_fkey";
alter table public."merchant_order_issues" add constraint "merchant_order_issues_reported_by_fkey" foreign key ("reported_by") references public."profiles" ("id") on update cascade on delete
restrict;

alter table public."merchant_order_issues" drop constraint if exists "merchant_order_issues_resolved_by_fkey";
alter table public."merchant_order_issues" add constraint "merchant_order_issues_resolved_by_fkey" foreign key ("resolved_by") references public."profiles" ("id") on update cascade on delete set
null;

alter table public."order_item_verifications" drop constraint if exists "order_item_verifications_order_item_id_fkey";
alter table public."order_item_verifications" add constraint "order_item_verifications_order_item_id_fkey" foreign key ("order_item_id") references public."order_items" ("id") on update cascade on
delete restrict;

alter table public."order_item_verifications" drop constraint if exists "order_item_verifications_verified_variant_id_fkey";
alter table public."order_item_verifications" add constraint "order_item_verifications_verified_variant_id_fkey" foreign key ("verified_variant_id") references public."product_variants" ("id") on
update cascade on delete set null;

alter table public."order_item_verifications" drop constraint if exists "order_item_verifications_verified_by_fkey";
alter table public."order_item_verifications" add constraint "order_item_verifications_verified_by_fkey" foreign key ("verified_by") references public."profiles" ("id") on update cascade on delete
restrict;

alter table public."payments" drop constraint if exists "payments_order_id_fkey";
alter table public."payments" add constraint "payments_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."payments" drop constraint if exists "payments_customer_id_fkey";
alter table public."payments" add constraint "payments_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete cascade;

alter table public."payment_events" drop constraint if exists "payment_events_payment_id_fkey";
alter table public."payment_events" add constraint "payment_events_payment_id_fkey" foreign key ("payment_id") references public."payments" ("id") on update cascade on delete set null;

alter table public."return_requests" drop constraint if exists "return_requests_order_id_fkey";
alter table public."return_requests" add constraint "return_requests_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."return_requests" drop constraint if exists "return_requests_customer_id_fkey";
```

```
alter table public."return_requests" add constraint "return_requests_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete
cascade;

alter table public."return_requests" drop constraint if exists "return_requests_shop_id_fkey";
alter table public."return_requests" add constraint "return_requests_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."return_items" drop constraint if exists "return_items_return_request_id_fkey";
alter table public."return_items" add constraint "return_items_return_request_id_fkey" foreign key ("return_request_id") references public."return_requests" ("id") on update cascade on delete
restrict;

alter table public."return_items" drop constraint if exists "return_items_order_item_id_fkey";
alter table public."return_items" add constraint "return_items_order_item_id_fkey" foreign key ("order_item_id") references public."order_items" ("id") on update cascade on delete restrict;

alter table public."return_items" drop constraint if exists "return_items_inspected_by_fkey";
alter table public."return_items" add constraint "return_items_inspected_by_fkey" foreign key ("inspected_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."return_evidence" drop constraint if exists "return_evidence_return_request_id_fkey";
alter table public."return_evidence" add constraint "return_evidence_return_request_id_fkey" foreign key ("return_request_id") references public."return_requests" ("id") on update cascade on delete
restrict;

alter table public."return_evidence" drop constraint if exists "return_evidence_uploaded_by_fkey";
alter table public."return_evidence" add constraint "return_evidence_uploaded_by_fkey" foreign key ("uploaded_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."return_status_history" drop constraint if exists "return_status_history_return_request_id_fkey";
alter table public."return_status_history" add constraint "return_status_history_return_request_id_fkey" foreign key ("return_request_id") references public."return_requests" ("id") on update cascade
on delete restrict;

alter table public."return_status_history" drop constraint if exists "return_status_history_changed_by_fkey";
alter table public."return_status_history" add constraint "return_status_history_changed_by_fkey" foreign key ("changed_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."refunds" drop constraint if exists "refunds_order_id_fkey";
alter table public."refunds" add constraint "refunds_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."refunds" drop constraint if exists "refunds_payment_id_fkey";
alter table public."refunds" add constraint "refunds_payment_id_fkey" foreign key ("payment_id") references public."payments" ("id") on update cascade on delete set null;

alter table public."refunds" drop constraint if exists "refunds_return_request_id_fkey";
alter table public."refunds" add constraint "refunds_return_request_id_fkey" foreign key ("return_request_id") references public."return_requests" ("id") on update cascade on delete set null;

alter table public."refunds" drop constraint if exists "refunds_initiated_by_fkey";
alter table public."refunds" add constraint "refunds_initiated_by_fkey" foreign key ("initiated_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."refunds" drop constraint if exists "refunds_approved_by_fkey";
alter table public."refunds" add constraint "refunds_approved_by_fkey" foreign key ("approved_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."merchant_settlements" drop constraint if exists "merchant_settlements_shop_id_fkey";
alter table public."merchant_settlements" add constraint "merchant_settlements_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete restrict;

alter table public."merchant_settlements" drop constraint if exists "merchant_settlements_bank_account_id_fkey";
alter table public."merchant_settlements" add constraint "merchant_settlements_bank_account_id_fkey" foreign key ("bank_account_id") references public."shop_bank_accounts" ("id") on update cascade on
delete restrict;

alter table public."merchant_settlement_items" drop constraint if exists "merchant_settlement_items_settlement_id_fkey";
alter table public."merchant_settlement_items" add constraint "merchant_settlement_items_settlement_id_fkey" foreign key ("settlement_id") references public."merchant_settlements" ("id") on update
cascade on delete restrict;

alter table public."merchant_settlement_items" drop constraint if exists "merchant_settlement_items_order_id_fkey";
alter table public."merchant_settlement_items" add constraint "merchant_settlement_items_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete set
null;

alter table public."merchant_settlement_items" drop constraint if exists "merchant_settlement_items_refund_id_fkey";
alter table public."merchant_settlement_items" add constraint "merchant_settlement_items_refund_id_fkey" foreign key ("refund_id") references public."refunds" ("id") on update cascade on delete set
null;

alter table public."captain_documents" drop constraint if exists "captain_documents_captain_id_fkey";
alter table public."captain_documents" add constraint "captain_documents_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade on delete
cascade;

alter table public."captain_documents" drop constraint if exists "captain_documents_verified_by_fkey";
alter table public."captain_documents" add constraint "captain_documents_verified_by_fkey" foreign key ("verified_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."delivery_tasks" drop constraint if exists "delivery_tasks_order_id_fkey";
alter table public."delivery_tasks" add constraint "delivery_tasks_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete set null;

alter table public."delivery_tasks" drop constraint if exists "delivery_tasks_return_request_id_fkey";
alter table public."delivery_tasks" add constraint "delivery_tasks_return_request_id_fkey" foreign key ("return_request_id") references public."return_requests" ("id") on update cascade on delete set
null;
```

```
alter table public."delivery_tasks" drop constraint if exists "delivery_tasks_pickup_shop_id_fkey";
alter table public."delivery_tasks" add constraint "delivery_tasks_pickup_shop_id_fkey" foreign key ("pickup_shop_id") references public."shops" ("id") on update cascade on delete set null;

alter table public."delivery_tasks" drop constraint if exists "delivery_tasks_assigned_captain_id_fkey";
alter table public."delivery_tasks" add constraint "delivery_tasks_assigned_captain_id_fkey" foreign key ("assigned_captain_id") references public."captain_profiles" ("user_id") on update cascade on
delete set null;

alter table public."delivery_assignments" drop constraint if exists "delivery_assignments_delivery_task_id_fkey";
alter table public."delivery_assignments" add constraint "delivery_assignments_delivery_task_id_fkey" foreign key ("delivery_task_id") references public."delivery_tasks" ("id") on update cascade on
delete restrict;

alter table public."delivery_assignments" drop constraint if exists "delivery_assignments_captain_id_fkey";
alter table public."delivery_assignments" add constraint "delivery_assignments_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade on delete
cascade;

alter table public."delivery_assignments" drop constraint if exists "delivery_assignments_assigned_by_user_id_fkey";
alter table public."delivery_assignments" add constraint "delivery_assignments_assigned_by_user_id_fkey" foreign key ("assigned_by_user_id") references public."profiles" ("id") on update cascade on
delete set null;

alter table public."captain_current_locations" drop constraint if exists "captain_current_locations_captain_id_fkey";
alter table public."captain_current_locations" add constraint "captain_current_locations_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade
on delete cascade;

alter table public."captain_current_locations" drop constraint if exists "captain_current_locations_active_delivery_task_id_fkey";
alter table public."captain_current_locations" add constraint "captain_current_locations_active_delivery_task_id_fkey" foreign key ("active_delivery_task_id") references public."delivery_tasks"
("id") on update cascade on delete set null;

alter table public."captain_location_history" drop constraint if exists "captain_location_history_captain_id_fkey";
alter table public."captain_location_history" add constraint "captain_location_history_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade
on delete cascade;

alter table public."captain_location_history" drop constraint if exists "captain_location_history_delivery_task_id_fkey";
alter table public."captain_location_history" add constraint "captain_location_history_delivery_task_id_fkey" foreign key ("delivery_task_id") references public."delivery_tasks" ("id") on update
cascade on delete set null;

alter table public."delivery_events" drop constraint if exists "delivery_events_delivery_task_id_fkey";
alter table public."delivery_events" add constraint "delivery_events_delivery_task_id_fkey" foreign key ("delivery_task_id") references public."delivery_tasks" ("id") on update cascade on delete
restrict;

alter table public."delivery_events" drop constraint if exists "delivery_events_actor_user_id_fkey";
alter table public."delivery_events" add constraint "delivery_events_actor_user_id_fkey" foreign key ("actor_user_id") references public."profiles" ("id") on update cascade on delete set null;

alter table public."cod_collections" drop constraint if exists "cod_collections_order_id_fkey";
alter table public."cod_collections" add constraint "cod_collections_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."cod_collections" drop constraint if exists "cod_collections_delivery_task_id_fkey";
alter table public."cod_collections" add constraint "cod_collections_delivery_task_id_fkey" foreign key ("delivery_task_id") references public."delivery_tasks" ("id") on update cascade on delete
restrict;

alter table public."cod_collections" drop constraint if exists "cod_collections_captain_id_fkey";
alter table public."cod_collections" add constraint "cod_collections_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade on delete cascade;

alter table public."cod_collections" drop constraint if exists "cod_collections_reconciled_by_fkey";
alter table public."cod_collections" add constraint "cod_collections_reconciled_by_fkey" foreign key ("reconciled_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."captain_earnings" drop constraint if exists "captain_earnings_captain_id_fkey";
alter table public."captain_earnings" add constraint "captain_earnings_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade on delete
cascade;

alter table public."captain_earnings" drop constraint if exists "captain_earnings_delivery_task_id_fkey";
alter table public."captain_earnings" add constraint "captain_earnings_delivery_task_id_fkey" foreign key ("delivery_task_id") references public."delivery_tasks" ("id") on update cascade on delete
restrict;

alter table public."captain_payouts" drop constraint if exists "captain_payouts_captain_id_fkey";
alter table public."captain_payouts" add constraint "captain_payouts_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade on delete cascade;

alter table public."captain_payout_items" drop constraint if exists "captain_payout_items_payout_id_fkey";
alter table public."captain_payout_items" add constraint "captain_payout_items_payout_id_fkey" foreign key ("payout_id") references public."captain_payouts" ("id") on update cascade on delete
restrict;

alter table public."captain_payout_items" drop constraint if exists "captain_payout_items_captain_earning_id_fkey";
alter table public."captain_payout_items" add constraint "captain_payout_items_captain_earning_id_fkey" foreign key ("captain_earning_id") references public."captain_earnings" ("id") on update
cascade on delete set null;

alter table public."style_groups" drop constraint if exists "style_groups_creator_customer_id_fkey";
alter table public."style_groups" add constraint "style_groups_creator_customer_id_fkey" foreign key ("creator_customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete restrict;
```

```
alter table public."style_group_members" drop constraint if exists "style_group_members_group_id_fkey";
alter table public."style_group_members" add constraint "style_group_members_group_id_fkey" foreign key ("group_id") references public."style_groups" ("id") on update cascade on delete restrict;

alter table public."style_group_members" drop constraint if exists "style_group_members_customer_id_fkey";
alter table public."style_group_members" add constraint "style_group_members_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete set null;

alter table public."style_group_member_preferences" drop constraint if exists "style_group_member_preferences_group_member_id_fkey";
alter table public."style_group_member_preferences" add constraint "style_group_member_preferences_group_member_id_fkey" foreign key ("group_member_id") references public."style_group_members" ("id")
on update cascade on delete restrict;

alter table public."style_group_member_preferences" drop constraint if exists "style_group_member_preferences_private_body_profile_id_fkey";
alter table public."style_group_member_preferences" add constraint "style_group_member_preferences_private_body_profile_id_fkey" foreign key ("private_body_profile_id") references
public."customer_body_profiles" ("id") on update cascade on delete set null;

alter table public."style_themes" drop constraint if exists "style_themes_group_id_fkey";
alter table public."style_themes" add constraint "style_themes_group_id_fkey" foreign key ("group_id") references public."style_groups" ("id") on update cascade on delete restrict;

alter table public."style_themes" drop constraint if exists "style_themes_created_by_fkey";
alter table public."style_themes" add constraint "style_themes_created_by_fkey" foreign key ("created_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."style_polls" drop constraint if exists "style_polls_group_id_fkey";
alter table public."style_polls" add constraint "style_polls_group_id_fkey" foreign key ("group_id") references public."style_groups" ("id") on update cascade on delete restrict;

alter table public."style_polls" drop constraint if exists "style_polls_created_by_fkey";
alter table public."style_polls" add constraint "style_polls_created_by_fkey" foreign key ("created_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."style_poll_options" drop constraint if exists "style_poll_options_poll_id_fkey";
alter table public."style_poll_options" add constraint "style_poll_options_poll_id_fkey" foreign key ("poll_id") references public."style_polls" ("id") on update cascade on delete cascade;

alter table public."style_poll_options" drop constraint if exists "style_poll_options_product_id_fkey";
alter table public."style_poll_options" add constraint "style_poll_options_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete set null;

alter table public."style_poll_options" drop constraint if exists "style_poll_options_theme_id_fkey";
alter table public."style_poll_options" add constraint "style_poll_options_theme_id_fkey" foreign key ("theme_id") references public."style_themes" ("id") on update cascade on delete set null;

alter table public."style_poll_votes" drop constraint if exists "style_poll_votes_poll_id_fkey";
alter table public."style_poll_votes" add constraint "style_poll_votes_poll_id_fkey" foreign key ("poll_id") references public."style_polls" ("id") on update cascade on delete cascade;

alter table public."style_poll_votes" drop constraint if exists "style_poll_votes_option_id_fkey";
alter table public."style_poll_votes" add constraint "style_poll_votes_option_id_fkey" foreign key ("option_id") references public."style_poll_options" ("id") on update cascade on delete cascade;

alter table public."style_poll_votes" drop constraint if exists "style_poll_votes_group_member_id_fkey";
alter table public."style_poll_votes" add constraint "style_poll_votes_group_member_id_fkey" foreign key ("group_member_id") references public."style_group_members" ("id") on update cascade on delete
cascade;

alter table public."style_moodboards" drop constraint if exists "style_moodboards_group_id_fkey";
alter table public."style_moodboards" add constraint "style_moodboards_group_id_fkey" foreign key ("group_id") references public."style_groups" ("id") on update cascade on delete restrict;

alter table public."style_moodboards" drop constraint if exists "style_moodboards_theme_id_fkey";
alter table public."style_moodboards" add constraint "style_moodboards_theme_id_fkey" foreign key ("theme_id") references public."style_themes" ("id") on update cascade on delete set null;

alter table public."style_moodboards" drop constraint if exists "style_moodboards_created_by_fkey";
alter table public."style_moodboards" add constraint "style_moodboards_created_by_fkey" foreign key ("created_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."style_moodboard_items" drop constraint if exists "style_moodboard_items_moodboard_id_fkey";
alter table public."style_moodboard_items" add constraint "style_moodboard_items_moodboard_id_fkey" foreign key ("moodboard_id") references public."style_moodboards" ("id") on update cascade on
delete restrict;

alter table public."style_moodboard_items" drop constraint if exists "style_moodboard_items_group_member_id_fkey";
alter table public."style_moodboard_items" add constraint "style_moodboard_items_group_member_id_fkey" foreign key ("group_member_id") references public."style_group_members" ("id") on update cascade
on delete set null;

alter table public."style_moodboard_items" drop constraint if exists "style_moodboard_items_product_id_fkey";
alter table public."style_moodboard_items" add constraint "style_moodboard_items_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete set null;

alter table public."style_moodboard_items" drop constraint if exists "style_moodboard_items_variant_id_fkey";
alter table public."style_moodboard_items" add constraint "style_moodboard_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete
set null;

alter table public."notifications" drop constraint if exists "notifications_user_id_fkey";
alter table public."notifications" add constraint "notifications_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."notification_deliveries" drop constraint if exists "notification_deliveries_notification_id_fkey";
alter table public."notification_deliveries" add constraint "notification_deliveries_notification_id_fkey" foreign key ("notification_id") references public."notifications" ("id") on update cascade
on delete restrict;

alter table public."notification_deliveries" drop constraint if exists "notification_deliveries_device_id_fkey";
```

```
alter table public."notification_deliveries" add constraint "notification_deliveries_device_id_fkey" foreign key ("device_id") references public."user_devices" ("id") on update cascade on delete set
null;

alter table public."notification_preferences" drop constraint if exists "notification_preferences_user_id_fkey";
alter table public."notification_preferences" add constraint "notification_preferences_user_id_fkey" foreign key ("user_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."reviews" drop constraint if exists "reviews_order_id_fkey";
alter table public."reviews" add constraint "reviews_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete restrict;

alter table public."reviews" drop constraint if exists "reviews_customer_id_fkey";
alter table public."reviews" add constraint "reviews_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete cascade;

alter table public."reviews" drop constraint if exists "reviews_shop_id_fkey";
alter table public."reviews" add constraint "reviews_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete set null;

alter table public."reviews" drop constraint if exists "reviews_product_id_fkey";
alter table public."reviews" add constraint "reviews_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete set null;

alter table public."reviews" drop constraint if exists "reviews_captain_id_fkey";
alter table public."reviews" add constraint "reviews_captain_id_fkey" foreign key ("captain_id") references public."captain_profiles" ("user_id") on update cascade on delete set null;

alter table public."customer_events" drop constraint if exists "customer_events_customer_id_fkey";
alter table public."customer_events" add constraint "customer_events_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete set
null;

alter table public."customer_events" drop constraint if exists "customer_events_shop_id_fkey";
alter table public."customer_events" add constraint "customer_events_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete set null;

alter table public."recommendation_sets" drop constraint if exists "recommendation_sets_customer_id_fkey";
alter table public."recommendation_sets" add constraint "recommendation_sets_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete set null;

alter table public."recommendation_items" drop constraint if exists "recommendation_items_recommendation_set_id_fkey";
alter table public."recommendation_items" add constraint "recommendation_items_recommendation_set_id_fkey" foreign key ("recommendation_set_id") references public."recommendation_sets" ("id") on
update cascade on delete restrict;

alter table public."recommendation_items" drop constraint if exists "recommendation_items_product_id_fkey";
alter table public."recommendation_items" add constraint "recommendation_items_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."recommendation_items" drop constraint if exists "recommendation_items_variant_id_fkey";
alter table public."recommendation_items" add constraint "recommendation_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete set
null;

alter table public."product_embeddings" drop constraint if exists "product_embeddings_product_id_fkey";
alter table public."product_embeddings" add constraint "product_embeddings_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."customer_embeddings" drop constraint if exists "customer_embeddings_customer_id_fkey";
alter table public."customer_embeddings" add constraint "customer_embeddings_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete cascade;

alter table public."size_charts" drop constraint if exists "size_charts_shop_id_fkey";
alter table public."size_charts" add constraint "size_charts_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete set null;

alter table public."size_charts" drop constraint if exists "size_charts_category_id_fkey";
alter table public."size_charts" add constraint "size_charts_category_id_fkey" foreign key ("category_id") references public."categories" ("id") on update cascade on delete restrict;

alter table public."size_chart_measurements" drop constraint if exists "size_chart_measurements_size_chart_id_fkey";
alter table public."size_chart_measurements" add constraint "size_chart_measurements_size_chart_id_fkey" foreign key ("size_chart_id") references public."size_charts" ("id") on update cascade on
delete restrict;

alter table public."customer_body_profiles" drop constraint if exists "customer_body_profiles_customer_id_fkey";
alter table public."customer_body_profiles" add constraint "customer_body_profiles_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete cascade;

alter table public."body_measurements" drop constraint if exists "body_measurements_body_profile_id_fkey";
alter table public."body_measurements" add constraint "body_measurements_body_profile_id_fkey" foreign key ("body_profile_id") references public."customer_body_profiles" ("id") on update cascade on
delete restrict;

alter table public."body_scan_sessions" drop constraint if exists "body_scan_sessions_customer_id_fkey";
alter table public."body_scan_sessions" add constraint "body_scan_sessions_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete
cascade;

alter table public."body_scan_sessions" drop constraint if exists "body_scan_sessions_body_profile_id_fkey";
alter table public."body_scan_sessions" add constraint "body_scan_sessions_body_profile_id_fkey" foreign key ("body_profile_id") references public."customer_body_profiles" ("id") on update cascade on
delete set null;

alter table public."fit_recommendations" drop constraint if exists "fit_recommendations_customer_id_fkey";
```

```
alter table public."fit_recommendations" add constraint "fit_recommendations_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete cascade;

alter table public."fit_recommendations" drop constraint if exists "fit_recommendations_body_profile_id_fkey";
alter table public."fit_recommendations" add constraint "fit_recommendations_body_profile_id_fkey" foreign key ("body_profile_id") references public."customer_body_profiles" ("id") on update cascade
on delete set null;

alter table public."fit_recommendations" drop constraint if exists "fit_recommendations_product_id_fkey";
alter table public."fit_recommendations" add constraint "fit_recommendations_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."fit_recommendations" drop constraint if exists "fit_recommendations_variant_id_fkey";
alter table public."fit_recommendations" add constraint "fit_recommendations_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete set
null;

alter table public."fit_feedback" drop constraint if exists "fit_feedback_customer_id_fkey";
alter table public."fit_feedback" add constraint "fit_feedback_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on delete cascade;

alter table public."fit_feedback" drop constraint if exists "fit_feedback_order_item_id_fkey";
alter table public."fit_feedback" add constraint "fit_feedback_order_item_id_fkey" foreign key ("order_item_id") references public."order_items" ("id") on update cascade on delete restrict;

alter table public."fit_feedback" drop constraint if exists "fit_feedback_fit_recommendation_id_fkey";
alter table public."fit_feedback" add constraint "fit_feedback_fit_recommendation_id_fkey" foreign key ("fit_recommendation_id") references public."fit_recommendations" ("id") on update cascade on
delete set null;

alter table public."virtual_tryon_sessions" drop constraint if exists "virtual_tryon_sessions_customer_id_fkey";
alter table public."virtual_tryon_sessions" add constraint "virtual_tryon_sessions_customer_id_fkey" foreign key ("customer_id") references public."customer_profiles" ("user_id") on update cascade on
delete cascade;

alter table public."virtual_tryon_sessions" drop constraint if exists "virtual_tryon_sessions_product_id_fkey";
alter table public."virtual_tryon_sessions" add constraint "virtual_tryon_sessions_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete
restrict;

alter table public."virtual_tryon_sessions" drop constraint if exists "virtual_tryon_sessions_variant_id_fkey";
alter table public."virtual_tryon_sessions" add constraint "virtual_tryon_sessions_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on delete
set null;

alter table public."virtual_tryon_sessions" drop constraint if exists "virtual_tryon_sessions_body_profile_id_fkey";
alter table public."virtual_tryon_sessions" add constraint "virtual_tryon_sessions_body_profile_id_fkey" foreign key ("body_profile_id") references public."customer_body_profiles" ("id") on update
cascade on delete set null;

alter table public."virtual_tryon_outputs" drop constraint if exists "virtual_tryon_outputs_tryon_session_id_fkey";
alter table public."virtual_tryon_outputs" add constraint "virtual_tryon_outputs_tryon_session_id_fkey" foreign key ("tryon_session_id") references public."virtual_tryon_sessions" ("id") on update
cascade on delete restrict;

alter table public."style_recommendation_sets" drop constraint if exists "style_recommendation_sets_group_id_fkey";
alter table public."style_recommendation_sets" add constraint "style_recommendation_sets_group_id_fkey" foreign key ("group_id") references public."style_groups" ("id") on update cascade on delete
restrict;

alter table public."style_recommendation_sets" drop constraint if exists "style_recommendation_sets_theme_id_fkey";
alter table public."style_recommendation_sets" add constraint "style_recommendation_sets_theme_id_fkey" foreign key ("theme_id") references public."style_themes" ("id") on update cascade on delete
set null;

alter table public."style_recommendation_items" drop constraint if exists "style_recommendation_items_recommendation_set_id_fkey";
alter table public."style_recommendation_items" add constraint "style_recommendation_items_recommendation_set_id_fkey" foreign key ("recommendation_set_id") references
public."style_recommendation_sets" ("id") on update cascade on delete restrict;

alter table public."style_recommendation_items" drop constraint if exists "style_recommendation_items_group_member_id_fkey";
alter table public."style_recommendation_items" add constraint "style_recommendation_items_group_member_id_fkey" foreign key ("group_member_id") references public."style_group_members" ("id") on
update cascade on delete restrict;

alter table public."style_recommendation_items" drop constraint if exists "style_recommendation_items_product_id_fkey";
alter table public."style_recommendation_items" add constraint "style_recommendation_items_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete
restrict;

alter table public."style_recommendation_items" drop constraint if exists "style_recommendation_items_variant_id_fkey";
alter table public."style_recommendation_items" add constraint "style_recommendation_items_variant_id_fkey" foreign key ("variant_id") references public."product_variants" ("id") on update cascade on
delete set null;

alter table public."support_tickets" drop constraint if exists "support_tickets_raised_by_user_id_fkey";
alter table public."support_tickets" add constraint "support_tickets_raised_by_user_id_fkey" foreign key ("raised_by_user_id") references public."profiles" ("id") on update cascade on delete
restrict;

alter table public."support_tickets" drop constraint if exists "support_tickets_order_id_fkey";
alter table public."support_tickets" add constraint "support_tickets_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete set null;

alter table public."support_tickets" drop constraint if exists "support_tickets_shop_id_fkey";
alter table public."support_tickets" add constraint "support_tickets_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete set null;
```

```
alter table public."support_tickets" drop constraint if exists "support_tickets_delivery_task_id_fkey";
alter table public."support_tickets" add constraint "support_tickets_delivery_task_id_fkey" foreign key ("delivery_task_id") references public."delivery_tasks" ("id") on update cascade on delete set null;

alter table public."support_tickets" drop constraint if exists "support_tickets_return_request_id_fkey";
alter table public."support_tickets" add constraint "support_tickets_return_request_id_fkey" foreign key ("return_request_id") references public."return_requests" ("id") on update cascade on delete set null;

alter table public."support_tickets" drop constraint if exists "support_tickets_assigned_to_fkey";
alter table public."support_tickets" add constraint "support_tickets_assigned_to_fkey" foreign key ("assigned_to") references public."profiles" ("id") on update cascade on delete set null;

alter table public."support_messages" drop constraint if exists "support_messages_ticket_id_fkey";
alter table public."support_messages" add constraint "support_messages_ticket_id_fkey" foreign key ("ticket_id") references public."support_tickets" ("id") on update cascade on delete cascade;

alter table public."support_messages" drop constraint if exists "support_messages_sender_id_fkey";
alter table public."support_messages" add constraint "support_messages_sender_id_fkey" foreign key ("sender_id") references public."profiles" ("id") on update cascade on delete cascade;

alter table public."campaigns" drop constraint if exists "campaigns_created_by_fkey";
alter table public."campaigns" add constraint "campaigns_created_by_fkey" foreign key ("created_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."campaigns" drop constraint if exists "campaigns_approved_by_fkey";
alter table public."campaigns" add constraint "campaigns_approved_by_fkey" foreign key ("approved_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."banners" drop constraint if exists "banners_campaign_id_fkey";
alter table public."banners" add constraint "banners_campaign_id_fkey" foreign key ("campaign_id") references public."campaigns" ("id") on update cascade on delete restrict;

alter table public."merchant_leads" drop constraint if exists "merchant_leads_assigned_executive_id_fkey";
alter table public."merchant_leads" add constraint "merchant_leads_assigned_executive_id_fkey" foreign key ("assigned_executive_id") references public."profiles" ("id") on update cascade on delete set null;

alter table public."field_visits" drop constraint if exists "field_visits_merchant_lead_id_fkey";
alter table public."field_visits" add constraint "field_visits_merchant_lead_id_fkey" foreign key ("merchant_lead_id") references public."merchant_leads" ("id") on update cascade on delete set null;

alter table public."field_visits" drop constraint if exists "field_visits_shop_id_fkey";
alter table public."field_visits" add constraint "field_visits_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete set null;

alter table public."field_visits" drop constraint if exists "field_visits_executive_id_fkey";
alter table public."field_visits" add constraint "field_visits_executive_id_fkey" foreign key ("executive_id") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."field_visit_checklist_items" drop constraint if exists "field_visit_checklist_items_field_visit_id_fkey";
alter table public."field_visit_checklist_items" add constraint "field_visit_checklist_items_field_visit_id_fkey" foreign key ("field_visit_id") references public."field_visits" ("id") on update cascade on delete restrict;

alter table public."field_visit_checklist_items" drop constraint if exists "field_visit_checklist_items_completed_by_fkey";
alter table public."field_visit_checklist_items" add constraint "field_visit_checklist_items_completed_by_fkey" foreign key ("completed_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."product_moderation_cases" drop constraint if exists "product_moderation_cases_product_id_fkey";
alter table public."product_moderation_cases" add constraint "product_moderation_cases_product_id_fkey" foreign key ("product_id") references public."products" ("id") on update cascade on delete restrict;

alter table public."product_moderation_cases" drop constraint if exists "product_moderation_cases_moderator_id_fkey";
alter table public."product_moderation_cases" add constraint "product_moderation_cases_moderator_id_fkey" foreign key ("moderator_id") references public."profiles" ("id") on update cascade on delete set null;

alter table public."approval_requests" drop constraint if exists "approval_requests_requested_by_fkey";
alter table public."approval_requests" add constraint "approval_requests_requested_by_fkey" foreign key ("requested_by") references public."profiles" ("id") on update cascade on delete restrict;

alter table public."approval_requests" drop constraint if exists "approval_requests_approved_by_fkey";
alter table public."approval_requests" add constraint "approval_requests_approved_by_fkey" foreign key ("approved_by") references public."profiles" ("id") on update cascade on delete set null;

alter table public."fraud_cases" drop constraint if exists "fraud_cases_subject_user_id_fkey";
alter table public."fraud_cases" add constraint "fraud_cases_subject_user_id_fkey" foreign key ("subject_user_id") references public."profiles" ("id") on update cascade on delete set null;

alter table public."fraud_cases" drop constraint if exists "fraud_cases_shop_id_fkey";
alter table public."fraud_cases" add constraint "fraud_cases_shop_id_fkey" foreign key ("shop_id") references public."shops" ("id") on update cascade on delete set null;

alter table public."fraud_cases" drop constraint if exists "fraud_cases_order_id_fkey";
alter table public."fraud_cases" add constraint "fraud_cases_order_id_fkey" foreign key ("order_id") references public."orders" ("id") on update cascade on delete set null;

alter table public."fraud_cases" drop constraint if exists "fraud_cases_assigned_to_fkey";
alter table public."fraud_cases" add constraint "fraud_cases_assigned_to_fkey" foreign key ("assigned_to") references public."profiles" ("id") on update cascade on delete set null;

alter table public."audit_logs" drop constraint if exists "audit_logs_actor_user_id_fkey";
alter table public."audit_logs" add constraint "audit_logs_actor_user_id_fkey" foreign key ("actor_user_id") references public."profiles" ("id") on update cascade on delete set null;

alter table public."audit_logs" drop constraint if exists "audit_logs_device_id_fkey";
alter table public."audit_logs" add constraint "audit_logs_device_id_fkey" foreign key ("device_id") references public."user_devices" ("id") on update cascade on delete set null;
```

```
alter table public."system_settings" drop constraint if exists "system_settings_updated_by_fkey";
alter table public."system_settings" add constraint "system_settings_updated_by_fkey" foreign key ("updated_by") references public."profiles" ("id") on update cascade on delete restrict;
```

0010_triggers_views_rpc.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


drop trigger if exists set_profiles_updated_at on public."profiles";
create trigger set_profiles_updated_at before update on public."profiles" for each row execute procedure private.set_updated_at();

drop trigger if exists set_user_devices_updated_at on public."user_devices";
create trigger set_user_devices_updated_at before update on public."user_devices" for each row execute procedure private.set_updated_at();

drop trigger if exists set_addresses_updated_at on public."addresses";
create trigger set_addresses_updated_at before update on public."addresses" for each row execute procedure private.set_updated_at();

drop trigger if exists set_customer_profiles_updated_at on public."customer_profiles";
create trigger set_customer_profiles_updated_at before update on public."customer_profiles" for each row execute procedure private.set_updated_at();

drop trigger if exists set_customer_preferences_updated_at on public."customer_preferences";
create trigger set_customer_preferences_updated_at before update on public."customer_preferences" for each row execute procedure private.set_updated_at();

drop trigger if exists set_merchant_profiles_updated_at on public."merchant_profiles";
create trigger set_merchant_profiles_updated_at before update on public."merchant_profiles" for each row execute procedure private.set_updated_at();

drop trigger if exists set_captain_profiles_updated_at on public."captain_profiles";
create trigger set_captain_profiles_updated_at before update on public."captain_profiles" for each row execute procedure private.set_updated_at();

drop trigger if exists set_admin_profiles_updated_at on public."admin_profiles";
create trigger set_admin_profiles_updated_at before update on public."admin_profiles" for each row execute procedure private.set_updated_at();

drop trigger if exists set_shops_updated_at on public."shops";
create trigger set_shops_updated_at before update on public."shops" for each row execute procedure private.set_updated_at();

drop trigger if exists set_shop_hours_updated_at on public."shop_hours";
create trigger set_shop_hours_updated_at before update on public."shop_hours" for each row execute procedure private.set_updated_at();

drop trigger if exists set_shop_bank_accounts_updated_at on public."shop_bank_accounts";
create trigger set_shop_bank_accounts_updated_at before update on public."shop_bank_accounts" for each row execute procedure private.set_updated_at();

drop trigger if exists set_categories_updated_at on public."categories";
create trigger set_categories_updated_at before update on public."categories" for each row execute procedure private.set_updated_at();

drop trigger if exists set_products_updated_at on public."products";
create trigger set_products_updated_at before update on public."products" for each row execute procedure private.set_updated_at();

drop trigger if exists set_product_variants_updated_at on public."product_variants";
create trigger set_product_variants_updated_at before update on public."product_variants" for each row execute procedure private.set_updated_at();

drop trigger if exists set_inventory_balances_updated_at on public."inventory_balances";
create trigger set_inventory_balances_updated_at before update on public."inventory_balances" for each row execute procedure private.set_updated_at();

drop trigger if exists set_shop_collections_updated_at on public."shop_collections";
create trigger set_shop_collections_updated_at before update on public."shop_collections" for each row execute procedure private.set_updated_at();

drop trigger if exists set_offers_updated_at on public."offers";
create trigger set_offers_updated_at before update on public."offers" for each row execute procedure private.set_updated_at();

drop trigger if exists set_carts_updated_at on public."carts";
create trigger set_carts_updated_at before update on public."carts" for each row execute procedure private.set_updated_at();

drop trigger if exists set_cart_items_updated_at on public."cart_items";
create trigger set_cart_items_updated_at before update on public."cart_items" for each row execute procedure private.set_updated_at();

drop trigger if exists set_orders_updated_at on public."orders";
create trigger set_orders_updated_at before update on public."orders" for each row execute procedure private.set_updated_at();

drop trigger if exists set_payments_updated_at on public."payments";
create trigger set_payments_updated_at before update on public."payments" for each row execute procedure private.set_updated_at();

drop trigger if exists set_return_requests_updated_at on public."return_requests";
create trigger set_return_requests_updated_at before update on public."return_requests" for each row execute procedure private.set_updated_at();

drop trigger if exists set_refunds_updated_at on public."refunds";
create trigger set_refunds_updated_at before update on public."refunds" for each row execute procedure private.set_updated_at();
```

```
drop trigger if exists set_merchant_settlements_updated_at on public."merchant_settlements";
create trigger set_merchant_settlements_updated_at before update on public."merchant_settlements" for each row execute procedure private.set_updated_at();

drop trigger if exists set_delivery_tasks_updated_at on public."delivery_tasks";
create trigger set_delivery_tasks_updated_at before update on public."delivery_tasks" for each row execute procedure private.set_updated_at();

drop trigger if exists set_captain_current_locations_updated_at on public."captain_current_locations";
create trigger set_captain_current_locations_updated_at before update on public."captain_current_locations" for each row execute procedure private.set_updated_at();

drop trigger if exists set_captain_payouts_updated_at on public."captain_payouts";
create trigger set_captain_payouts_updated_at before update on public."captain_payouts" for each row execute procedure private.set_updated_at();

drop trigger if exists set_style_groups_updated_at on public."style_groups";
create trigger set_style_groups_updated_at before update on public."style_groups" for each row execute procedure private.set_updated_at();

drop trigger if exists set_style_group_member_preferences_updated_at on public."style_group_member_preferences";
create trigger set_style_group_member_preferences_updated_at before update on public."style_group_member_preferences" for each row execute procedure private.set_updated_at();

drop trigger if exists set_style_themes_updated_at on public."style_themes";
create trigger set_style_themes_updated_at before update on public."style_themes" for each row execute procedure private.set_updated_at();

drop trigger if exists set_style_moodboards_updated_at on public."style_moodboards";
create trigger set_style_moodboards_updated_at before update on public."style_moodboards" for each row execute procedure private.set_updated_at();

drop trigger if exists set_notification_preferences_updated_at on public."notification_preferences";
create trigger set_notification_preferences_updated_at before update on public."notification_preferences" for each row execute procedure private.set_updated_at();

drop trigger if exists set_reviews_updated_at on public."reviews";
create trigger set_reviews_updated_at before update on public."reviews" for each row execute procedure private.set_updated_at();

drop trigger if exists set_product_embeddings_updated_at on public."product_embeddings";
create trigger set_product_embeddings_updated_at before update on public."product_embeddings" for each row execute procedure private.set_updated_at();

drop trigger if exists set_customer_embeddings_updated_at on public."customer_embeddings";
create trigger set_customer_embeddings_updated_at before update on public."customer_embeddings" for each row execute procedure private.set_updated_at();

drop trigger if exists set_size_charts_updated_at on public."size_charts";
create trigger set_size_charts_updated_at before update on public."size_charts" for each row execute procedure private.set_updated_at();

drop trigger if exists set_customer_body_profiles_updated_at on public."customer_body_profiles";
create trigger set_customer_body_profiles_updated_at before update on public."customer_body_profiles" for each row execute procedure private.set_updated_at();

drop trigger if exists set_support_tickets_updated_at on public."support_tickets";
create trigger set_support_tickets_updated_at before update on public."support_tickets" for each row execute procedure private.set_updated_at();

drop trigger if exists set_campaigns_updated_at on public."campaigns";
create trigger set_campaigns_updated_at before update on public."campaigns" for each row execute procedure private.set_updated_at();

drop trigger if exists set_merchant_leads_updated_at on public."merchant_leads";
create trigger set_merchant_leads_updated_at before update on public."merchant_leads" for each row execute procedure private.set_updated_at();

drop trigger if exists set_field_visits_updated_at on public."field_visits";
create trigger set_field_visits_updated_at before update on public."field_visits" for each row execute procedure private.set_updated_at();

drop trigger if exists set_system_settings_updated_at on public."system_settings";
create trigger set_system_settings_updated_at before update on public."system_settings" for each row execute procedure private.set_updated_at();

drop trigger if exists set_service_zones_updated_at on public."service_zones";
create trigger set_service_zones_updated_at before update on public."service_zones" for each row execute procedure private.set_updated_at();


-- Safe computed inventory availability.
create or replace view public.inventory_availability
with (security_invoker = true)
as
select
  ib.id,
  ib.shop_id,
  ib.variant_id,
  ib.stock_on_hand,
  ib.reserved_quantity,
  ib.damaged_quantity,
  greatest(ib.stock_on_hand - ib.reserved_quantity - ib.damaged_quantity, 0) as available_quantity,
  ib.reorder_level,
  ib.version,
  ib.updated_at
from public.inventory_balances ib;
```

```
-- Catalogue search vector. This can support MVP search before moving to Typesense/OpenSearch.
create or replace function private.products_search_vector()
returns trigger
language plpgsql
security invoker
set search_path = ''
as $$
begin
  new.search_vector :=
    setweight(to_tsvector('simple', coalesce(new.name,'')), 'A') ||
    setweight(to_tsvector('simple', coalesce(new.brand,'')), 'B') ||
    setweight(to_tsvector('simple', coalesce(new.description,'')), 'C');
  return new;
end;
$$;

drop trigger if exists products_search_vector_trigger on public.products;
create trigger products_search_vector_trigger
before insert or update of name, brand, description on public.products
for each row execute procedure private.products_search_vector();

-- RLS helper: merchant owns a shop.
create or replace function private.owns_shop(p_shop_id uuid)
returns boolean
language sql
stable
security definer
set search_path = ''
as $$
  select exists (
    select 1 from public.shops s
    where s.id = p_shop_id
      and s.merchant_id = (select auth.uid())
      and s.deleted_at is null
  );
$$;
revoke all on function private.owns_shop(uuid) from public;
grant execute on function private.owns_shop(uuid) to authenticated;

-- RLS helper: user has an active permission through any active role.
create or replace function private.has_permission(p_permission text)
returns boolean
language sql
stable
security definer
set search_path = ''
as $$
  select exists (
    select 1
    from public.user_roles ur
    join public.role_permissions rp on rp.role_id = ur.role_id
    join public.permissions p on p.id = rp.permission_id
    where ur.user_id = (select auth.uid())
      and ur.revoked_at is null
      and p.code = p_permission
  );
$$;
revoke all on function private.has_permission(text) from public;
grant execute on function private.has_permission(text) to authenticated;

-- RLS helper: current user is a member or creator of a Group Style group.
create or replace function private.is_style_group_member(p_group_id uuid)
returns boolean
language sql
stable
security definer
set search_path = ''
as $$
  select exists (
    select 1 from public.style_groups g
    where g.id = p_group_id and g.creator_customer_id = (select auth.uid())
  ) or exists (
    select 1 from public.style_group_members m
    where m.group_id = p_group_id
      and m.customer_id = (select auth.uid())
      and m.status = 'JOINED'
  );
$$;
```

```
revoke all on function private.is_style_group_member(uuid) from public;
grant execute on function private.is_style_group_member(uuid) to authenticated;

-- Strict inventory adjustment primitive. Call only from trusted backend/Edge Function.
create or replace function private.apply_inventory_delta(
  p_shop_id uuid,
  p_variant_id uuid,
  p_stock_delta integer,
  p_reserved_delta integer,
  p_damaged_delta integer,
  p_movement_type text,
  p_source_method text,
  p_reference_type text default null,
  p_reference_id uuid default null,
  p_reason text default null,
  p_actor uuid default null
)
returns public.inventory_balances
language plpgsql
security definer
set search_path = ''
as $$
declare
  before_row public.inventory_balances;
  after_row public.inventory_balances;
begin
  select * into before_row
  from public.inventory_balances
  where shop_id = p_shop_id and variant_id = p_variant_id
  for update;

  if not found then
    insert into public.inventory_balances(shop_id, variant_id)
    values (p_shop_id, p_variant_id)
    returning * into before_row;
  end if;

  update public.inventory_balances
  set stock_on_hand = stock_on_hand + p_stock_delta,
      reserved_quantity = reserved_quantity + p_reserved_delta,
      damaged_quantity = damaged_quantity + p_damaged_delta,
      version = version + 1
  where id = before_row.id
  returning * into after_row;

  if after_row.stock_on_hand < 0
     or after_row.reserved_quantity < 0
     or after_row.damaged_quantity < 0
     or after_row.stock_on_hand < after_row.reserved_quantity + after_row.damaged_quantity then
    raise exception 'Invalid inventory delta for variant %', p_variant_id;
  end if;

  insert into public.inventory_movements(
    shop_id, variant_id, movement_type,
    quantity_change, reserved_change, damaged_change,
    stock_before, stock_after, reserved_before, reserved_after,
    damaged_before, damaged_after, reference_type, reference_id,
    reason, performed_by, source_method
  ) values (
    p_shop_id, p_variant_id, p_movement_type,
    p_stock_delta, p_reserved_delta, p_damaged_delta,
    before_row.stock_on_hand, after_row.stock_on_hand,
    before_row.reserved_quantity, after_row.reserved_quantity,
    before_row.damaged_quantity, after_row.damaged_quantity,
    p_reference_type, p_reference_id, p_reason, p_actor, p_source_method
  );
  return after_row;
end;
$$;
revoke all on function private.apply_inventory_delta(uuid,uuid,integer,integer,integer,text,text,text,uuid,text,uuid) from public, anon, authenticated;
```

0011_rls_policies.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Enable RLS on every exposed public table. No policy means deny-by-default.

alter table public."profiles" enable row level security;
alter table public."profiles" force row level security;
alter table public."user_devices" enable row level security;
alter table public."user_devices" force row level security;
alter table public."addresses" enable row level security;
alter table public."addresses" force row level security;
alter table public."customer_profiles" enable row level security;
alter table public."customer_profiles" force row level security;
alter table public."customer_preferences" enable row level security;
alter table public."customer_preferences" force row level security;
alter table public."merchant_profiles" enable row level security;
alter table public."merchant_profiles" force row level security;
alter table public."captain_profiles" enable row level security;
alter table public."captain_profiles" force row level security;
alter table public."admin_profiles" enable row level security;
alter table public."admin_profiles" force row level security;
alter table public."roles" enable row level security;
alter table public."roles" force row level security;
alter table public."permissions" enable row level security;
alter table public."permissions" force row level security;
alter table public."user_roles" enable row level security;
alter table public."user_roles" force row level security;
alter table public."role_permissions" enable row level security;
alter table public."role_permissions" force row level security;
alter table public."shops" enable row level security;
alter table public."shops" force row level security;
alter table public."shop_hours" enable row level security;
alter table public."shop_hours" force row level security;
alter table public."shop_documents" enable row level security;
alter table public."shop_documents" force row level security;
alter table public."shop_bank_accounts" enable row level security;
alter table public."shop_bank_accounts" force row level security;
alter table public."categories" enable row level security;
alter table public."categories" force row level security;
alter table public."products" enable row level security;
alter table public."products" force row level security;
alter table public."product_images" enable row level security;
alter table public."product_images" force row level security;
alter table public."product_variants" enable row level security;
alter table public."product_variants" force row level security;
alter table public."variant_barcodes" enable row level security;
alter table public."variant_barcodes" force row level security;
alter table public."inventory_balances" enable row level security;
alter table public."inventory_balances" force row level security;
alter table public."inventory_movements" enable row level security;
alter table public."inventory_movements" force row level security;
alter table public."inventory_reservations" enable row level security;
alter table public."inventory_reservations" force row level security;
alter table public."product_photo_matches" enable row level security;
alter table public."product_photo_matches" force row level security;
alter table public."shop_favourites" enable row level security;
alter table public."shop_favourites" force row level security;
alter table public."wishlist_items" enable row level security;
alter table public."wishlist_items" force row level security;
alter table public."shop_collections" enable row level security;
alter table public."shop_collections" force row level security;
alter table public."shop_collection_items" enable row level security;
alter table public."shop_collection_items" force row level security;
alter table public."offers" enable row level security;
alter table public."offers" force row level security;
alter table public."offer_products" enable row level security;
alter table public."offer_products" force row level security;
alter table public."offer_redemptions" enable row level security;
alter table public."offer_redemptions" force row level security;
alter table public."offline_sales" enable row level security;
alter table public."offline_sales" force row level security;
alter table public."offline_sale_items" enable row level security;
```

```
alter table public."offline_sale_items" force row level security;
alter table public."carts" enable row level security;
alter table public."carts" force row level security;
alter table public."cart_items" enable row level security;
alter table public."cart_items" force row level security;
alter table public."orders" enable row level security;
alter table public."orders" force row level security;
alter table public."order_items" enable row level security;
alter table public."order_items" force row level security;
alter table public."order_status_history" enable row level security;
alter table public."order_status_history" force row level security;
alter table public."merchant_order_alerts" enable row level security;
alter table public."merchant_order_alerts" force row level security;
alter table public."merchant_order_issues" enable row level security;
alter table public."merchant_order_issues" force row level security;
alter table public."order_item_verifications" enable row level security;
alter table public."order_item_verifications" force row level security;
alter table public."payments" enable row level security;
alter table public."payments" force row level security;
alter table public."payment_events" enable row level security;
alter table public."payment_events" force row level security;
alter table public."return_requests" enable row level security;
alter table public."return_requests" force row level security;
alter table public."return_items" enable row level security;
alter table public."return_items" force row level security;
alter table public."return_evidence" enable row level security;
alter table public."return_evidence" force row level security;
alter table public."return_status_history" enable row level security;
alter table public."return_status_history" force row level security;
alter table public."refunds" enable row level security;
alter table public."refunds" force row level security;
alter table public."merchant_settlements" enable row level security;
alter table public."merchant_settlements" force row level security;
alter table public."merchant_settlement_items" enable row level security;
alter table public."merchant_settlement_items" force row level security;
alter table public."captain_documents" enable row level security;
alter table public."captain_documents" force row level security;
alter table public."delivery_tasks" enable row level security;
alter table public."delivery_tasks" force row level security;
alter table public."delivery_assignments" enable row level security;
alter table public."delivery_assignments" force row level security;
alter table public."captain_current_locations" enable row level security;
alter table public."captain_current_locations" force row level security;
alter table public."captain_location_history" enable row level security;
alter table public."captain_location_history" force row level security;
alter table public."delivery_events" enable row level security;
alter table public."delivery_events" force row level security;
alter table public."cod_collections" enable row level security;
alter table public."cod_collections" force row level security;
alter table public."captain_earnings" enable row level security;
alter table public."captain_earnings" force row level security;
alter table public."captain_payouts" enable row level security;
alter table public."captain_payouts" force row level security;
alter table public."captain_payout_items" enable row level security;
alter table public."captain_payout_items" force row level security;
alter table public."style_groups" enable row level security;
alter table public."style_groups" force row level security;
alter table public."style_group_members" enable row level security;
alter table public."style_group_members" force row level security;
alter table public."style_group_member_preferences" enable row level security;
alter table public."style_group_member_preferences" force row level security;
alter table public."style_themes" enable row level security;
alter table public."style_themes" force row level security;
alter table public."style_polls" enable row level security;
alter table public."style_polls" force row level security;
alter table public."style_poll_options" enable row level security;
alter table public."style_poll_options" force row level security;
alter table public."style_poll_votes" enable row level security;
alter table public."style_poll_votes" force row level security;
alter table public."style_moodboards" enable row level security;
alter table public."style_moodboards" force row level security;
alter table public."style_moodboard_items" enable row level security;
alter table public."style_moodboard_items" force row level security;
alter table public."notifications" enable row level security;
alter table public."notifications" force row level security;
alter table public."notification_deliveries" enable row level security;
alter table public."notification_deliveries" force row level security;
alter table public."notification_preferences" enable row level security;
```

```
alter table public."notification_preferences" force row level security;
alter table public."reviews" enable row level security;
alter table public."reviews" force row level security;
alter table public."customer_events" enable row level security;
alter table public."customer_events" force row level security;
alter table public."recommendation_sets" enable row level security;
alter table public."recommendation_sets" force row level security;
alter table public."recommendation_items" enable row level security;
alter table public."recommendation_items" force row level security;
alter table public."product_embeddings" enable row level security;
alter table public."product_embeddings" force row level security;
alter table public."customer_embeddings" enable row level security;
alter table public."customer_embeddings" force row level security;
alter table public."size_charts" enable row level security;
alter table public."size_charts" force row level security;
alter table public."size_chart_measurements" enable row level security;
alter table public."size_chart_measurements" force row level security;
alter table public."customer_body_profiles" enable row level security;
alter table public."customer_body_profiles" force row level security;
alter table public."body_measurements" enable row level security;
alter table public."body_measurements" force row level security;
alter table public."body_scan_sessions" enable row level security;
alter table public."body_scan_sessions" force row level security;
alter table public."fit_recommendations" enable row level security;
alter table public."fit_recommendations" force row level security;
alter table public."fit_feedback" enable row level security;
alter table public."fit_feedback" force row level security;
alter table public."virtual_tryon_sessions" enable row level security;
alter table public."virtual_tryon_sessions" force row level security;
alter table public."virtual_tryon_outputs" enable row level security;
alter table public."virtual_tryon_outputs" force row level security;
alter table public."style_recommendation_sets" enable row level security;
alter table public."style_recommendation_sets" force row level security;
alter table public."style_recommendation_items" enable row level security;
alter table public."style_recommendation_items" force row level security;
alter table public."support_tickets" enable row level security;
alter table public."support_tickets" force row level security;
alter table public."support_messages" enable row level security;
alter table public."support_messages" force row level security;
alter table public."campaigns" enable row level security;
alter table public."campaigns" force row level security;
alter table public."banners" enable row level security;
alter table public."banners" force row level security;
alter table public."merchant_leads" enable row level security;
alter table public."merchant_leads" force row level security;
alter table public."field_visits" enable row level security;
alter table public."field_visits" force row level security;
alter table public."field_visit_checklist_items" enable row level security;
alter table public."field_visit_checklist_items" force row level security;
alter table public."product_moderation_cases" enable row level security;
alter table public."product_moderation_cases" force row level security;
alter table public."approval_requests" enable row level security;
alter table public."approval_requests" force row level security;
alter table public."fraud_cases" enable row level security;
alter table public."fraud_cases" force row level security;
alter table public."audit_logs" enable row level security;
alter table public."audit_logs" force row level security;
alter table public."system_settings" enable row level security;
alter table public."system_settings" force row level security;
alter table public."service_zones" enable row level security;
alter table public."service_zones" force row level security;

-- Basic grants: policies still decide which rows are accessible.

grant select, insert, update, delete on public."profiles" to authenticated;
grant select, insert, update, delete on public."user_devices" to authenticated;
grant select, insert, update, delete on public."addresses" to authenticated;
grant select, insert, update, delete on public."customer_profiles" to authenticated;
grant select, insert, update, delete on public."customer_preferences" to authenticated;
grant select, insert, update, delete on public."merchant_profiles" to authenticated;
grant select, insert, update, delete on public."captain_profiles" to authenticated;
grant select, insert, update, delete on public."admin_profiles" to authenticated;
grant select, insert, update, delete on public."roles" to authenticated;
grant select, insert, update, delete on public."permissions" to authenticated;
grant select, insert, update, delete on public."user_roles" to authenticated;
grant select, insert, update, delete on public."role_permissions" to authenticated;
grant select, insert, update, delete on public."shops" to authenticated;
grant select, insert, update, delete on public."shop_hours" to authenticated;
```

```
grant select, insert, update, delete on public."shop_documents" to authenticated;
grant select, insert, update, delete on public."shop_bank_accounts" to authenticated;
grant select, insert, update, delete on public."categories" to authenticated;
grant select, insert, update, delete on public."products" to authenticated;
grant select, insert, update, delete on public."product_images" to authenticated;
grant select, insert, update, delete on public."product_variants" to authenticated;
grant select, insert, update, delete on public."variant_barcodes" to authenticated;
grant select, insert, update, delete on public."inventory_balances" to authenticated;
grant select, insert, update, delete on public."inventory_movements" to authenticated;
grant select, insert, update, delete on public."inventory_reservations" to authenticated;
grant select, insert, update, delete on public."product_photo_matches" to authenticated;
grant select, insert, update, delete on public."shop_favourites" to authenticated;
grant select, insert, update, delete on public."wishlist_items" to authenticated;
grant select, insert, update, delete on public."shop_collections" to authenticated;
grant select, insert, update, delete on public."shop_collection_items" to authenticated;
grant select, insert, update, delete on public."offers" to authenticated;
grant select, insert, update, delete on public."offer_products" to authenticated;
grant select, insert, update, delete on public."offer_redemptions" to authenticated;
grant select, insert, update, delete on public."offline_sales" to authenticated;
grant select, insert, update, delete on public."offline_sale_items" to authenticated;
grant select, insert, update, delete on public."carts" to authenticated;
grant select, insert, update, delete on public."cart_items" to authenticated;
grant select, insert, update, delete on public."orders" to authenticated;
grant select, insert, update, delete on public."order_items" to authenticated;
grant select, insert, update, delete on public."order_status_history" to authenticated;
grant select, insert, update, delete on public."merchant_order_alerts" to authenticated;
grant select, insert, update, delete on public."merchant_order_issues" to authenticated;
grant select, insert, update, delete on public."order_item_verifications" to authenticated;
grant select, insert, update, delete on public."payments" to authenticated;
grant select, insert, update, delete on public."payment_events" to authenticated;
grant select, insert, update, delete on public."return_requests" to authenticated;
grant select, insert, update, delete on public."return_items" to authenticated;
grant select, insert, update, delete on public."return_evidence" to authenticated;
grant select, insert, update, delete on public."return_status_history" to authenticated;
grant select, insert, update, delete on public."refunds" to authenticated;
grant select, insert, update, delete on public."merchant_settlements" to authenticated;
grant select, insert, update, delete on public."merchant_settlement_items" to authenticated;
grant select, insert, update, delete on public."captain_documents" to authenticated;
grant select, insert, update, delete on public."delivery_tasks" to authenticated;
grant select, insert, update, delete on public."delivery_assignments" to authenticated;
grant select, insert, update, delete on public."captain_current_locations" to authenticated;
grant select, insert, update, delete on public."captain_location_history" to authenticated;
grant select, insert, update, delete on public."delivery_events" to authenticated;
grant select, insert, update, delete on public."cod_collections" to authenticated;
grant select, insert, update, delete on public."captain_earnings" to authenticated;
grant select, insert, update, delete on public."captain_payouts" to authenticated;
grant select, insert, update, delete on public."captain_payout_items" to authenticated;
grant select, insert, update, delete on public."style_groups" to authenticated;
grant select, insert, update, delete on public."style_group_members" to authenticated;
grant select, insert, update, delete on public."style_group_member_preferences" to authenticated;
grant select, insert, update, delete on public."style_themes" to authenticated;
grant select, insert, update, delete on public."style_polls" to authenticated;
grant select, insert, update, delete on public."style_poll_options" to authenticated;
grant select, insert, update, delete on public."style_poll_votes" to authenticated;
grant select, insert, update, delete on public."style_moodboards" to authenticated;
grant select, insert, update, delete on public."style_moodboard_items" to authenticated;
grant select, insert, update, delete on public."notifications" to authenticated;
grant select, insert, update, delete on public."notification_deliveries" to authenticated;
grant select, insert, update, delete on public."notification_preferences" to authenticated;
grant select, insert, update, delete on public."reviews" to authenticated;
grant select, insert, update, delete on public."customer_events" to authenticated;
grant select, insert, update, delete on public."recommendation_sets" to authenticated;
grant select, insert, update, delete on public."recommendation_items" to authenticated;
grant select, insert, update, delete on public."product_embeddings" to authenticated;
grant select, insert, update, delete on public."customer_embeddings" to authenticated;
grant select, insert, update, delete on public."size_charts" to authenticated;
grant select, insert, update, delete on public."size_chart_measurements" to authenticated;
grant select, insert, update, delete on public."customer_body_profiles" to authenticated;
grant select, insert, update, delete on public."body_measurements" to authenticated;
grant select, insert, update, delete on public."body_scan_sessions" to authenticated;
grant select, insert, update, delete on public."fit_recommendations" to authenticated;
grant select, insert, update, delete on public."fit_feedback" to authenticated;
grant select, insert, update, delete on public."virtual_tryon_sessions" to authenticated;
grant select, insert, update, delete on public."virtual_tryon_outputs" to authenticated;
grant select, insert, update, delete on public."style_recommendation_sets" to authenticated;
grant select, insert, update, delete on public."style_recommendation_items" to authenticated;
grant select, insert, update, delete on public."support_tickets" to authenticated;
grant select, insert, update, delete on public."support_messages" to authenticated;
```

```
grant select, insert, update, delete on public."campaigns" to authenticated;
grant select, insert, update, delete on public."banners" to authenticated;
grant select, insert, update, delete on public."merchant_leads" to authenticated;
grant select, insert, update, delete on public."field_visits" to authenticated;
grant select, insert, update, delete on public."field_visit_checklist_items" to authenticated;
grant select, insert, update, delete on public."product_moderation_cases" to authenticated;
grant select, insert, update, delete on public."approval_requests" to authenticated;
grant select, insert, update, delete on public."fraud_cases" to authenticated;
grant select, insert, update, delete on public."audit_logs" to authenticated;
grant select, insert, update, delete on public."system_settings" to authenticated;
grant select, insert, update, delete on public."service_zones" to authenticated;
grant select on public."shops" to anon;
create policy "shops_public_read" on public."shops" for select to anon, authenticated using (deleted_at is null and verification_status = 'VERIFIED' and operational_status not in
('SUSPENDED','PAUSED'));
grant select on public."shop_hours" to anon;
create policy "shop_hours_public_read" on public."shop_hours" for select to anon, authenticated using (true);
grant select on public."categories" to anon;
create policy "categories_public_read" on public."categories" for select to anon, authenticated using (true);
grant select on public."products" to anon;
create policy "products_public_read" on public."products" for select to anon, authenticated using (deleted_at is null and is_active and moderation_status = 'APPROVED');
grant select on public."product_images" to anon;
create policy "product_images_public_read" on public."product_images" for select to anon, authenticated using (true);
grant select on public."product_variants" to anon;
create policy "product_variants_public_read" on public."product_variants" for select to anon, authenticated using (is_active);
grant select on public."variant_barcodes" to anon;
create policy "variant_barcodes_public_read" on public."variant_barcodes" for select to anon, authenticated using (true);
grant select on public."shop_collections" to anon;
create policy "shop_collections_public_read" on public."shop_collections" for select to anon, authenticated using (true);
grant select on public."shop_collection_items" to anon;
create policy "shop_collection_items_public_read" on public."shop_collection_items" for select to anon, authenticated using (true);
grant select on public."offers" to anon;
create policy "offers_public_read" on public."offers" for select to anon, authenticated using (status = 'ACTIVE' and now() between starts_at and ends_at);
grant select on public."offer_products" to anon;
create policy "offer_products_public_read" on public."offer_products" for select to anon, authenticated using (true);
grant select on public."reviews" to anon;
create policy "reviews_public_read" on public."reviews" for select to anon, authenticated using (true);
grant select on public."service_zones" to anon;
create policy "service_zones_public_read" on public."service_zones" for select to anon, authenticated using (true);
grant select on public."banners" to anon;
create policy "banners_public_read" on public."banners" for select to anon, authenticated using (true);
grant select on public."campaigns" to anon;
create policy "campaigns_public_read" on public."campaigns" for select to anon, authenticated using (status = 'ACTIVE' and now() between starts_at and ends_at);
create policy "profiles_read_self" on public."profiles" for select to authenticated using (id = (select auth.uid()));
create policy "profiles_update_self" on public."profiles" for update to authenticated using (id = (select auth.uid())) with check (id = (select auth.uid()));
create policy "devices_self" on public."user_devices" for all to authenticated using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
create policy "addresses_self" on public."addresses" for all to authenticated using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
create policy "customer_profile_self" on public."customer_profiles" for all to authenticated using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
create policy "customer_preferences_self" on public."customer_preferences" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "notification_preferences_self" on public."notification_preferences" for all to authenticated using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
create policy "notifications_self_read" on public."notifications" for select to authenticated using (user_id = (select auth.uid()));
create policy "notifications_self_update" on public."notifications" for update to authenticated using (user_id = (select auth.uid())) with check (user_id = (select auth.uid()));
create policy "shops_merchant_read" on public."shops" for select to authenticated using ((select private.owns_shop(id)));
create policy "shops_merchant_write" on public."shops" for all to authenticated using ((select private.owns_shop(id))) with check ((select private.owns_shop(id)));
create policy "shop_hours_merchant_read" on public."shop_hours" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "shop_hours_merchant_write" on public."shop_hours" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "shop_documents_merchant_read" on public."shop_documents" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "shop_documents_merchant_write" on public."shop_documents" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "shop_bank_accounts_merchant_read" on public."shop_bank_accounts" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "shop_bank_accounts_merchant_write" on public."shop_bank_accounts" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "products_merchant_read" on public."products" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "products_merchant_write" on public."products" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "product_variants_merchant_read" on public."product_variants" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "product_variants_merchant_write" on public."product_variants" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "inventory_balances_merchant_read" on public."inventory_balances" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "inventory_balances_merchant_write" on public."inventory_balances" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "inventory_movements_merchant_read" on public."inventory_movements" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "inventory_reservations_merchant_read" on public."inventory_reservations" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "inventory_reservations_merchant_write" on public."inventory_reservations" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select
private.owns_shop(shop_id)));
create policy "product_photo_matches_merchant_read" on public."product_photo_matches" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "product_photo_matches_merchant_write" on public."product_photo_matches" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select
private.owns_shop(shop_id)));
create policy "shop_collections_merchant_read" on public."shop_collections" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "shop_collections_merchant_write" on public."shop_collections" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "offers_merchant_read" on public."offers" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "offers_merchant_write" on public."offers" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "offline_sales_merchant_read" on public."offline_sales" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "offline_sales_merchant_write" on public."offline_sales" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
```

```
create policy "orders_merchant_read" on public."orders" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "orders_merchant_write" on public."orders" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select private.owns_shop(shop_id)));
create policy "merchant_order_alerts_merchant_read" on public."merchant_order_alerts" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "merchant_order_alerts_merchant_write" on public."merchant_order_alerts" for all to authenticated using ((select private.owns_shop(shop_id))) with check ((select
private.owns_shop(shop_id)));
create policy "merchant_settlements_merchant_read" on public."merchant_settlements" for select to authenticated using ((select private.owns_shop(shop_id)));
create policy "product_images_merchant_access" on public."product_images" for all to authenticated using (exists (select 1 from public.products p where p.id = product_id and
private.owns_shop(p.shop_id))) with check (exists (select 1 from public.products p where p.id = product_id and private.owns_shop(p.shop_id)));
create policy "variant_barcodes_merchant_access" on public."variant_barcodes" for all to authenticated using (exists (select 1 from public.product_variants v where v.id = variant_id and
private.owns_shop(v.shop_id))) with check (exists (select 1 from public.product_variants v where v.id = variant_id and private.owns_shop(v.shop_id)));
create policy "shop_collection_items_merchant_access" on public."shop_collection_items" for all to authenticated using (exists (select 1 from public.shop_collections c where c.id = collection_id and
private.owns_shop(c.shop_id))) with check (exists (select 1 from public.shop_collections c where c.id = collection_id and private.owns_shop(c.shop_id)));
create policy "offer_products_merchant_access" on public."offer_products" for all to authenticated using (exists (select 1 from public.offers o where o.id = offer_id and
private.owns_shop(o.shop_id))) with check (exists (select 1 from public.offers o where o.id = offer_id and private.owns_shop(o.shop_id)));
create policy "offline_sale_items_merchant_access" on public."offline_sale_items" for all to authenticated using (exists (select 1 from public.offline_sales s where s.id = offline_sale_id and
private.owns_shop(s.shop_id))) with check (exists (select 1 from public.offline_sales s where s.id = offline_sale_id and private.owns_shop(s.shop_id)));
create policy "order_items_merchant_access" on public."order_items" for all to authenticated using (exists (select 1 from public.orders o where o.id = order_id and private.owns_shop(o.shop_id))) with
check (exists (select 1 from public.orders o where o.id = order_id and private.owns_shop(o.shop_id)));
create policy "merchant_order_issues_merchant_access" on public."merchant_order_issues" for all to authenticated using (exists (select 1 from public.orders o where o.id = order_id and
private.owns_shop(o.shop_id))) with check (exists (select 1 from public.orders o where o.id = order_id and private.owns_shop(o.shop_id)));
create policy "order_item_verifications_merchant_access" on public."order_item_verifications" for all to authenticated using (exists (select 1 from public.order_items oi join public.orders o on
o.id=oi.order_id where oi.id=order_item_id and private.owns_shop(o.shop_id))) with check (exists (select 1 from public.order_items oi join public.orders o on o.id=oi.order_id where
oi.id=order_item_id and private.owns_shop(o.shop_id)));
create policy "return_requests_merchant_access" on public."return_requests" for all to authenticated using (private.owns_shop(shop_id)) with check (private.owns_shop(shop_id));
create policy "return_items_merchant_access" on public."return_items" for all to authenticated using (exists (select 1 from public.return_requests r where r.id=return_request_id and
private.owns_shop(r.shop_id))) with check (exists (select 1 from public.return_requests r where r.id=return_request_id and private.owns_shop(r.shop_id)));
create policy "return_evidence_merchant_access" on public."return_evidence" for all to authenticated using (exists (select 1 from public.return_requests r where r.id=return_request_id and
private.owns_shop(r.shop_id))) with check (exists (select 1 from public.return_requests r where r.id=return_request_id and private.owns_shop(r.shop_id)));
create policy "merchant_settlement_items_merchant_access" on public."merchant_settlement_items" for all to authenticated using (exists (select 1 from public.merchant_settlements s where
s.id=settlement_id and private.owns_shop(s.shop_id))) with check (exists (select 1 from public.merchant_settlements s where s.id=settlement_id and private.owns_shop(s.shop_id)));
create policy "shop_favourites_customer_access" on public."shop_favourites" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "wishlist_items_customer_access" on public."wishlist_items" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "carts_customer_access" on public."carts" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "orders_customer_access" on public."orders" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "payments_customer_access" on public."payments" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "return_requests_customer_access" on public."return_requests" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "customer_events_customer_access" on public."customer_events" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "recommendation_sets_customer_access" on public."recommendation_sets" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "customer_body_profiles_customer_access" on public."customer_body_profiles" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select
auth.uid()));
create policy "fit_recommendations_customer_access" on public."fit_recommendations" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "fit_feedback_customer_access" on public."fit_feedback" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select auth.uid()));
create policy "virtual_tryon_sessions_customer_access" on public."virtual_tryon_sessions" for all to authenticated using (customer_id = (select auth.uid())) with check (customer_id = (select
auth.uid()));
create policy "cart_items_customer_access" on public."cart_items" for all to authenticated using (exists (select 1 from public.carts c where c.id=cart_id and c.customer_id=(select auth.uid()))) with
check (exists (select 1 from public.carts c where c.id=cart_id and c.customer_id=(select auth.uid())));
create policy "order_items_customer_access" on public."order_items" for all to authenticated using (exists (select 1 from public.orders o where o.id=order_id and o.customer_id=(select auth.uid())))
with check (exists (select 1 from public.orders o where o.id=order_id and o.customer_id=(select auth.uid())));
create policy "order_status_history_customer_access" on public."order_status_history" for all to authenticated using (exists (select 1 from public.orders o where o.id=order_id and
o.customer_id=(select auth.uid()))) with check (exists (select 1 from public.orders o where o.id=order_id and o.customer_id=(select auth.uid())));
create policy "payment_events_customer_access" on public."payment_events" for all to authenticated using (exists (select 1 from public.payments p where p.id=payment_id and p.customer_id=(select
auth.uid()))) with check (exists (select 1 from public.payments p where p.id=payment_id and p.customer_id=(select auth.uid())));
create policy "refunds_customer_access" on public."refunds" for all to authenticated using (exists (select 1 from public.orders o where o.id=order_id and o.customer_id=(select auth.uid()))) with
check (exists (select 1 from public.orders o where o.id=order_id and o.customer_id=(select auth.uid())));
create policy "return_items_customer_access" on public."return_items" for all to authenticated using (exists (select 1 from public.return_requests r where r.id=return_request_id and
r.customer_id=(select auth.uid()))) with check (exists (select 1 from public.return_requests r where r.id=return_request_id and r.customer_id=(select auth.uid())));
create policy "return_evidence_customer_access" on public."return_evidence" for all to authenticated using (exists (select 1 from public.return_requests r where r.id=return_request_id and
r.customer_id=(select auth.uid()))) with check (exists (select 1 from public.return_requests r where r.id=return_request_id and r.customer_id=(select auth.uid())));
create policy "return_status_history_customer_access" on public."return_status_history" for all to authenticated using (exists (select 1 from public.return_requests r where r.id=return_request_id and
r.customer_id=(select auth.uid()))) with check (exists (select 1 from public.return_requests r where r.id=return_request_id and r.customer_id=(select auth.uid())));
create policy "recommendation_items_customer_access" on public."recommendation_items" for all to authenticated using (exists (select 1 from public.recommendation_sets rs where
rs.id=recommendation_set_id and rs.customer_id=(select auth.uid()))) with check (exists (select 1 from public.recommendation_sets rs where rs.id=recommendation_set_id and rs.customer_id=(select
auth.uid())));
create policy "body_measurements_customer_access" on public."body_measurements" for all to authenticated using (exists (select 1 from public.customer_body_profiles bp where bp.id=body_profile_id and
bp.customer_id=(select auth.uid()))) with check (exists (select 1 from public.customer_body_profiles bp where bp.id=body_profile_id and bp.customer_id=(select auth.uid())));
create policy "body_scan_sessions_customer_access" on public."body_scan_sessions" for all to authenticated using (customer_id=(select auth.uid())) with check (customer_id=(select auth.uid()));
create policy "virtual_tryon_outputs_customer_access" on public."virtual_tryon_outputs" for all to authenticated using (exists (select 1 from public.virtual_tryon_sessions s where
s.id=tryon_session_id and s.customer_id=(select auth.uid()))) with check (exists (select 1 from public.virtual_tryon_sessions s where s.id=tryon_session_id and s.customer_id=(select auth.uid())));
create policy "style_groups_member_read" on public."style_groups" for select to authenticated using (private.is_style_group_member(id));
create policy "style_groups_creator_write" on public."style_groups" for all to authenticated using (creator_customer_id=(select auth.uid())) with check (creator_customer_id=(select auth.uid()));
create policy "style_group_members_read" on public."style_group_members" for select to authenticated using (private.is_style_group_member(group_id));
create policy "style_group_members_write" on public."style_group_members" for all to authenticated using (private.is_style_group_member(group_id)) with check
(private.is_style_group_member(group_id));
create policy "group_preferences_owner" on public."style_group_member_preferences" for all to authenticated using (exists (select 1 from public.style_group_members m where m.id=group_member_id and
m.customer_id=(select auth.uid()))) with check (exists (select 1 from public.style_group_members m where m.id=group_member_id and m.customer_id=(select auth.uid())));
create policy "style_themes_group_access" on public."style_themes" for all to authenticated using (private.is_style_group_member(group_id)) with check (private.is_style_group_member(group_id));
create policy "style_polls_group_access" on public."style_polls" for all to authenticated using (private.is_style_group_member(group_id)) with check (private.is_style_group_member(group_id));
```

```
create policy "style_moodboards_group_access" on public."style_moodboards" for all to authenticated using (private.is_style_group_member(group_id)) with check
(private.is_style_group_member(group_id));
create policy "style_recommendation_sets_group_access" on public."style_recommendation_sets" for all to authenticated using (private.is_style_group_member(group_id)) with check
(private.is_style_group_member(group_id));
create policy "captain_profile_self" on public."captain_profiles" for select to authenticated using (user_id=(select auth.uid()));
create policy "captain_profile_self_update" on public."captain_profiles" for update to authenticated using (user_id=(select auth.uid())) with check (user_id=(select auth.uid()));
create policy "captain_documents_self" on public."captain_documents" for all to authenticated using (captain_id=(select auth.uid())) with check (captain_id=(select auth.uid()));
create policy "delivery_tasks_captain" on public."delivery_tasks" for select to authenticated using (assigned_captain_id=(select auth.uid()));
create policy "delivery_assignments_captain" on public."delivery_assignments" for select to authenticated using (captain_id=(select auth.uid()));
create policy "captain_location_self" on public."captain_current_locations" for all to authenticated using (captain_id=(select auth.uid())) with check (captain_id=(select auth.uid()));
create policy "captain_location_history_self_insert" on public."captain_location_history" for insert to authenticated with check (captain_id=(select auth.uid()));
create policy "delivery_events_captain" on public."delivery_events" for select to authenticated using (exists (select 1 from public.delivery_tasks d where d.id=delivery_task_id and
d.assigned_captain_id=(select auth.uid())));
create policy "captain_earnings_self" on public."captain_earnings" for select to authenticated using (captain_id=(select auth.uid()));
create policy "captain_payouts_self" on public."captain_payouts" for select to authenticated using (captain_id=(select auth.uid()));
create policy "support_ticket_requester" on public."support_tickets" for select to authenticated using (raised_by_user_id=(select auth.uid()) or assigned_to=(select auth.uid()));
create policy "support_ticket_create" on public."support_tickets" for insert to authenticated with check (raised_by_user_id=(select auth.uid()));
create policy "support_message_access" on public."support_messages" for select to authenticated using (exists (select 1 from public.support_tickets t where t.id=ticket_id and
(t.raised_by_user_id=(select auth.uid()) or t.assigned_to=(select auth.uid()))));
create policy "support_message_create" on public."support_messages" for insert to authenticated with check (sender_id=(select auth.uid()) and exists (select 1 from public.support_tickets t where
t.id=ticket_id and (t.raised_by_user_id=(select auth.uid()) or t.assigned_to=(select auth.uid()))));
create policy "profiles_admin_read" on public."profiles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "profiles_admin_write" on public."profiles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "user_devices_admin_read" on public."user_devices" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "user_devices_admin_write" on public."user_devices" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "addresses_admin_read" on public."addresses" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "addresses_admin_write" on public."addresses" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "customer_profiles_admin_read" on public."customer_profiles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "customer_profiles_admin_write" on public."customer_profiles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "customer_preferences_admin_read" on public."customer_preferences" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "customer_preferences_admin_write" on public."customer_preferences" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "merchant_profiles_admin_read" on public."merchant_profiles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "merchant_profiles_admin_write" on public."merchant_profiles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "captain_profiles_admin_read" on public."captain_profiles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "captain_profiles_admin_write" on public."captain_profiles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "admin_profiles_admin_read" on public."admin_profiles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "admin_profiles_admin_write" on public."admin_profiles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "roles_admin_read" on public."roles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "roles_admin_write" on public."roles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select private.has_permission('platform.write')));
create policy "permissions_admin_read" on public."permissions" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "permissions_admin_write" on public."permissions" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "user_roles_admin_read" on public."user_roles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "user_roles_admin_write" on public."user_roles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "role_permissions_admin_read" on public."role_permissions" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "role_permissions_admin_write" on public."role_permissions" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "shops_admin_read" on public."shops" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "shops_admin_write" on public."shops" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select private.has_permission('platform.write')));
create policy "shop_hours_admin_read" on public."shop_hours" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "shop_hours_admin_write" on public."shop_hours" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "shop_documents_admin_read" on public."shop_documents" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "shop_documents_admin_write" on public."shop_documents" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "shop_bank_accounts_admin_read" on public."shop_bank_accounts" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "shop_bank_accounts_admin_write" on public."shop_bank_accounts" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "categories_admin_read" on public."categories" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "categories_admin_write" on public."categories" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "products_admin_read" on public."products" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "products_admin_write" on public."products" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "product_images_admin_read" on public."product_images" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "product_images_admin_write" on public."product_images" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "product_variants_admin_read" on public."product_variants" for select to authenticated using ((select private.has_permission('platform.read')));
```

```
create policy "product_variants_admin_write" on public."product_variants" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "variant_barcodes_admin_read" on public."variant_barcodes" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "variant_barcodes_admin_write" on public."variant_barcodes" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "inventory_balances_admin_read" on public."inventory_balances" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "inventory_balances_admin_write" on public."inventory_balances" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "inventory_movements_admin_read" on public."inventory_movements" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "inventory_movements_admin_write" on public."inventory_movements" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "inventory_reservations_admin_read" on public."inventory_reservations" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "inventory_reservations_admin_write" on public."inventory_reservations" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "product_photo_matches_admin_read" on public."product_photo_matches" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "product_photo_matches_admin_write" on public."product_photo_matches" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "shop_favourites_admin_read" on public."shop_favourites" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "shop_favourites_admin_write" on public."shop_favourites" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "wishlist_items_admin_read" on public."wishlist_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "wishlist_items_admin_write" on public."wishlist_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "shop_collections_admin_read" on public."shop_collections" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "shop_collections_admin_write" on public."shop_collections" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "shop_collection_items_admin_read" on public."shop_collection_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "shop_collection_items_admin_write" on public."shop_collection_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "offers_admin_read" on public."offers" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "offers_admin_write" on public."offers" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "offer_products_admin_read" on public."offer_products" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "offer_products_admin_write" on public."offer_products" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "offer_redemptions_admin_read" on public."offer_redemptions" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "offer_redemptions_admin_write" on public."offer_redemptions" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "offline_sales_admin_read" on public."offline_sales" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "offline_sales_admin_write" on public."offline_sales" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "offline_sale_items_admin_read" on public."offline_sale_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "offline_sale_items_admin_write" on public."offline_sale_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "carts_admin_read" on public."carts" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "carts_admin_write" on public."carts" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select private.has_permission('platform.write')));
create policy "cart_items_admin_read" on public."cart_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "cart_items_admin_write" on public."cart_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "orders_admin_read" on public."orders" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "orders_admin_write" on public."orders" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "order_items_admin_read" on public."order_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "order_items_admin_write" on public."order_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "order_status_history_admin_read" on public."order_status_history" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "order_status_history_admin_write" on public."order_status_history" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "merchant_order_alerts_admin_read" on public."merchant_order_alerts" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "merchant_order_alerts_admin_write" on public."merchant_order_alerts" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "merchant_order_issues_admin_read" on public."merchant_order_issues" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "merchant_order_issues_admin_write" on public."merchant_order_issues" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "order_item_verifications_admin_read" on public."order_item_verifications" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "order_item_verifications_admin_write" on public."order_item_verifications" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "payments_admin_read" on public."payments" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "payments_admin_write" on public."payments" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "payment_events_admin_read" on public."payment_events" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "payment_events_admin_write" on public."payment_events" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "return_requests_admin_read" on public."return_requests" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "return_requests_admin_write" on public."return_requests" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "return_items_admin_read" on public."return_items" for select to authenticated using ((select private.has_permission('platform.read')));
```

```
create policy "return_items_admin_write" on public."return_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "return_evidence_admin_read" on public."return_evidence" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "return_evidence_admin_write" on public."return_evidence" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "return_status_history_admin_read" on public."return_status_history" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "return_status_history_admin_write" on public."return_status_history" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "refunds_admin_read" on public."refunds" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "refunds_admin_write" on public."refunds" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "merchant_settlements_admin_read" on public."merchant_settlements" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "merchant_settlements_admin_write" on public."merchant_settlements" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "merchant_settlement_items_admin_read" on public."merchant_settlement_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "merchant_settlement_items_admin_write" on public."merchant_settlement_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "captain_documents_admin_read" on public."captain_documents" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "captain_documents_admin_write" on public."captain_documents" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "delivery_tasks_admin_read" on public."delivery_tasks" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "delivery_tasks_admin_write" on public."delivery_tasks" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "delivery_assignments_admin_read" on public."delivery_assignments" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "delivery_assignments_admin_write" on public."delivery_assignments" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "captain_current_locations_admin_read" on public."captain_current_locations" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "captain_current_locations_admin_write" on public."captain_current_locations" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "captain_location_history_admin_read" on public."captain_location_history" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "captain_location_history_admin_write" on public."captain_location_history" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "delivery_events_admin_read" on public."delivery_events" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "delivery_events_admin_write" on public."delivery_events" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "cod_collections_admin_read" on public."cod_collections" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "cod_collections_admin_write" on public."cod_collections" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "captain_earnings_admin_read" on public."captain_earnings" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "captain_earnings_admin_write" on public."captain_earnings" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "captain_payouts_admin_read" on public."captain_payouts" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "captain_payouts_admin_write" on public."captain_payouts" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "captain_payout_items_admin_read" on public."captain_payout_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "captain_payout_items_admin_write" on public."captain_payout_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_groups_admin_read" on public."style_groups" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_groups_admin_write" on public."style_groups" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_group_members_admin_read" on public."style_group_members" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_group_members_admin_write" on public."style_group_members" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_group_member_preferences_admin_read" on public."style_group_member_preferences" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_group_member_preferences_admin_write" on public."style_group_member_preferences" for all to authenticated using ((select private.has_permission('platform.write'))) with check
((select private.has_permission('platform.write')));
create policy "style_themes_admin_read" on public."style_themes" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_themes_admin_write" on public."style_themes" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_polls_admin_read" on public."style_polls" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_polls_admin_write" on public."style_polls" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_poll_options_admin_read" on public."style_poll_options" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_poll_options_admin_write" on public."style_poll_options" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_poll_votes_admin_read" on public."style_poll_votes" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_poll_votes_admin_write" on public."style_poll_votes" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_moodboards_admin_read" on public."style_moodboards" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_moodboards_admin_write" on public."style_moodboards" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_moodboard_items_admin_read" on public."style_moodboard_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_moodboard_items_admin_write" on public."style_moodboard_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "notifications_admin_read" on public."notifications" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "notifications_admin_write" on public."notifications" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "notification_deliveries_admin_read" on public."notification_deliveries" for select to authenticated using ((select private.has_permission('platform.read')));
```

```
create policy "notification_deliveries_admin_write" on public."notification_deliveries" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "notification_preferences_admin_read" on public."notification_preferences" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "notification_preferences_admin_write" on public."notification_preferences" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "reviews_admin_read" on public."reviews" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "reviews_admin_write" on public."reviews" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "customer_events_admin_read" on public."customer_events" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "customer_events_admin_write" on public."customer_events" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "recommendation_sets_admin_read" on public."recommendation_sets" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "recommendation_sets_admin_write" on public."recommendation_sets" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "recommendation_items_admin_read" on public."recommendation_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "recommendation_items_admin_write" on public."recommendation_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "product_embeddings_admin_read" on public."product_embeddings" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "product_embeddings_admin_write" on public."product_embeddings" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "customer_embeddings_admin_read" on public."customer_embeddings" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "customer_embeddings_admin_write" on public."customer_embeddings" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "size_charts_admin_read" on public."size_charts" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "size_charts_admin_write" on public."size_charts" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "size_chart_measurements_admin_read" on public."size_chart_measurements" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "size_chart_measurements_admin_write" on public."size_chart_measurements" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "customer_body_profiles_admin_read" on public."customer_body_profiles" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "customer_body_profiles_admin_write" on public."customer_body_profiles" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "body_measurements_admin_read" on public."body_measurements" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "body_measurements_admin_write" on public."body_measurements" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "body_scan_sessions_admin_read" on public."body_scan_sessions" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "body_scan_sessions_admin_write" on public."body_scan_sessions" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "fit_recommendations_admin_read" on public."fit_recommendations" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "fit_recommendations_admin_write" on public."fit_recommendations" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "fit_feedback_admin_read" on public."fit_feedback" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "fit_feedback_admin_write" on public."fit_feedback" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "virtual_tryon_sessions_admin_read" on public."virtual_tryon_sessions" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "virtual_tryon_sessions_admin_write" on public."virtual_tryon_sessions" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "virtual_tryon_outputs_admin_read" on public."virtual_tryon_outputs" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "virtual_tryon_outputs_admin_write" on public."virtual_tryon_outputs" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_recommendation_sets_admin_read" on public."style_recommendation_sets" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_recommendation_sets_admin_write" on public."style_recommendation_sets" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "style_recommendation_items_admin_read" on public."style_recommendation_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "style_recommendation_items_admin_write" on public."style_recommendation_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "support_tickets_admin_read" on public."support_tickets" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "support_tickets_admin_write" on public."support_tickets" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "support_messages_admin_read" on public."support_messages" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "support_messages_admin_write" on public."support_messages" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "campaigns_admin_read" on public."campaigns" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "campaigns_admin_write" on public."campaigns" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "banners_admin_read" on public."banners" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "banners_admin_write" on public."banners" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "merchant_leads_admin_read" on public."merchant_leads" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "merchant_leads_admin_write" on public."merchant_leads" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "field_visits_admin_read" on public."field_visits" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "field_visits_admin_write" on public."field_visits" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "field_visit_checklist_items_admin_read" on public."field_visit_checklist_items" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "field_visit_checklist_items_admin_write" on public."field_visit_checklist_items" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "product_moderation_cases_admin_read" on public."product_moderation_cases" for select to authenticated using ((select private.has_permission('platform.read')));
```

```
create policy "product_moderation_cases_admin_write" on public."product_moderation_cases" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "approval_requests_admin_read" on public."approval_requests" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "approval_requests_admin_write" on public."approval_requests" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "fraud_cases_admin_read" on public."fraud_cases" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "fraud_cases_admin_write" on public."fraud_cases" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "audit_logs_admin_read" on public."audit_logs" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "audit_logs_admin_write" on public."audit_logs" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "system_settings_admin_read" on public."system_settings" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "system_settings_admin_write" on public."system_settings" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
create policy "service_zones_admin_read" on public."service_zones" for select to authenticated using ((select private.has_permission('platform.read')));
create policy "service_zones_admin_write" on public."service_zones" for all to authenticated using ((select private.has_permission('platform.write'))) with check ((select
private.has_permission('platform.write')));
```

0012_indexes_storage_realtime.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


create index if not exists "profiles_id_idx" on public."profiles" ("id");
create index if not exists "profiles_status_idx" on public."profiles" ("status");
create index if not exists "profiles_created_at_idx" on public."profiles" ("created_at");
create index if not exists "profiles_updated_at_idx" on public."profiles" ("updated_at");
create index if not exists "user_devices_user_id_idx" on public."user_devices" ("user_id");
create index if not exists "user_devices_created_at_idx" on public."user_devices" ("created_at");
create index if not exists "user_devices_updated_at_idx" on public."user_devices" ("updated_at");
create index if not exists "addresses_user_id_idx" on public."addresses" ("user_id");
create index if not exists "addresses_city_idx" on public."addresses" ("city");
create index if not exists "addresses_created_at_idx" on public."addresses" ("created_at");
create index if not exists "addresses_updated_at_idx" on public."addresses" ("updated_at");
create index if not exists "customer_profiles_user_id_idx" on public."customer_profiles" ("user_id");
create index if not exists "customer_profiles_default_address_id_idx" on public."customer_profiles" ("default_address_id");
create index if not exists "customer_profiles_created_at_idx" on public."customer_profiles" ("created_at");
create index if not exists "customer_profiles_updated_at_idx" on public."customer_profiles" ("updated_at");
create index if not exists "customer_preferences_customer_id_idx" on public."customer_preferences" ("customer_id");
create index if not exists "customer_preferences_updated_at_idx" on public."customer_preferences" ("updated_at");
create index if not exists "merchant_profiles_user_id_idx" on public."merchant_profiles" ("user_id");
create index if not exists "merchant_profiles_approved_by_idx" on public."merchant_profiles" ("approved_by");
create index if not exists "merchant_profiles_created_at_idx" on public."merchant_profiles" ("created_at");
create index if not exists "merchant_profiles_updated_at_idx" on public."merchant_profiles" ("updated_at");
create index if not exists "captain_profiles_user_id_idx" on public."captain_profiles" ("user_id");
create index if not exists "captain_profiles_created_at_idx" on public."captain_profiles" ("created_at");
create index if not exists "captain_profiles_updated_at_idx" on public."captain_profiles" ("updated_at");
create index if not exists "admin_profiles_user_id_idx" on public."admin_profiles" ("user_id");
create index if not exists "admin_profiles_manager_id_idx" on public."admin_profiles" ("manager_id");
create index if not exists "admin_profiles_created_at_idx" on public."admin_profiles" ("created_at");
create index if not exists "admin_profiles_updated_at_idx" on public."admin_profiles" ("updated_at");
create index if not exists "roles_created_at_idx" on public."roles" ("created_at");
create index if not exists "permissions_created_at_idx" on public."permissions" ("created_at");
create index if not exists "user_roles_user_id_idx" on public."user_roles" ("user_id");
create index if not exists "user_roles_role_id_idx" on public."user_roles" ("role_id");
create index if not exists "user_roles_assigned_by_idx" on public."user_roles" ("assigned_by");
create index if not exists "role_permissions_role_id_idx" on public."role_permissions" ("role_id");
create index if not exists "role_permissions_permission_id_idx" on public."role_permissions" ("permission_id");
create index if not exists "role_permissions_created_at_idx" on public."role_permissions" ("created_at");
create index if not exists "shops_merchant_id_idx" on public."shops" ("merchant_id");
create index if not exists "shops_city_idx" on public."shops" ("city");
create index if not exists "shops_created_at_idx" on public."shops" ("created_at");
create index if not exists "shops_updated_at_idx" on public."shops" ("updated_at");
create index if not exists "shop_hours_shop_id_idx" on public."shop_hours" ("shop_id");
create index if not exists "shop_hours_created_at_idx" on public."shop_hours" ("created_at");
create index if not exists "shop_hours_updated_at_idx" on public."shop_hours" ("updated_at");
create index if not exists "shop_documents_shop_id_idx" on public."shop_documents" ("shop_id");
create index if not exists "shop_documents_uploaded_by_idx" on public."shop_documents" ("uploaded_by");
create index if not exists "shop_documents_verified_by_idx" on public."shop_documents" ("verified_by");
create index if not exists "shop_documents_created_at_idx" on public."shop_documents" ("created_at");
create index if not exists "shop_bank_accounts_shop_id_idx" on public."shop_bank_accounts" ("shop_id");
create index if not exists "shop_bank_accounts_created_at_idx" on public."shop_bank_accounts" ("created_at");
create index if not exists "shop_bank_accounts_updated_at_idx" on public."shop_bank_accounts" ("updated_at");
create index if not exists "categories_parent_id_idx" on public."categories" ("parent_id");
create index if not exists "categories_created_at_idx" on public."categories" ("created_at");
create index if not exists "categories_updated_at_idx" on public."categories" ("updated_at");
create index if not exists "products_shop_id_idx" on public."products" ("shop_id");
create index if not exists "products_category_id_idx" on public."products" ("category_id");
create index if not exists "products_created_at_idx" on public."products" ("created_at");
create index if not exists "products_updated_at_idx" on public."products" ("updated_at");
create index if not exists "product_images_product_id_idx" on public."product_images" ("product_id");
create index if not exists "product_images_variant_id_idx" on public."product_images" ("variant_id");
create index if not exists "product_images_created_at_idx" on public."product_images" ("created_at");
create index if not exists "product_variants_product_id_idx" on public."product_variants" ("product_id");
create index if not exists "product_variants_shop_id_idx" on public."product_variants" ("shop_id");
create index if not exists "product_variants_created_at_idx" on public."product_variants" ("created_at");
create index if not exists "product_variants_updated_at_idx" on public."product_variants" ("updated_at");
create index if not exists "variant_barcodes_variant_id_idx" on public."variant_barcodes" ("variant_id");
create index if not exists "variant_barcodes_created_by_idx" on public."variant_barcodes" ("created_by");
create index if not exists "variant_barcodes_created_at_idx" on public."variant_barcodes" ("created_at");
create index if not exists "inventory_balances_shop_id_idx" on public."inventory_balances" ("shop_id");
create index if not exists "inventory_balances_variant_id_idx" on public."inventory_balances" ("variant_id");
```

```
create index if not exists "inventory_balances_updated_at_idx" on public."inventory_balances" ("updated_at");
create index if not exists "inventory_movements_shop_id_idx" on public."inventory_movements" ("shop_id");
create index if not exists "inventory_movements_variant_id_idx" on public."inventory_movements" ("variant_id");
create index if not exists "inventory_movements_performed_by_idx" on public."inventory_movements" ("performed_by");
create index if not exists "inventory_movements_created_at_idx" on public."inventory_movements" ("created_at");
create index if not exists "inventory_reservations_shop_id_idx" on public."inventory_reservations" ("shop_id");
create index if not exists "inventory_reservations_variant_id_idx" on public."inventory_reservations" ("variant_id");
create index if not exists "inventory_reservations_cart_id_idx" on public."inventory_reservations" ("cart_id");
create index if not exists "inventory_reservations_order_id_idx" on public."inventory_reservations" ("order_id");
create index if not exists "inventory_reservations_status_idx" on public."inventory_reservations" ("status");
create index if not exists "inventory_reservations_created_at_idx" on public."inventory_reservations" ("created_at");
create index if not exists "product_photo_matches_shop_id_idx" on public."product_photo_matches" ("shop_id");
create index if not exists "product_photo_matches_uploaded_by_idx" on public."product_photo_matches" ("uploaded_by");
create index if not exists "product_photo_matches_matched_variant_id_idx" on public."product_photo_matches" ("matched_variant_id");
create index if not exists "product_photo_matches_created_at_idx" on public."product_photo_matches" ("created_at");
create index if not exists "shop_favourites_customer_id_idx" on public."shop_favourites" ("customer_id");
create index if not exists "shop_favourites_shop_id_idx" on public."shop_favourites" ("shop_id");
create index if not exists "wishlist_items_customer_id_idx" on public."wishlist_items" ("customer_id");
create index if not exists "wishlist_items_product_id_idx" on public."wishlist_items" ("product_id");
create index if not exists "wishlist_items_variant_id_idx" on public."wishlist_items" ("variant_id");
create index if not exists "wishlist_items_created_at_idx" on public."wishlist_items" ("created_at");
create index if not exists "shop_collections_shop_id_idx" on public."shop_collections" ("shop_id");
create index if not exists "shop_collections_created_at_idx" on public."shop_collections" ("created_at");
create index if not exists "shop_collections_updated_at_idx" on public."shop_collections" ("updated_at");
create index if not exists "shop_collection_items_collection_id_idx" on public."shop_collection_items" ("collection_id");
create index if not exists "shop_collection_items_product_id_idx" on public."shop_collection_items" ("product_id");
create index if not exists "shop_collection_items_variant_id_idx" on public."shop_collection_items" ("variant_id");
create index if not exists "shop_collection_items_created_at_idx" on public."shop_collection_items" ("created_at");
create index if not exists "offers_shop_id_idx" on public."offers" ("shop_id");
create index if not exists "offers_status_idx" on public."offers" ("status");
create index if not exists "offers_created_at_idx" on public."offers" ("created_at");
create index if not exists "offers_updated_at_idx" on public."offers" ("updated_at");
create index if not exists "offer_products_offer_id_idx" on public."offer_products" ("offer_id");
create index if not exists "offer_products_product_id_idx" on public."offer_products" ("product_id");
create index if not exists "offer_products_variant_id_idx" on public."offer_products" ("variant_id");
create index if not exists "offer_products_created_at_idx" on public."offer_products" ("created_at");
create index if not exists "offer_redemptions_offer_id_idx" on public."offer_redemptions" ("offer_id");
create index if not exists "offer_redemptions_customer_id_idx" on public."offer_redemptions" ("customer_id");
create index if not exists "offer_redemptions_order_id_idx" on public."offer_redemptions" ("order_id");
create index if not exists "offer_redemptions_created_at_idx" on public."offer_redemptions" ("created_at");
create index if not exists "offline_sales_shop_id_idx" on public."offline_sales" ("shop_id");
create index if not exists "offline_sales_merchant_id_idx" on public."offline_sales" ("merchant_id");
create index if not exists "offline_sales_status_idx" on public."offline_sales" ("status");
create index if not exists "offline_sales_recorded_by_idx" on public."offline_sales" ("recorded_by");
create index if not exists "offline_sales_created_at_idx" on public."offline_sales" ("created_at");
create index if not exists "offline_sale_items_offline_sale_id_idx" on public."offline_sale_items" ("offline_sale_id");
create index if not exists "offline_sale_items_variant_id_idx" on public."offline_sale_items" ("variant_id");
create index if not exists "offline_sale_items_created_at_idx" on public."offline_sale_items" ("created_at");
create index if not exists "carts_customer_id_idx" on public."carts" ("customer_id");
create index if not exists "carts_shop_id_idx" on public."carts" ("shop_id");
create index if not exists "carts_status_idx" on public."carts" ("status");
create index if not exists "carts_created_at_idx" on public."carts" ("created_at");
create index if not exists "carts_updated_at_idx" on public."carts" ("updated_at");
create index if not exists "cart_items_cart_id_idx" on public."cart_items" ("cart_id");
create index if not exists "cart_items_variant_id_idx" on public."cart_items" ("variant_id");
create index if not exists "cart_items_updated_at_idx" on public."cart_items" ("updated_at");
create index if not exists "orders_customer_id_idx" on public."orders" ("customer_id");
create index if not exists "orders_shop_id_idx" on public."orders" ("shop_id");
create index if not exists "orders_cart_id_idx" on public."orders" ("cart_id");
create index if not exists "orders_style_group_id_idx" on public."orders" ("style_group_id");
create index if not exists "orders_delivery_address_id_idx" on public."orders" ("delivery_address_id");
create index if not exists "orders_status_idx" on public."orders" ("status");
create index if not exists "orders_created_at_idx" on public."orders" ("created_at");
create index if not exists "orders_updated_at_idx" on public."orders" ("updated_at");
create index if not exists "order_items_order_id_idx" on public."order_items" ("order_id");
create index if not exists "order_items_product_id_idx" on public."order_items" ("product_id");
create index if not exists "order_items_variant_id_idx" on public."order_items" ("variant_id");
create index if not exists "order_items_created_at_idx" on public."order_items" ("created_at");
create index if not exists "order_status_history_order_id_idx" on public."order_status_history" ("order_id");
create index if not exists "order_status_history_changed_by_user_id_idx" on public."order_status_history" ("changed_by_user_id");
create index if not exists "order_status_history_created_at_idx" on public."order_status_history" ("created_at");
create index if not exists "merchant_order_alerts_order_id_idx" on public."merchant_order_alerts" ("order_id");
create index if not exists "merchant_order_alerts_shop_id_idx" on public."merchant_order_alerts" ("shop_id");
create index if not exists "merchant_order_alerts_device_id_idx" on public."merchant_order_alerts" ("device_id");
create index if not exists "merchant_order_alerts_acknowledged_by_idx" on public."merchant_order_alerts" ("acknowledged_by");
create index if not exists "merchant_order_alerts_created_at_idx" on public."merchant_order_alerts" ("created_at");
create index if not exists "merchant_order_issues_order_id_idx" on public."merchant_order_issues" ("order_id");
create index if not exists "merchant_order_issues_order_item_id_idx" on public."merchant_order_issues" ("order_item_id");
```

```
create index if not exists "merchant_order_issues_reported_by_idx" on public."merchant_order_issues" ("reported_by");
create index if not exists "merchant_order_issues_resolved_by_idx" on public."merchant_order_issues" ("resolved_by");
create index if not exists "merchant_order_issues_created_at_idx" on public."merchant_order_issues" ("created_at");
create index if not exists "order_item_verifications_order_item_id_idx" on public."order_item_verifications" ("order_item_id");
create index if not exists "order_item_verifications_verified_variant_id_idx" on public."order_item_verifications" ("verified_variant_id");
create index if not exists "order_item_verifications_verified_by_idx" on public."order_item_verifications" ("verified_by");
create index if not exists "payments_order_id_idx" on public."payments" ("order_id");
create index if not exists "payments_customer_id_idx" on public."payments" ("customer_id");
create index if not exists "payments_status_idx" on public."payments" ("status");
create index if not exists "payments_created_at_idx" on public."payments" ("created_at");
create index if not exists "payments_updated_at_idx" on public."payments" ("updated_at");
create index if not exists "payment_events_payment_id_idx" on public."payment_events" ("payment_id");
create index if not exists "return_requests_order_id_idx" on public."return_requests" ("order_id");
create index if not exists "return_requests_customer_id_idx" on public."return_requests" ("customer_id");
create index if not exists "return_requests_shop_id_idx" on public."return_requests" ("shop_id");
create index if not exists "return_requests_status_idx" on public."return_requests" ("status");
create index if not exists "return_requests_created_at_idx" on public."return_requests" ("created_at");
create index if not exists "return_requests_updated_at_idx" on public."return_requests" ("updated_at");
create index if not exists "return_items_return_request_id_idx" on public."return_items" ("return_request_id");
create index if not exists "return_items_order_item_id_idx" on public."return_items" ("order_item_id");
create index if not exists "return_items_inspected_by_idx" on public."return_items" ("inspected_by");
create index if not exists "return_evidence_return_request_id_idx" on public."return_evidence" ("return_request_id");
create index if not exists "return_evidence_uploaded_by_idx" on public."return_evidence" ("uploaded_by");
create index if not exists "return_evidence_created_at_idx" on public."return_evidence" ("created_at");
create index if not exists "return_status_history_return_request_id_idx" on public."return_status_history" ("return_request_id");
create index if not exists "return_status_history_changed_by_idx" on public."return_status_history" ("changed_by");
create index if not exists "return_status_history_created_at_idx" on public."return_status_history" ("created_at");
create index if not exists "refunds_order_id_idx" on public."refunds" ("order_id");
create index if not exists "refunds_payment_id_idx" on public."refunds" ("payment_id");
create index if not exists "refunds_return_request_id_idx" on public."refunds" ("return_request_id");
create index if not exists "refunds_status_idx" on public."refunds" ("status");
create index if not exists "refunds_initiated_by_idx" on public."refunds" ("initiated_by");
create index if not exists "refunds_approved_by_idx" on public."refunds" ("approved_by");
create index if not exists "refunds_created_at_idx" on public."refunds" ("created_at");
create index if not exists "refunds_updated_at_idx" on public."refunds" ("updated_at");
create index if not exists "merchant_settlements_shop_id_idx" on public."merchant_settlements" ("shop_id");
create index if not exists "merchant_settlements_bank_account_id_idx" on public."merchant_settlements" ("bank_account_id");
create index if not exists "merchant_settlements_status_idx" on public."merchant_settlements" ("status");
create index if not exists "merchant_settlements_created_at_idx" on public."merchant_settlements" ("created_at");
create index if not exists "merchant_settlements_updated_at_idx" on public."merchant_settlements" ("updated_at");
create index if not exists "merchant_settlement_items_settlement_id_idx" on public."merchant_settlement_items" ("settlement_id");
create index if not exists "merchant_settlement_items_order_id_idx" on public."merchant_settlement_items" ("order_id");
create index if not exists "merchant_settlement_items_refund_id_idx" on public."merchant_settlement_items" ("refund_id");
create index if not exists "merchant_settlement_items_created_at_idx" on public."merchant_settlement_items" ("created_at");
create index if not exists "captain_documents_captain_id_idx" on public."captain_documents" ("captain_id");
create index if not exists "captain_documents_verified_by_idx" on public."captain_documents" ("verified_by");
create index if not exists "captain_documents_created_at_idx" on public."captain_documents" ("created_at");
create index if not exists "delivery_tasks_order_id_idx" on public."delivery_tasks" ("order_id");
create index if not exists "delivery_tasks_return_request_id_idx" on public."delivery_tasks" ("return_request_id");
create index if not exists "delivery_tasks_pickup_shop_id_idx" on public."delivery_tasks" ("pickup_shop_id");
create index if not exists "delivery_tasks_status_idx" on public."delivery_tasks" ("status");
create index if not exists "delivery_tasks_assigned_captain_id_idx" on public."delivery_tasks" ("assigned_captain_id");
create index if not exists "delivery_tasks_created_at_idx" on public."delivery_tasks" ("created_at");
create index if not exists "delivery_tasks_updated_at_idx" on public."delivery_tasks" ("updated_at");
create index if not exists "delivery_assignments_delivery_task_id_idx" on public."delivery_assignments" ("delivery_task_id");
create index if not exists "delivery_assignments_captain_id_idx" on public."delivery_assignments" ("captain_id");
create index if not exists "delivery_assignments_assigned_by_user_id_idx" on public."delivery_assignments" ("assigned_by_user_id");
create index if not exists "delivery_assignments_created_at_idx" on public."delivery_assignments" ("created_at");
create index if not exists "captain_current_locations_captain_id_idx" on public."captain_current_locations" ("captain_id");
create index if not exists "captain_current_locations_active_delivery_task_id_idx" on public."captain_current_locations" ("active_delivery_task_id");
create index if not exists "captain_current_locations_updated_at_idx" on public."captain_current_locations" ("updated_at");
create index if not exists "captain_location_history_captain_id_idx" on public."captain_location_history" ("captain_id");
create index if not exists "captain_location_history_delivery_task_id_idx" on public."captain_location_history" ("delivery_task_id");
create index if not exists "delivery_events_delivery_task_id_idx" on public."delivery_events" ("delivery_task_id");
create index if not exists "delivery_events_actor_user_id_idx" on public."delivery_events" ("actor_user_id");
create index if not exists "delivery_events_created_at_idx" on public."delivery_events" ("created_at");
create index if not exists "cod_collections_order_id_idx" on public."cod_collections" ("order_id");
create index if not exists "cod_collections_delivery_task_id_idx" on public."cod_collections" ("delivery_task_id");
create index if not exists "cod_collections_captain_id_idx" on public."cod_collections" ("captain_id");
create index if not exists "cod_collections_status_idx" on public."cod_collections" ("status");
create index if not exists "cod_collections_reconciled_by_idx" on public."cod_collections" ("reconciled_by");
create index if not exists "cod_collections_created_at_idx" on public."cod_collections" ("created_at");
create index if not exists "captain_earnings_captain_id_idx" on public."captain_earnings" ("captain_id");
create index if not exists "captain_earnings_delivery_task_id_idx" on public."captain_earnings" ("delivery_task_id");
create index if not exists "captain_earnings_status_idx" on public."captain_earnings" ("status");
create index if not exists "captain_earnings_created_at_idx" on public."captain_earnings" ("created_at");
create index if not exists "captain_payouts_captain_id_idx" on public."captain_payouts" ("captain_id");
create index if not exists "captain_payouts_status_idx" on public."captain_payouts" ("status");
```

```
create index if not exists "captain_payouts_created_at_idx" on public."captain_payouts" ("created_at");
create index if not exists "captain_payouts_updated_at_idx" on public."captain_payouts" ("updated_at");
create index if not exists "captain_payout_items_payout_id_idx" on public."captain_payout_items" ("payout_id");
create index if not exists "captain_payout_items_captain_earning_id_idx" on public."captain_payout_items" ("captain_earning_id");
create index if not exists "captain_payout_items_created_at_idx" on public."captain_payout_items" ("created_at");
create index if not exists "style_groups_creator_customer_id_idx" on public."style_groups" ("creator_customer_id");
create index if not exists "style_groups_status_idx" on public."style_groups" ("status");
create index if not exists "style_groups_created_at_idx" on public."style_groups" ("created_at");
create index if not exists "style_groups_updated_at_idx" on public."style_groups" ("updated_at");
create index if not exists "style_group_members_group_id_idx" on public."style_group_members" ("group_id");
create index if not exists "style_group_members_customer_id_idx" on public."style_group_members" ("customer_id");
create index if not exists "style_group_members_status_idx" on public."style_group_members" ("status");
create index if not exists "style_group_members_created_at_idx" on public."style_group_members" ("created_at");
create index if not exists "style_group_member_preferences_group_member_id_idx" on public."style_group_member_preferences" ("group_member_id");
create index if not exists "style_group_member_preferences_private_body_profile_id_idx" on public."style_group_member_preferences" ("private_body_profile_id");
create index if not exists "style_group_member_preferences_updated_at_idx" on public."style_group_member_preferences" ("updated_at");
create index if not exists "style_themes_group_id_idx" on public."style_themes" ("group_id");
create index if not exists "style_themes_created_by_idx" on public."style_themes" ("created_by");
create index if not exists "style_themes_created_at_idx" on public."style_themes" ("created_at");
create index if not exists "style_themes_updated_at_idx" on public."style_themes" ("updated_at");
create index if not exists "style_polls_group_id_idx" on public."style_polls" ("group_id");
create index if not exists "style_polls_status_idx" on public."style_polls" ("status");
create index if not exists "style_polls_created_by_idx" on public."style_polls" ("created_by");
create index if not exists "style_polls_created_at_idx" on public."style_polls" ("created_at");
create index if not exists "style_poll_options_poll_id_idx" on public."style_poll_options" ("poll_id");
create index if not exists "style_poll_options_product_id_idx" on public."style_poll_options" ("product_id");
create index if not exists "style_poll_options_theme_id_idx" on public."style_poll_options" ("theme_id");
create index if not exists "style_poll_options_created_at_idx" on public."style_poll_options" ("created_at");
create index if not exists "style_poll_votes_poll_id_idx" on public."style_poll_votes" ("poll_id");
create index if not exists "style_poll_votes_option_id_idx" on public."style_poll_votes" ("option_id");
create index if not exists "style_poll_votes_group_member_id_idx" on public."style_poll_votes" ("group_member_id");
create index if not exists "style_poll_votes_created_at_idx" on public."style_poll_votes" ("created_at");
create index if not exists "style_moodboards_group_id_idx" on public."style_moodboards" ("group_id");
create index if not exists "style_moodboards_theme_id_idx" on public."style_moodboards" ("theme_id");
create index if not exists "style_moodboards_status_idx" on public."style_moodboards" ("status");
create index if not exists "style_moodboards_created_by_idx" on public."style_moodboards" ("created_by");
create index if not exists "style_moodboards_created_at_idx" on public."style_moodboards" ("created_at");
create index if not exists "style_moodboards_updated_at_idx" on public."style_moodboards" ("updated_at");
create index if not exists "style_moodboard_items_moodboard_id_idx" on public."style_moodboard_items" ("moodboard_id");
create index if not exists "style_moodboard_items_group_member_id_idx" on public."style_moodboard_items" ("group_member_id");
create index if not exists "style_moodboard_items_product_id_idx" on public."style_moodboard_items" ("product_id");
create index if not exists "style_moodboard_items_variant_id_idx" on public."style_moodboard_items" ("variant_id");
create index if not exists "style_moodboard_items_created_at_idx" on public."style_moodboard_items" ("created_at");
create index if not exists "notifications_user_id_idx" on public."notifications" ("user_id");
create index if not exists "notifications_created_at_idx" on public."notifications" ("created_at");
create index if not exists "notification_deliveries_notification_id_idx" on public."notification_deliveries" ("notification_id");
create index if not exists "notification_deliveries_device_id_idx" on public."notification_deliveries" ("device_id");
create index if not exists "notification_deliveries_status_idx" on public."notification_deliveries" ("status");
create index if not exists "notification_deliveries_created_at_idx" on public."notification_deliveries" ("created_at");
create index if not exists "notification_preferences_user_id_idx" on public."notification_preferences" ("user_id");
create index if not exists "notification_preferences_updated_at_idx" on public."notification_preferences" ("updated_at");
create index if not exists "reviews_order_id_idx" on public."reviews" ("order_id");
create index if not exists "reviews_customer_id_idx" on public."reviews" ("customer_id");
create index if not exists "reviews_shop_id_idx" on public."reviews" ("shop_id");
create index if not exists "reviews_product_id_idx" on public."reviews" ("product_id");
create index if not exists "reviews_captain_id_idx" on public."reviews" ("captain_id");
create index if not exists "reviews_created_at_idx" on public."reviews" ("created_at");
create index if not exists "reviews_updated_at_idx" on public."reviews" ("updated_at");
create index if not exists "customer_events_customer_id_idx" on public."customer_events" ("customer_id");
create index if not exists "customer_events_shop_id_idx" on public."customer_events" ("shop_id");
create index if not exists "recommendation_sets_customer_id_idx" on public."recommendation_sets" ("customer_id");
create index if not exists "recommendation_items_recommendation_set_id_idx" on public."recommendation_items" ("recommendation_set_id");
create index if not exists "recommendation_items_product_id_idx" on public."recommendation_items" ("product_id");
create index if not exists "recommendation_items_variant_id_idx" on public."recommendation_items" ("variant_id");
create index if not exists "recommendation_items_created_at_idx" on public."recommendation_items" ("created_at");
create index if not exists "product_embeddings_product_id_idx" on public."product_embeddings" ("product_id");
create index if not exists "product_embeddings_updated_at_idx" on public."product_embeddings" ("updated_at");
create index if not exists "customer_embeddings_customer_id_idx" on public."customer_embeddings" ("customer_id");
create index if not exists "customer_embeddings_updated_at_idx" on public."customer_embeddings" ("updated_at");
create index if not exists "size_charts_shop_id_idx" on public."size_charts" ("shop_id");
create index if not exists "size_charts_category_id_idx" on public."size_charts" ("category_id");
create index if not exists "size_charts_created_at_idx" on public."size_charts" ("created_at");
create index if not exists "size_charts_updated_at_idx" on public."size_charts" ("updated_at");
create index if not exists "size_chart_measurements_size_chart_id_idx" on public."size_chart_measurements" ("size_chart_id");
create index if not exists "size_chart_measurements_created_at_idx" on public."size_chart_measurements" ("created_at");
create index if not exists "customer_body_profiles_customer_id_idx" on public."customer_body_profiles" ("customer_id");
create index if not exists "customer_body_profiles_created_at_idx" on public."customer_body_profiles" ("created_at");
create index if not exists "customer_body_profiles_updated_at_idx" on public."customer_body_profiles" ("updated_at");
```

```
create index if not exists "body_measurements_body_profile_id_idx" on public."body_measurements" ("body_profile_id");
create index if not exists "body_scan_sessions_customer_id_idx" on public."body_scan_sessions" ("customer_id");
create index if not exists "body_scan_sessions_body_profile_id_idx" on public."body_scan_sessions" ("body_profile_id");
create index if not exists "body_scan_sessions_status_idx" on public."body_scan_sessions" ("status");
create index if not exists "body_scan_sessions_created_at_idx" on public."body_scan_sessions" ("created_at");
create index if not exists "fit_recommendations_customer_id_idx" on public."fit_recommendations" ("customer_id");
create index if not exists "fit_recommendations_body_profile_id_idx" on public."fit_recommendations" ("body_profile_id");
create index if not exists "fit_recommendations_product_id_idx" on public."fit_recommendations" ("product_id");
create index if not exists "fit_recommendations_variant_id_idx" on public."fit_recommendations" ("variant_id");
create index if not exists "fit_recommendations_created_at_idx" on public."fit_recommendations" ("created_at");
create index if not exists "fit_feedback_customer_id_idx" on public."fit_feedback" ("customer_id");
create index if not exists "fit_feedback_order_item_id_idx" on public."fit_feedback" ("order_item_id");
create index if not exists "fit_feedback_fit_recommendation_id_idx" on public."fit_feedback" ("fit_recommendation_id");
create index if not exists "fit_feedback_created_at_idx" on public."fit_feedback" ("created_at");
create index if not exists "virtual_tryon_sessions_customer_id_idx" on public."virtual_tryon_sessions" ("customer_id");
create index if not exists "virtual_tryon_sessions_product_id_idx" on public."virtual_tryon_sessions" ("product_id");
create index if not exists "virtual_tryon_sessions_variant_id_idx" on public."virtual_tryon_sessions" ("variant_id");
create index if not exists "virtual_tryon_sessions_body_profile_id_idx" on public."virtual_tryon_sessions" ("body_profile_id");
create index if not exists "virtual_tryon_sessions_status_idx" on public."virtual_tryon_sessions" ("status");
create index if not exists "virtual_tryon_sessions_created_at_idx" on public."virtual_tryon_sessions" ("created_at");
create index if not exists "virtual_tryon_outputs_tryon_session_id_idx" on public."virtual_tryon_outputs" ("tryon_session_id");
create index if not exists "virtual_tryon_outputs_created_at_idx" on public."virtual_tryon_outputs" ("created_at");
create index if not exists "style_recommendation_sets_group_id_idx" on public."style_recommendation_sets" ("group_id");
create index if not exists "style_recommendation_sets_theme_id_idx" on public."style_recommendation_sets" ("theme_id");
create index if not exists "style_recommendation_sets_status_idx" on public."style_recommendation_sets" ("status");
create index if not exists "style_recommendation_sets_created_at_idx" on public."style_recommendation_sets" ("created_at");
create index if not exists "style_recommendation_items_recommendation_set_id_idx" on public."style_recommendation_items" ("recommendation_set_id");
create index if not exists "style_recommendation_items_group_member_id_idx" on public."style_recommendation_items" ("group_member_id");
create index if not exists "style_recommendation_items_product_id_idx" on public."style_recommendation_items" ("product_id");
create index if not exists "style_recommendation_items_variant_id_idx" on public."style_recommendation_items" ("variant_id");
create index if not exists "style_recommendation_items_created_at_idx" on public."style_recommendation_items" ("created_at");
create index if not exists "support_tickets_raised_by_user_id_idx" on public."support_tickets" ("raised_by_user_id");
create index if not exists "support_tickets_order_id_idx" on public."support_tickets" ("order_id");
create index if not exists "support_tickets_shop_id_idx" on public."support_tickets" ("shop_id");
create index if not exists "support_tickets_delivery_task_id_idx" on public."support_tickets" ("delivery_task_id");
create index if not exists "support_tickets_return_request_id_idx" on public."support_tickets" ("return_request_id");
create index if not exists "support_tickets_status_idx" on public."support_tickets" ("status");
create index if not exists "support_tickets_assigned_to_idx" on public."support_tickets" ("assigned_to");
create index if not exists "support_tickets_created_at_idx" on public."support_tickets" ("created_at");
create index if not exists "support_tickets_updated_at_idx" on public."support_tickets" ("updated_at");
create index if not exists "support_messages_ticket_id_idx" on public."support_messages" ("ticket_id");
create index if not exists "support_messages_sender_id_idx" on public."support_messages" ("sender_id");
create index if not exists "support_messages_created_at_idx" on public."support_messages" ("created_at");
create index if not exists "campaigns_status_idx" on public."campaigns" ("status");
create index if not exists "campaigns_created_by_idx" on public."campaigns" ("created_by");
create index if not exists "campaigns_approved_by_idx" on public."campaigns" ("approved_by");
create index if not exists "campaigns_created_at_idx" on public."campaigns" ("created_at");
create index if not exists "campaigns_updated_at_idx" on public."campaigns" ("updated_at");
create index if not exists "banners_campaign_id_idx" on public."banners" ("campaign_id");
create index if not exists "banners_created_at_idx" on public."banners" ("created_at");
create index if not exists "merchant_leads_assigned_executive_id_idx" on public."merchant_leads" ("assigned_executive_id");
create index if not exists "merchant_leads_created_at_idx" on public."merchant_leads" ("created_at");
create index if not exists "merchant_leads_updated_at_idx" on public."merchant_leads" ("updated_at");
create index if not exists "field_visits_merchant_lead_id_idx" on public."field_visits" ("merchant_lead_id");
create index if not exists "field_visits_shop_id_idx" on public."field_visits" ("shop_id");
create index if not exists "field_visits_executive_id_idx" on public."field_visits" ("executive_id");
create index if not exists "field_visits_status_idx" on public."field_visits" ("status");
create index if not exists "field_visits_created_at_idx" on public."field_visits" ("created_at");
create index if not exists "field_visits_updated_at_idx" on public."field_visits" ("updated_at");
create index if not exists "field_visit_checklist_items_field_visit_id_idx" on public."field_visit_checklist_items" ("field_visit_id");
create index if not exists "field_visit_checklist_items_completed_by_idx" on public."field_visit_checklist_items" ("completed_by");
create index if not exists "product_moderation_cases_product_id_idx" on public."product_moderation_cases" ("product_id");
create index if not exists "product_moderation_cases_status_idx" on public."product_moderation_cases" ("status");
create index if not exists "product_moderation_cases_moderator_id_idx" on public."product_moderation_cases" ("moderator_id");
create index if not exists "product_moderation_cases_created_at_idx" on public."product_moderation_cases" ("created_at");
create index if not exists "approval_requests_requested_by_idx" on public."approval_requests" ("requested_by");
create index if not exists "approval_requests_status_idx" on public."approval_requests" ("status");
create index if not exists "approval_requests_approved_by_idx" on public."approval_requests" ("approved_by");
create index if not exists "approval_requests_created_at_idx" on public."approval_requests" ("created_at");
create index if not exists "fraud_cases_subject_user_id_idx" on public."fraud_cases" ("subject_user_id");
create index if not exists "fraud_cases_shop_id_idx" on public."fraud_cases" ("shop_id");
create index if not exists "fraud_cases_order_id_idx" on public."fraud_cases" ("order_id");
create index if not exists "fraud_cases_status_idx" on public."fraud_cases" ("status");
create index if not exists "fraud_cases_assigned_to_idx" on public."fraud_cases" ("assigned_to");
create index if not exists "fraud_cases_created_at_idx" on public."fraud_cases" ("created_at");
create index if not exists "audit_logs_actor_user_id_idx" on public."audit_logs" ("actor_user_id");
create index if not exists "audit_logs_device_id_idx" on public."audit_logs" ("device_id");
create index if not exists "audit_logs_created_at_idx" on public."audit_logs" ("created_at");
```

```
create index if not exists "system_settings_updated_by_idx" on public."system_settings" ("updated_by");
create index if not exists "system_settings_updated_at_idx" on public."system_settings" ("updated_at");
create index if not exists "service_zones_city_idx" on public."service_zones" ("city");
create index if not exists "service_zones_created_at_idx" on public."service_zones" ("created_at");
create index if not exists "service_zones_updated_at_idx" on public."service_zones" ("updated_at");
create index if not exists "shops_location_gist" on public."shops" using gist ("location");
create index if not exists "service_zones_boundary_gist" on public."service_zones" using gist ("boundary");
create index if not exists "delivery_tasks_pickup_location_gist" on public."delivery_tasks" using gist ("pickup_location");
create index if not exists "delivery_tasks_drop_location_gist" on public."delivery_tasks" using gist ("drop_location");
create index if not exists "captain_current_locations_location_gist" on public."captain_current_locations" using gist ("location");
create index if not exists "captain_location_history_location_gist" on public."captain_location_history" using gist ("location");
create index if not exists "field_visits_start_location_gist" on public."field_visits" using gist ("start_location");
create index if not exists products_search_vector_gin on public.products using gin (search_vector);
create index if not exists products_style_tags_gin on public.products using gin (style_tags);
create index if not exists products_occasion_tags_gin on public.products using gin (occasion_tags);
create index if not exists products_name_trgm on public.products using gin (name gin_trgm_ops);
create index if not exists shops_name_trgm on public.shops using gin (name gin_trgm_ops);
create index if not exists inventory_available_lookup on public.inventory_balances (shop_id, variant_id, stock_on_hand, reserved_quantity, damaged_quantity);
create index if not exists active_orders_shop_status on public.orders (shop_id, status, created_at desc);
create index if not exists customer_orders_recent on public.orders (customer_id, created_at desc);
create index if not exists pending_alerts on public.merchant_order_alerts (shop_id, alert_status, expires_at);
create index if not exists dispatch_candidates on public.captain_profiles (availability_status) where availability_status = 'AVAILABLE';
create index if not exists customer_events_customer_time on public.customer_events (customer_id, occurred_at desc);
create index if not exists audit_logs_entity_time on public.audit_logs (entity_type, entity_id, created_at desc);

-- Storage buckets. Run from a privileged migration or create in Dashboard.
insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
values
  ('product-images','product-images',true,10485760,array['image/jpeg','image/png','image/webp']),
  ('shop-images','shop-images',true,10485760,array['image/jpeg','image/png','image/webp']),
  ('merchant-documents','merchant-documents',false,15728640,array['image/jpeg','image/png','application/pdf']),
  ('captain-documents','captain-documents',false,15728640,array['image/jpeg','image/png','application/pdf']),
  ('return-evidence','return-evidence',false,26214400,array['image/jpeg','image/png','image/webp','video/mp4']),
  ('body-scans','body-scans',false,26214400,array['image/jpeg','image/png','image/webp']),
  ('virtual-tryon','virtual-tryon',false,26214400,array['image/jpeg','image/png','image/webp']),
  ('support-attachments','support-attachments',false,26214400,array['image/jpeg','image/png','application/pdf','video/mp4'])
on conflict (id) do update set
  public = excluded.public,
  file_size_limit = excluded.file_size_limit,
  allowed_mime_types = excluded.allowed_mime_types;

-- Public read for catalogue media only. Uploads remain authenticated and path-scoped.
drop policy if exists "public product images read" on storage.objects;
create policy "public product images read" on storage.objects for select to anon, authenticated
using (bucket_id in ('product-images','shop-images'));

drop policy if exists "authenticated product media upload" on storage.objects;
create policy "authenticated product media upload" on storage.objects for insert to authenticated
with check (
  bucket_id in ('product-images','shop-images')
  and (storage.foldername(name))[1] = (select auth.uid())::text
);

drop policy if exists "owner manages uploaded product media" on storage.objects;
create policy "owner manages uploaded product media" on storage.objects for update to authenticated
using (owner_id = (select auth.uid())::text)
with check (owner_id = (select auth.uid())::text);

drop policy if exists "owner deletes uploaded product media" on storage.objects;
create policy "owner deletes uploaded product media" on storage.objects for delete to authenticated
using (owner_id = (select auth.uid())::text);

-- Private uploads are stored under the authenticated user's first folder segment.
drop policy if exists "private bucket owner upload" on storage.objects;
create policy "private bucket owner upload" on storage.objects for insert to authenticated
with check (
  bucket_id in ('merchant-documents','captain-documents','return-evidence','body-scans','virtual-tryon','support-attachments')
  and (storage.foldername(name))[1] = (select auth.uid())::text
);

drop policy if exists "private bucket owner read" on storage.objects;
create policy "private bucket owner read" on storage.objects for select to authenticated
using (
  bucket_id in ('merchant-documents','captain-documents','return-evidence','body-scans','virtual-tryon','support-attachments')
  and owner_id = (select auth.uid())::text
);

drop policy if exists "private bucket owner delete" on storage.objects;
```

```
create policy "private bucket owner delete" on storage.objects for delete to authenticated
using (
  bucket_id in ('merchant-documents','captain-documents','return-evidence','body-scans','virtual-tryon','support-attachments')
  and owner_id = (select auth.uid())::text
);

-- Realtime publication. Foreground UI subscribes to these tables; background ringing still needs FCM/APNs.
do $$
begin
  alter publication supabase_realtime add table public.orders;
exception when duplicate_object then null;
end $$;
do $$ begin alter publication supabase_realtime add table public.order_status_history; exception when duplicate_object then null; end $$;
do $$ begin alter publication supabase_realtime add table public.merchant_order_alerts; exception when duplicate_object then null; end $$;
do $$ begin alter publication supabase_realtime add table public.delivery_tasks; exception when duplicate_object then null; end $$;
do $$ begin alter publication supabase_realtime add table public.delivery_events; exception when duplicate_object then null; end $$;
do $$ begin alter publication supabase_realtime add table public.notifications; exception when duplicate_object then null; end $$;
do $$ begin alter publication supabase_realtime add table public.support_messages; exception when duplicate_object then null; end $$;
```

seed.sql

```
-- Vastra Supabase schema starter
-- Generated for PostgreSQL/Supabase. Review in local and staging environments before production.
-- All money values are BIGINT paise; timestamps are TIMESTAMPTZ; geospatial fields use PostGIS.


-- Seed roles, permissions, root categories and initial settings.

insert into public.roles(code,name,description,is_system_role) values ('SUPER_ADMIN','Super Admin','Super Admin role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('CITY_OPERATIONS','City Operations','City Operations role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('ORDER_OPERATIONS','Order Operations','Order Operations role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('MERCHANT_SUPPORT','Merchant Support','Merchant Support role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('MERCHANT_ONBOARDING','Merchant Onboarding','Merchant Onboarding role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('CATALOGUE_MODERATOR','Catalogue Moderator','Catalogue Moderator role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('DELIVERY_OPERATIONS','Delivery Operations','Delivery Operations role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('CUSTOMER_SUPPORT','Customer Support','Customer Support role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('FINANCE','Finance','Finance role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('MARKETING','Marketing','Marketing role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('FRAUD_RISK','Fraud & Risk','Fraud & Risk role',true) on conflict (code) do nothing;
insert into public.roles(code,name,description,is_system_role) values ('ANALYTICS_VIEWER','Analytics Viewer','Analytics Viewer role',true) on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('platform.read','Platform','platform.read','Read operational data') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('platform.write','Platform','platform.write','Perform allowed administrative changes') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('merchant.kyc.review','Merchant','merchant.kyc.review','Review merchant KYC') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('merchant.approve','Merchant','merchant.approve','Approve merchant') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('order.reassign_captain','Orders','order.reassign_captain','Reassign captain') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('refund.create','Finance','refund.create','Create refund') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('refund.approve_large','Finance','refund.approve_large','Approve large refund') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('settlement.adjust','Finance','settlement.adjust','Adjust settlement') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('campaign.publish','Marketing','campaign.publish','Publish campaign') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('account.suspend','Risk','account.suspend','Suspend account') on conflict (code) do nothing;
insert into public.permissions(code,module,name,description) values ('audit.view','Governance','audit.view','View audit logs') on conflict (code) do nothing;
insert into public.role_permissions(role_id, permission_id) select r.id,p.id from public.roles r cross join public.permissions p where r.code='SUPER_ADMIN' on conflict do nothing;
insert into public.role_permissions(role_id, permission_id) select r.id,p.id from public.roles r join public.permissions p on p.code='platform.read' where r.code='ANALYTICS_VIEWER' on conflict do
nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Clothing','clothing',1,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Footwear','footwear',2,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Accessories','accessories',3,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Men','men',4,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Women','women',5,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Kids','kids',6,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Belts','belts',7,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Watches','watches',8,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Bags','bags',9,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Jewellery','jewellery',10,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Pocket Squares','pocket-squares',11,true) on conflict (slug) do nothing;
insert into public.categories(name,slug,display_order,is_active) values ('Handkerchiefs','handkerchiefs',12,true) on conflict (slug) do nothing;
insert into public.system_settings(setting_key,setting_value,value_type,scope_type,scope_id,updated_by) select 'merchant_order_response_seconds','120'::jsonb,'NUMBER','GLOBAL',null,id from
public.profiles where account_type='ADMIN' order by created_at limit 1 on conflict do nothing;
```